



UNIVERSIDAD LATINA DE PANAMÁ

FACULTAD DE INGENIERÍA

ELABORACIÓN DE MANUAL DE VIDEOJUEGO

MOTOR UNREAL ENGINE 4

Proyecto final de graduación presentado como requisito para optar por la
Licenciatura en Ingeniería de Sistemas Informáticos en la Universidad Latina de
Panamá

Rubén Eduardo Villasmil Molina

Pasaporte: 101399843

Profesor asesor:

Carlos Alberto Fernández Valdés

Panamá, República de Panamá

2018

DEDICATORIA

A mi familia, en especial mis padres, hermanas, abuelo, tíos, tías y primos por los buenos momentos y su influencia en mi crecimiento como persona.

Al lector dispuesto a aprender de este manual.

A mis maestros, en especial Sissy y Evelin que me ayudaron en la mejora no solo académica sino en la social que no podría llegar a enfrentar el mundo sin ellas.

A la decana Dra. Belkis Lara, por su guía y asistencia para hacer posible el desarrollo de la tesis, y al profesor Carlos Fernández por su ayuda y observaciones en el desarrollo de la tesis.

AGRADECIMIENTOS

Agradezco a la Universidad Latina de Panamá, por su recibimiento y oportunidad de estudiar para lograr la culminación de mi formación académica, y alcanzar la meta de mi título profesional, a pesar de las adversidades.

A la Facultad de Ingeniería y sus profesores, por la oportunidad de probarme tanto en las clases, como en los eventos que me ayudarán a lo largo de mi carrera profesional.

A mi tutor Carlos Fernández, por el apoyo en los proyectos personales planteados y presentados en la universidad, así como su asistencia en dudas y complicaciones confrontadas en la universidad.

A la decana Belkis Lara, por su apoyo en la presentación de los proyectos y su asistencia en el desarrollo de la tesis así como también, de las fuentes de referencia que me ayudaron a fortalecer mi documentación.

A mi familia, por siempre estar ahí cuando los necesitaba y su apoyo incondicional.

ÍNDICE GENERAL

	Página
AGRADECIMIENTO.....	i
DEDICATORIA.....	ii
DECLARACIÓN JURADA.....	iii
ÍNDICE GENERAL.....	iv
ÍNDICE DE TABLAS.....	vii
ÍNDICE DE ILUSTRACIONES.....	viii
ÍNDICE DE GRÁFICOS.....	ix
INTRODUCCIÓN.....	1
CAPÍTULO 1.0 EL PROBLEMA.....	3
1.1 ANTECEDENTES DEL PROBLEMA.....	3
1.2 PLANTEAMIENTO DEL PROBLEMA.....	4
1.3 JUSTIFICACIÓN DE LA INVESTIGACIÓN.....	5
1.4 OBJETIVOS.....	5
1.4.1 OBJETIVOS GENERALES.....	5
1.4.2 OBJETIVOS ESPECÍFICOS.....	5
1.5 ALCANCE Y LÍMITES DE LA INVESTIGACIÓN.....	5
1.5.1 ALCANCE.....	6
1.5.2 LÍMITES.....	6
1.5.3 LÍNEA DE INVESTIGACIÓN A LA QUE PERTENECE EL GRUPO.....	7

	Página
CAPÍTULO 2 MARCO TEÓRICO.....	8
2.1 MARCO REFERENCIAL.....	9
2.2 GLOSARIO DE TÉRMINOS.....	10
2.3 ANTECEDENTES.....	16
CAPÍTULO 3.0 METODOLOGÍA.....	19
3.1 TIPO Y DISEÑO DE LA INVESTIGACIÓN.....	21
3.2 POBLACIÓN Y/O MUESTRA.....	22
3.2.1 CÁLCULO DEL MUESTREO.....	22
3.3 CASO DE ESTUDIO.....	22
3.3.1 COMPONENTES DEL MANUAL.....	22
3.3.2 PRUEBAS DE EJECUCIÓN.....	23
3.3.3 LIMITACIONES DE EJECUCIÓN.....	24
3.3.4 PRUEBAS DE EXPORTACIÓN E IMPORTACIÓN DE ACTIVOS.	24
3.3.5 LIMITACIONES DE EXPORTACIÓN E IMPORTACIÓN DE ACTIVOS.....	25
CAPÍTULO 4.0 ANÁLISIS E INTERPRETACIÓN DE LOS RESULTADOS.....	30
4.1 ANÁLISIS E INTERPRETACIÓN DE LOS RESULTADOS.....	31
CAPÍTULO 5.0 PROPUESTA DE LA TESIS.....	34
5.1 INTRODUCCIÓN DE LA PROPUESTA.....	35
5.2 JUSTIFICACIÓN DE LA PROPUESTA.....	35
5.3 OBJETIVOS DE LA PROPUESTA.....	37

	Página
5.4 METAS A ALCANZAR.....	38
5.5 BENEFICIOS DE LA PROPUESTA.....	39
5.6 CRONOGRAMA DE LA PROPUESTA.....	40
5.7 PRESUPUESTO DE LA PROPUESTA.....	40
5.8 DISEÑO DE LA PROPUESTA.....	40
ANEXOS.....	41
BIBLIOGRAFÍA.....	166

ÍNDICE DE TABLAS

Tabla N°	Título	Página
1	Ventajas y desventajas del manual.....	14
2	Ventajas y desventajas de Unreal Engine 4.....	15
3	Fases de desarrollo del proyecto de investigación.....	28

ÍNDICE DE ILUSTRACIONES

Figura N°	Título	Página
1	Representación gráfica de nodos interconectados.....	12
2	<i>Blueprint</i> usado para movilizar un vehículo en base al <i>joystick</i> .	12
3	Ejemplo de varios activos con enfoque solo en modelos 3D.....	13
4	Foto de Tim Sweeney fundador de Epic Games y miembro principal del equipo de programación de Unreal Engine 4.....	17
5	Captura del proyecto en ejecución.....	23
6	Lista de formatos multimedia que pueden ser exportados en Unreal Engine 4 parte 1.....	25
7	Lista de formatos multimedia que pueden ser exportados en Unreal Engine 4 parte 2.....	26
8	Lista de formatos multimedia que pueden ser exportados en Unreal Engine 4 parte 3.....	26
9	Lista de formatos multimedia que pueden ser importados desde Unreal Engine 4.....	27
10	Cronograma del desarrollo del manual.....	39

ÍNDICE DE GRÁFICOS

Gráfica N°	Título	Página
1	Encuesta estudiante 1.....	31
2	Encuesta estudiante 2.....	32
3	Encuesta estudiante 3.....	32
4	Encuesta estudiante 4.....	33
5	Encuesta estudiante 5.....	33

INTRODUCCIÓN

Los videojuegos son un medio de entretenimiento electrónico, en el que uno o más usuarios interactúan para completar objetivos o retos a través de dispositivos periféricos; generalmente denominados controles, y vistos a través de uno o variados dispositivos que despliegan las imágenes de video.

La producción del primer videojuego digital hasta la fecha, fue con el popular Pong de la empresa Atari, el cual inicio un nuevo mercado de entretenimiento, que ha evolucionado exponencialmente a no solo en la calidad del *hardware* y *software* de las variadas plataformas que procesan los juegos; sino también, a nivel de complejidad de los controles, funciones y componentes de jugabilidad e historias que las integraban, que hasta las empresas pioneras de muchas sagas famosas se han adaptado en la actualidad que su estructura para hacer sus productos, han crecido en diferentes divisiones para hacerlos un éxito comercial, que ahora cuentan con equipos de diseño, animación, programación y hasta mercadeo para garantizar ganancias con su salida.

Debido a la complejidad que abarca la programación tradicional por líneas de código de grandes proyectos, como puede ser un videojuego con la disposición de varios motores de videojuegos, se notó la falta de conocimiento del uso por parte de los estudiantes de la Universidad Latina de Panamá; por lo que se elaborará un manual que abarque la teoría, el planteamiento, la lógica detrás de los videojuegos y la aplicación de lo aprendido, adaptado a la programación del motor Unreal Engine 4, para desarrollar destrezas en la elaboración de los mismos, con mayor eficacia y velocidad al ponerlas en práctica en distintos proyectos; los cuales ayudan a adquirir

la experiencia, habilidad y creatividad que pueden impulsar a los interesados a proyectos más grandes y complejos.

No existe investigación documentada que se relacione con el motor Unreal Engine 4 en la Universidad Latina de Panamá, y al momento, no se puede determinar si existe o no otras investigaciones de este u otros motores en el ámbito nacional.

A partir de la realización de esta investigación existe la posibilidad de crear un manual que contribuya en el desarrollo académico y profesional de los estudiantes; ya sean de sistemas, animación o de otras carreras interesadas en la culminación de proyectos relacionados con videojuegos o cinemáticas, con una calidad que satisface una amplia gama de necesidades, así como exigencias en nuestra actualidad.

A partir de la investigación se elaborará un manual que asista a los jóvenes estudiantes de la Universidad Latina de Panamá, que posibilitará la creación de proyectos con mayor efectividad de aprendizaje, amplias posibilidades de desarrollo a crear contenido original, e inclusive la adición de efectos como otras características estéticas por la que destaca el motor en comparación con los otros.

Este trabajo es capaz de dar pie a otras investigaciones de la misma área y que fomente una base en futuras investigaciones y proyectos relacionados.

CAPÍTULO 1.0

EL PROBLEMA

1.1 ANTECEDENTES DEL PROBLEMA

Elaborar un videojuego es una tarea extensa y complicada, que no solo abarca el planteamiento creativo y estético, sino también la lógica aplicada en matemáticas y programación, que ya por el lado de programación abarca mucho tiempo por líneas de códigos, los ensayos y errores que se deben corregir, cuando no funciona como se desea; también el tiempo, la dificultad aumenta al anexar más elementos en el proyecto del videojuego, que hace difícil su culminación total con un equipo de desarrollo completo, ni hablar de una sola persona.

No fue sino por la creación de los motores de videojuegos, que desarrolladores compactaron la codificación con elementos, funciones y procedimientos de tal forma que al programar videojuegos ahorrarán tiempo, ensayo y con ello, dinero en hacer y culminar sus proyectos con los resultados esperados; debido a eso, distintas empresas tanto profesionales, como las pequeñas y hasta los desarrolladores independientes, programaron y configuraron sus propios motores para uso exclusivamente propio.

Hasta la fecha, con la salida del motor de videojuegos Unreal Engine 4, fue inicialmente de pago y hasta exclusivo como muchos otros motores disponibles, y no fue sino hasta hace unos años que la empresa que la desarrolló Epic Games, cambió su política del motor, y la hizo gratis para los usuarios interesados como pasatiempo o estudiantes que la descargaran y con la condición de que en caso de vender y generar ingresos superiores a los tres mil (3.000) dólares, a la empresa le correspondía el 5 por ciento de esas ganancias, que empezaron no solo grandes

desarrolladoras de videojuegos, sino todo aquel que haya logrado hacer y poner a la venta su videojuego (Sweeney, 2015).

1.2 PLANTEAMIENTO DEL PROBLEMA

¿Cómo elaborar un manual o guía, que mejore la producción de videojuegos de los estudiantes de la Universidad Latina de Panamá?

1.3 JUSTIFICACIÓN DE LA INVESTIGACIÓN

Esta investigación se realizará debido a que, en la actualidad existe un alto número de estudiantes en la Universidad Latina de Panamá, que desconocen el programa Unreal Engine 4, a nivel teórico y práctico; así como también, la utilidad del mismo. Su aporte será de gran utilidad para las diferentes escuelas de la institución, y los estudiantes que muestren interés en elaborar este tipo de proyectos, se les facilitará la comprensión del motor y sus componentes.

1.4 OBJETIVOS

1.4.1 OBJETIVOS GENERALES

Desarrollar un Manual de videojuego del motor Unreal Engine 4, enfocado a mejorar la producción de videojuegos de los estudiantes de la Universidad Latina de Panamá.

1.4.2 OBJETIVOS ESPECÍFICOS

- Determinar los procedimientos de gestión de la plataforma de videojuegos Unreal Engine 4.
- Establecer los componentes, y el uso de Unreal Engine 4, en elementos claves para desarrollar un videojuego.
- Diseñar los procedimientos que componen el manual del motor de videojuego.

- Brindar material de capacitación para futuros proyectos relacionados con el área de videojuegos, con un nivel intermedio en programación y matemáticas.

1.5 ALCANCE Y LÍMITES DE LA INVESTIGACIÓN

1.5.1 ALCANCE

Este proyecto permitirá establecer los elementos que debe contener el manual o guía; y al mismo tiempo, contendrá información general sobre el tema, el cual se espera sea de utilidad para los estudiantes de la Universidad Latina de Panamá; sin embargo, puede ser utilizado por otros estudiantes de otras universidades nacionales, o personas que se interesen en el tema.

1.5.2 LÍMITES

La falta de información de los estudiantes de la Universidad Latina de Panamá, con respecto al uso de motores en la producción de videojuegos, complica la aplicación de estos y la creación de proyectos, por lo que se emplea más tiempo y esfuerzo en su entendimiento.

El tiempo necesario para resumir y hacer entendible los elementos del motor en el manual; de tal forma que lo pueda entender una amplia gama de estudiantes de la Universidad Latina de Panamá.

Tener a disposición un equipo con especificaciones a nivel de *hardware* y *software*, que cumpla con las exigencias mínimas para la ejecución del programa; así como su desempeño de procesamiento, no solo de la lógica de programación, sino también del procesado estético, abarcando desde los materiales, las partículas y hasta la iluminación dinámica siendo pocos los que pueden ejecutarla.

La posible falta de cooperación de los encuestados, al momento de responder las preguntas que se establecerán en esta investigación.

1.5.3 LÍNEA DE INVESTIGACIÓN A LA QUE PERTENECE EL ESTUDIO

Línea de Investigación en Ingeniería, Sistemas Informáticos.

CAPÍTULO 2.0

MARCO TEÓRICO

2.1 MARCO REFERENCIAL

Luego de una exhaustiva búsqueda en la biblioteca física y virtual de la Universidad Latina de Panamá, no se ha encontrado material que comparta algún parentesco con el proyecto de investigación desarrollado específicamente en el motor Unreal Engine 4.

Luego de proceder a buscar por internet se han encontrado múltiples bibliografías tanto en inglés como en español, provenientes de España, Perú, Bolivia e inclusive Finlandia, también variadas filmotecas que explican a nivel práctico el uso del programa y sus funciones, que al compararse cada uno de estos elementos se ha determinado temas en la que comparten similitudes.

Gracias a estos exámenes se ha podido conocer los puntos fundamentales y claves para desarrollar el proyecto de investigación, con respecto al motor Unreal Engine 4 y el diseño de un manual basado en este.

En las investigaciones del material bibliográfico, luego de ser observado todo su contenido y haber sido analizado todo lo referente a videojuegos, se ha identificado un enorme potencial en el desarrollo de proyectos en este programa; más allá de la creación de videojuegos, existen amplios usos de este en la animación, modelado, estructuras, simulaciones y muchas otras ideas que pueden ser plasmadas y desarrolladas con este.

También con, se cuenta con los términos y condiciones de plataforma gratuita, por desarrollo personal o educativo, sus funciones permiten a los interesados usarlo sin preocuparse por cuotas mensuales, a diferencia de otros motores; en especial en países, cuya moneda es mucho más débil que el dólar estadounidense.

Surgen posibilidades de desarrollo en variados proyectos por estudiantes o profesionales de distintas carreras con aportes recreativos, investigativos, etc.

2.2 GLOSARIO DE TÉRMINOS

API. Interfaz de Programa de Aplicaciones (Application Programming Interface) (API), es un conjunto de subrutinas, funciones y procedimientos de programación orientada a objetos que ofrecen ciertas bibliotecas para ser utilizado por otro *software* como una capa de abstracción usada generalmente en las bibliotecas de programación.

Assets. Traducido como activos son los elementos que serán introducidos al videojuego. Estos elementos incluyen Modelos 3D, personajes, texturas, materiales, animaciones, *scripts*, sonidos, y algunos elementos específicos de cada motor. Cada motor trabaja de una manera distinta a otros lo cual puede aceptar activos que otros motores no pueden manejar, cuando son importados en un formato predeterminado es convertido a uno que el motor pueda reconocer y manejar fácilmente y en ciertos motores puede importarse en formatos variados.

Blueprints. Es un sistema de *script* visual de juego completo, usando una interfaz basada en nodos para crear juegos dentro del editor, usado para definir clases orientadas a objetos u objetos en el editor, a medida que se use UE4 se encontraran objetos definidos como planos que serán denominados coloquialmente como *blueprints*.

CPU. Unidad Central de Procesamiento (*Central Processing Unit*) (CPU), *hardware* dentro de un ordenador u otros dispositivos programables, que interpreta las instrucciones de un programa informático mediante la realización de las operaciones básicas aritméticas, lógicas y de entrada/salida del sistema.

DirectX. Colección de API, desarrolladas para facilitar las complejas tareas relacionadas con multimedia, especialmente programación de juegos y vídeos en la plataforma Microsoft Windows.

GPU. Unidad de Procesamiento Gráfico (*Graphics Processing Unit*) (GPU), coprocesador dedicado al procesamiento de gráficos u operaciones de coma flotante, para aligerar la carga de trabajo del procesador central en aplicaciones como los videojuegos o aplicaciones 3D interactivas. De esta forma, mientras gran parte de lo relacionado con los gráficos se procesa en la GPU, la unidad central de procesamiento (CPU) puede dedicarse a otro tipo de cálculos (como la inteligencia artificial o los cálculos mecánicos en el caso de los videojuegos).

Hardware. Son los componentes físicos de la computadora (pantalla, teclado, etc.).

Nodo. Es un punto de intersección, conexión o unión de varios elementos que confluyen en el mismo lugar.

S.O. Sistema Operativo (Operating System) (O.S), *software* principal o conjunto de programas de un sistema informático, que gestiona los recursos del *hardware* y provee servicios a los programas de aplicación de *software*; ejecutándose en modo privilegiado, respecto de los restantes (aunque puede que parte de él se ejecute en un espacio de usuario).

RAM. Memoria de Acceso Aleatorio (Random Access Memory) (RAM), memoria de trabajo de computadoras para el sistema operativo, los programas y la mayor parte del *software*.

Scripts. Es un lenguaje de programación que ejecuta diversas funciones en el interior de un programa de computador, el lenguaje puede variar.

SDK. Kit de Desarrollo de *Software* (*Software Development Kit*) (SDK), un conjunto de herramientas que le permite al programador o desarrollador de *software* crear una aplicación informática para un sistema concreto, por ejemplo *frameworks*, ciertos paquete de *software*, plataformas de *hardware*, computadoras, videoconsolas, sistemas operativos, etc.

Software. Son los programas que realizan tareas específicas en la computadora (explorador de archivos, internet, etc.).

UE4. Unreal Engine 4 (UE4), motor de videojuegos y animación creada por la empresa Epic Games.

VRAM. Memoria Gráfica de Acceso Aleatorio (*Video Random Access Memories*) (VRAM), es una memoria RAM que maneja toda la información visual que le manda la CPU del sistema.

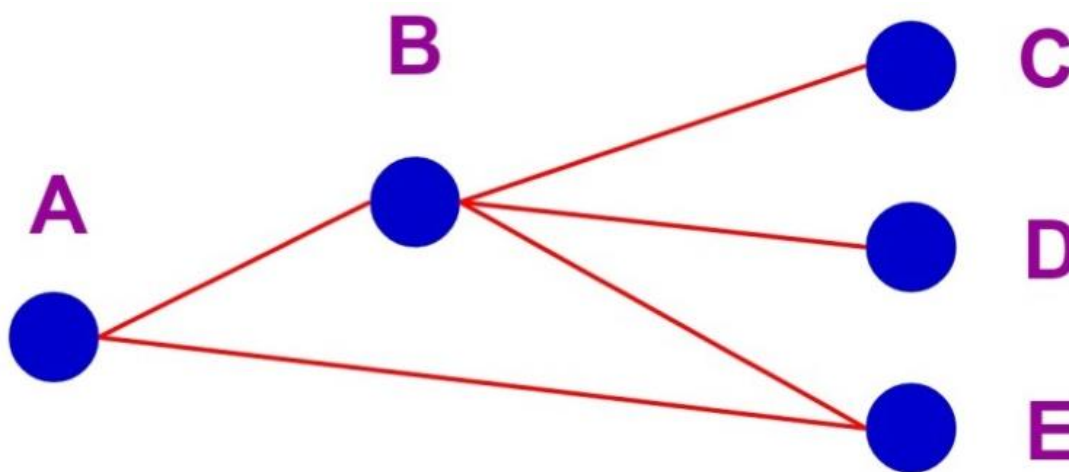


Figura 1. Representación gráfica de nodos interconectados

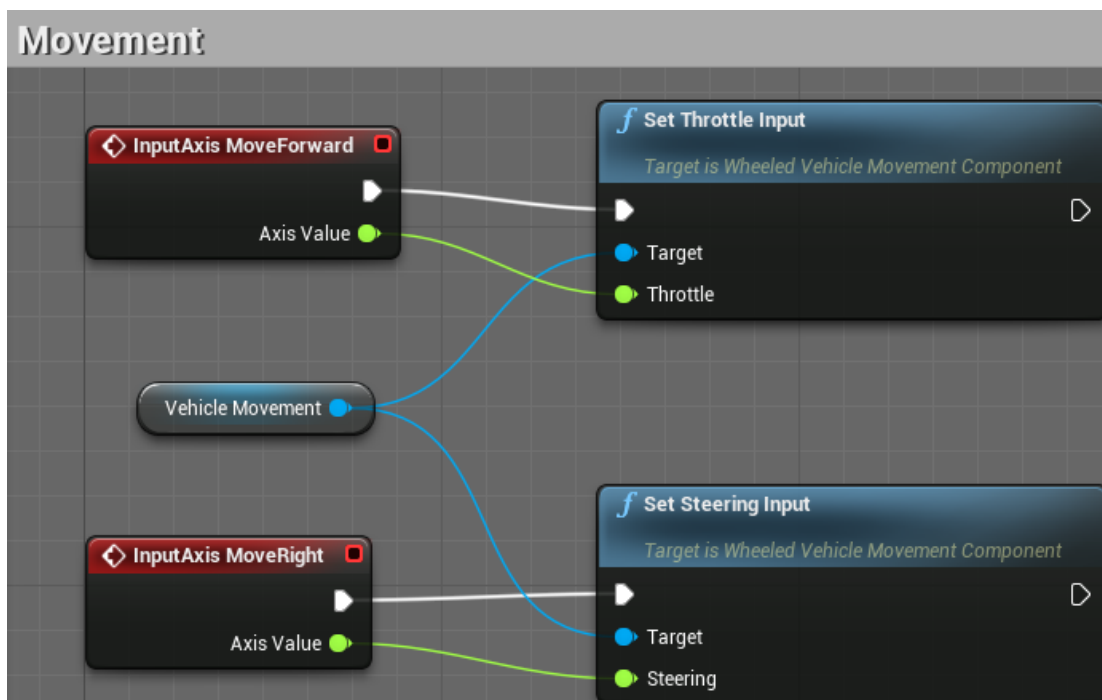


Figura 2. *Blueprint* usado para movilizar un vehículo en base al joystick



Figura 3. Ejemplo de varios activos con enfoque solo en modelos 3D

Las ventajas y desventajas de usar el manual se mencionan en la tabla 1.

Tabla 1. Ventajas y desventajas del manual

Ventajas	Desventajas
• Ayuda a los lectores a comprender mejor el motor, funciones, aplicaciones y programación. • Dispondrá de manera detallada los pasos para desarrollar los elementos básicos y esenciales que tienen la mayoría de los videojuegos. • Explica teóricamente las bases fundamentales de los videojuegos. • Explica paso a paso la lógica de los videojuegos y su aplicación práctica en la programación por nodos.	• Se debe tener un conocimiento intermedio de programación. • Se necesita un equipo de gama media-alta que cumpla con los requisitos para ejecutar el motor. • Falta de contenido en el manual por considerarse no esencial. • Posibles problemas de comprensión por parte de lectores con entusiasmo pero carentes de los conocimientos de programación y lógica requeridas.

Las ventajas y desventajas de usar el motor Unreal Engine 4 se mencionan en la tabla 2.

Tabla 2. Ventajas y desventajas de Unreal Engine 4

Ventajas	Desventajas
• Unreal Engine 4 está lista para generar diferentes géneros de videojuegos en plantillas predeterminadas. • Al momento de crear el juego ofrece las plataformas vigentes actuales (Xbox One, PS4, Switch, PC, Android, etc). • El uso del motor para proyectos personales o de estudios es gratis. • Su programación por nodos ayuda a agilizar el desarrollo de los elementos del juego así como las revisiones de errores y ediciones. • Existe una amplia gama de documentaciones y foros de grupos con respecto a problemas o planteamientos de desarrollo de proyectos.	• Los proyectos una vez desarrollados pueden llegar a ocupar 1.5GB solo con los elementos predeterminados. • Al momento de generar el juego para la plataforma específica dependiendo del equipo y el contenido que abarca el proyecto puede tomar varias horas la compilación y generación del mismo. • Se necesita de una computadora o <i>laptop</i> de gama media-alta para poder usar el programa. • Se requiere un conocimiento intermedio de programación y lógica para poder desarrollar y editar el proyecto.

2.3 ANTECEDENTES

En la actualidad, la industria de los videojuegos ha tenido un auge sin precedentes compitiendo con la del cine, la televisión y hasta la música, con un crecimiento exponencial debido a las nuevas tecnologías; así como los motores de videojuegos que permiten no solo a empresas AAA, a generar juegos de calidad y exitosos; sino también a que desarrolladores independientes lancen sus proyectos a pesar de no contar con la fama o una amplia financiación como los otros desarrolladores, y estos han incrementado desde que la empresa Epic Games cambió su política con su motor Unreal Engine 4, ofreciéndolo gratis con todas sus funciones como una estrategia comercial agresiva contra sus competidores (Sweeney, 2015); declaró que el estado de Unreal es fuerte y se han dado cuenta de que a medida que eliminan barreras, más personas pueden cumplir sus visiones creativas y dar forma al futuro del medio que aman.

Se eliminó la última barrera para entrar al precio de la cuota mensual por la licencia, y ahora es gratis. Gracias a esto se ha considerado trabajar en este motor, pues en el caso de muchos otros motores, sus versiones gratuitas presentan funciones limitadas, y sus versiones completas se deben pagar en cuotas mensuales haciéndolo inaccesible para estudiantes o desarrolladores pequeños; en especial, en países extranjeros donde la moneda puede llegar a ser más débil que el dólar americano, haciendo aún más complicado pagar la mensualidad.



Figura 4. Foto de Tim Sweeney fundador de Epic Games y miembro principal del equipo de programación de Unreal Engine 4

Luego de la selección del motor, se procedió a buscar bibliografías que compartan semejanzas con los objetivos planteados en el desarrollo del manual y según (Festini Wendorff & Torres Ferreyros, 2017), se desarrollaron para variadas plataformas, distintos juegos con enfoques psicológicos y sociales; también mencionan los pasos para el desarrollo de un juego de producción de entretenimiento así como la creación de su proyecto denominado “Vermillion”; que es la base para desarrollar los capítulos con los pasos de creación del juego en el manual, con algunas variaciones y enfoque en lo más vital, para evitar demasiada y tediosa la lectura; y lograr, la aplicación inmediata de las prácticas de los pasos del manual.

En cuanto a lo teórico (López, 2017), desarrolló también un manual para el uso de Unreal Engine 4, en el cual anexó desde la comparativa de los motores actuales, hasta el desarrollo de proyectos, y con base en su trabajo se anexó en la teoría la historia de los videojuegos hasta sus géneros, siendo importante entender el origen de lo que es hoy una industria con fuerte influencia internacional, abarcando el

origen de las grandes desarrolladoras de videojuegos, genios de las épocas; así como el lanzamiento de grandes éxitos vigentes hasta la actualidad. También destacar que, el desarrollo y programación cambian la dificultad según el género que escoja el lector, pues algunos tienen funciones y lógicas más complejas que las otras.

CAPÍTULO 3.0

METODOLOGÍA

3.1 TIPO Y DISEÑO DE LA INVESTIGACIÓN

Este trabajo es exploratorio porque profundiza en el desarrollo teórico y práctico de videojuegos, además es descriptivo porque como un manual tiene la finalidad de explicar el motor de videojuegos Unreal Engine 4, sus características y cómo aplicarlo en la creación de videojuegos, que es lo más complejo de desarrollar y con base en estos tener las herramientas de aprendizaje para expandir el desarrollo de varios proyectos, además de los videojuegos, como simulación, animación, entre muchos otros.

El desarrollo del manual se basó en tres aspectos: La documentación del motor con sus componentes, herramientas, funciones e interfaz, los pasos para usar los anteriormente mencionados en el desarrollo de los elementos que componen cualquier videojuego y el contenido teórico de los videojuegos abarcando desde los orígenes de estos, su rama de géneros y la explicación de la lógica de programación que usa el motor para realizar los proyectos.

Fue difícil la búsqueda de los elementos más esenciales para interpretarlos de una manera entendible para los usuarios con conocimiento intermedio de programación; también la explicación de la lógica aplicada en los videojuegos con la adición de ejemplos de otros juegos existentes, algunos desarrollados con el motor Unreal Engine

El manual está conformado por seis (6) capítulos. (El primer capítulo abarca los videojuegos desde su historia hasta los géneros existentes en la actualidad, el segundo explica los requisitos y los pasos para instalar el programa en una computadora con sistema operativo Windows o Mac, el tercer capítulo abarca la historia de Unreal Engine y las características de la interfaz de la versión 4 en la que

se basa el manual de desarrollo de videojuegos, el cuarto capítulo abarca la programación por nodos por la que el motor ha ganado popularidad logrando el desarrollo de videojuegos de una manera mucho más rápida y eficaz en comparación con la programación tradicional a través de líneas de código, el quinto capítulo explica el desarrollo de un juego a partir de la selección de una de las plantillas de juego predeterminado por el sistema, también se menciona la creación, adición y modificación de los elementos que van a componer el juego como tal ,incluyendo no solo modelos 3D, sino también los materiales estéticos e inclusive la función por la que el motor ha sido aclamado, su sistema de partículas en el que se realizan variados efectos visuales para mejorar a nivel estético el juego desarrollado.

El sexto y último capítulo del manual mencionara los requisitos y los pasos para la configuración y generación del ejecutable del videojuego para su ejecución en dispositivos desde los móviles (Android, IOS y Windows) a las plataformas de computación y consolas vigentes a la fecha (IOS, Windows, Linux, PS4, Xbox One e inclusive Nintendo Switch).

3.2 POBLACIÓN Y/O MUESTRA

Este tema se investigó a los estudiantes de ingeniería informática de la Universidad Latina de Panamá con un conocimiento intermedio-alto en programación. La población seleccionada fue de diez (10) estudiantes actualmente inscritos en el cuatrimestre 2018-3 en la Licenciatura de Ingeniera en Sistemas Informáticos.

3.2.1 CÁLCULO DEL MUESTREO

Debido al número limitado de estudiantes de la Universidad Latina de Panamá, con conocimiento intermedio-alto del contenido de la carrera que están actualmente inscritos en el cuatrimestre 2018-3 en la Licenciatura de Ingeniera en Sistemas Informáticos, no existe un amplio número de variables para realizar el cálculo de la muestra.

3.3 CASO DE ESTUDIO

El punto de partida de este trabajo fue el diseño del manual para el uso del motor Unreal Engine 4; debido a la ausencia de manuales de este motor en la universidad se optó por la búsqueda de bibliografías extranjeras por internet que coincidieran en la investigación, encontrándose una amplia variedad proveniente de Perú y Argentina, pero gran parte del material obtenido proviene de Europa, principalmente España, y de esta forma estará a la disposición de alumnos interesados en el desarrollo de proyectos a través de él.

En el diseño se redactaron todos los temas considerados esenciales a nivel teórico y práctico para mejorar la comprensión de los lectores; y el proyecto realizado en el motor ya estaba previamente diseñado y listo para su uso.

3.3.1 COMPONENTES DEL MANUAL

El manual está dividido en seis capítulos que abarcan desde la historia de los videojuegos, los géneros y subgéneros existentes, la historia de Unreal Engine hasta su avance actual, características, su interfaz de usuario, los elementos tanto conceptuales como prácticos de los videojuegos usados en el motor, la lógica así como el diseño de programación aplicada en el proyecto, las ediciones estéticas incluyendo

la creación y edición tanto de materiales como de partículas y las configuraciones para crear el proyecto listo en las múltiples plataformas existentes desde las consolas actuales más populares hasta dispositivos móviles.

3.3.2 PRUEBAS DE EJECUCIÓN

El equipo de prueba para la ejecución del juego fue una *laptop* ASUS con procesador Intel Core i7-5500U con velocidad de procesamiento de 2.4GHz capaz de potenciarse hasta 3.0GHz, tarjeta gráfica Nvidia Geforce 940M (VRAM de 2GB), Memoria RAM de 12GB con memoria de almacenamiento de hasta 1TB y sistema operativo Windows 8.1.

Se realizaron varias pruebas de las funciones que ofrece el motor en el desarrollo de varios proyectos para aplicarlos en uno solo que abarque lo más esencial en lo que respecta a aspectos estéticos, lógicos, programáticos, animación y configuración; también la creación del proyecto ejecutable para una plataforma específica para su apropiada documentación en el manual, haciéndolo de manera entendible y completa.

El proyecto creado con el motor será de género de aventura para la plataforma de computadoras en el cual se aplicaron pasos detallados en el manual.

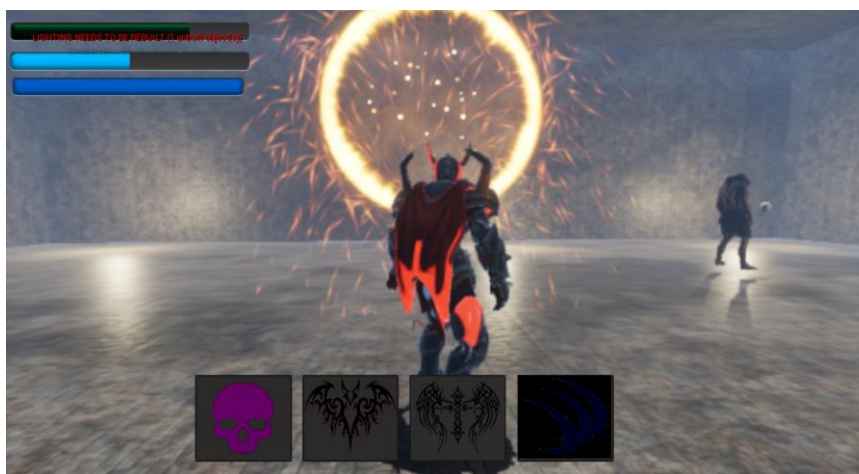


Figura 5. Captura del proyecto en ejecución

3.3.3 LIMITACIONES EJECUCIÓN

De acuerdo con las pruebas experimentales, los proyectos desarrollados al crearse el juego listo para ser ejecutado, presentaron bajo rendimiento de fotogramas por segundo al tener de manera predeterminada los elementos gráficos en calidad máxima haciendo que el rendimiento del procesador y la tarjeta gráfica enfocara todos sus recursos en la renderización del nivel afectando negativamente la experiencia del juego.

Con base en estos argumentos se consideró que el juego funcionará correctamente personalizando la configuración de los gráficos para garantizar su funcionamiento en distintos dispositivos, siempre y cuando cumplan con los requisitos mínimos; también se puede jugar con los gráficos máximos de tener un equipo computacional de alta gama.

3.3.4 PRUEBAS DE EXPORTACIÓN E IMPORTACIÓN DE ACTIVOS

Usando el motor se ha procedido a subir diferentes tipos de assets dentro del proyecto respecto a formatos de modelos 3D se realizó la prueba de subir obj, fbx, stl, dae, kmz, 3ds, ply, abc, x3d. También formatos de audio como mp3, wma, m4a, luego formatos variados de video como mp4, avi, mkv, flv, entre muchas otras para ser determinado el alcance y limitaciones de formatos que puede manejar el motor para ser detallado en los pasos que funciona y que no respecto a ellos pues si bien existen múltiples formatos no todos los programas los manejan a todos y algunos solo manejan uno en específico exclusivo del mismo.

3.3.5 LIMITACIONES DE EXPORTACIÓN E IMPORTACIÓN DE ACTIVOS

De acuerdo con las pruebas efectuadas de la exportación e importación de modelos en el motor Unreal Engine 4, el motor solo acepta en formato de modelos 3D el obj y el fbx, este último es el más recomendado especialmente porque puede albergar desde el modelo, hasta su esqueleto y animaciones que lo complementan.

Con los formatos de video tiene soporte de una amplia gama como mp4, m4v, mov, avi, 3gp, los formatos de audio abarcan mp3, M4A, wav, snd, etc.

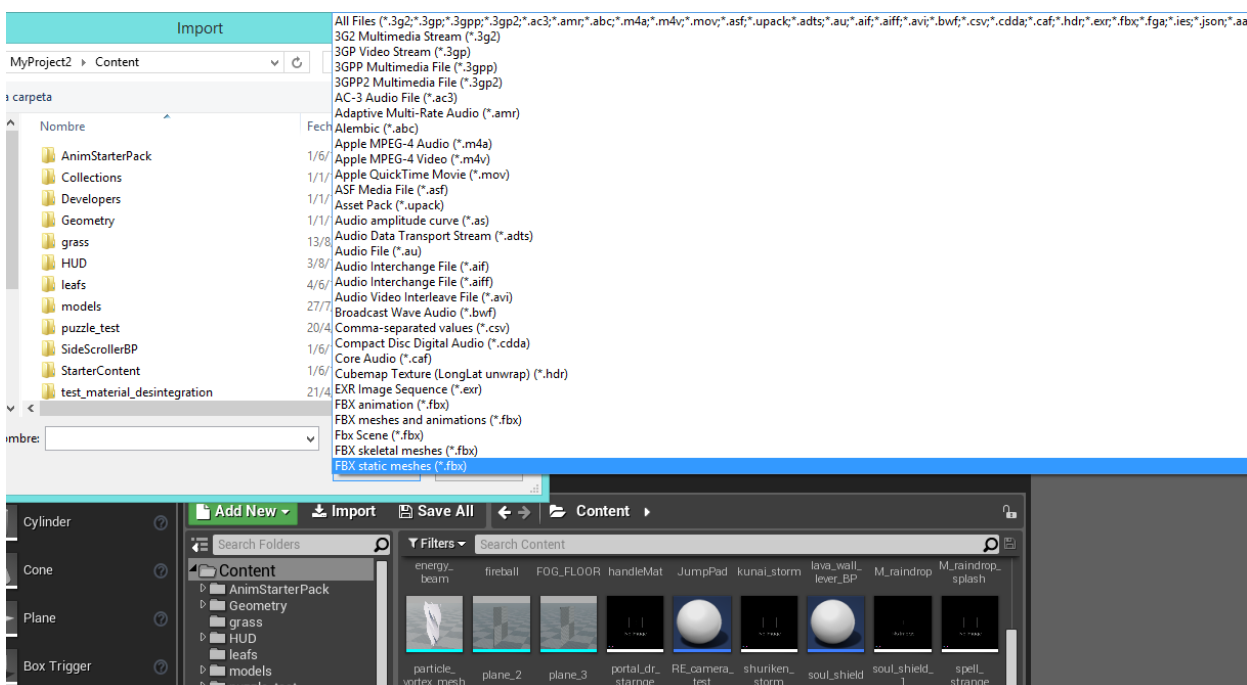


Figura 6. Lista de formatos multimedia que pueden ser exportados en Unreal Engine 4 parte 1

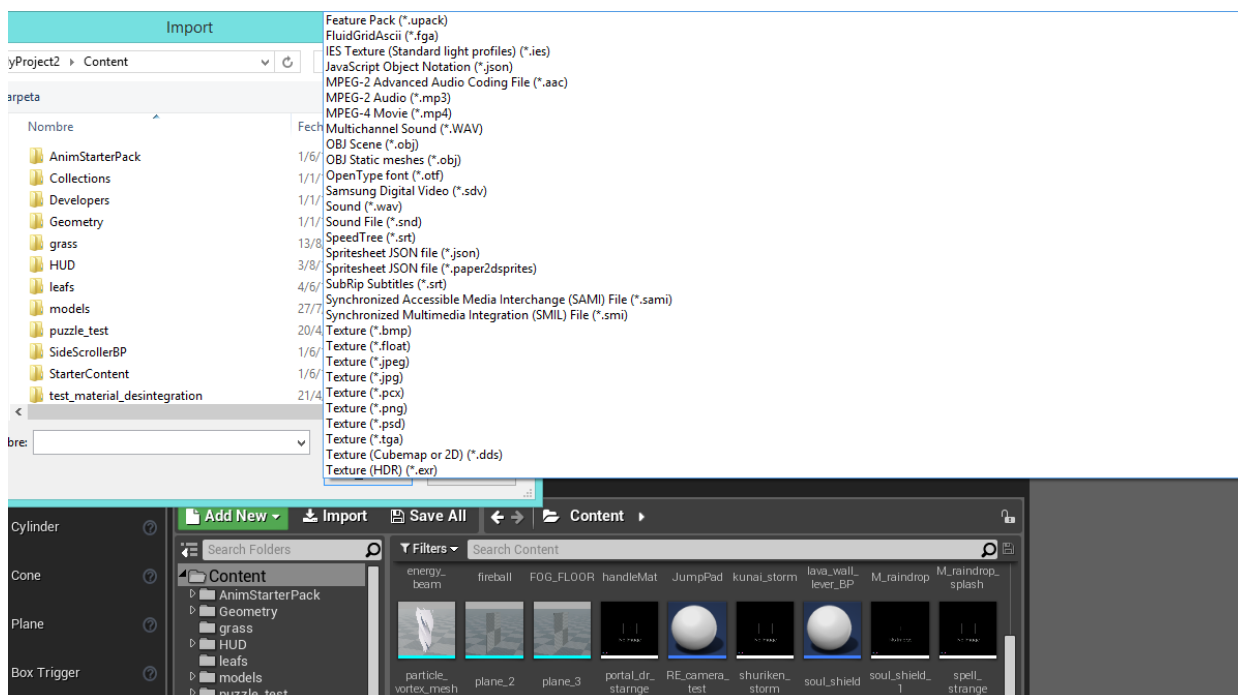


Figura 7. Lista de formatos multimedia que pueden ser exportados en Unreal Engine 4 parte 2

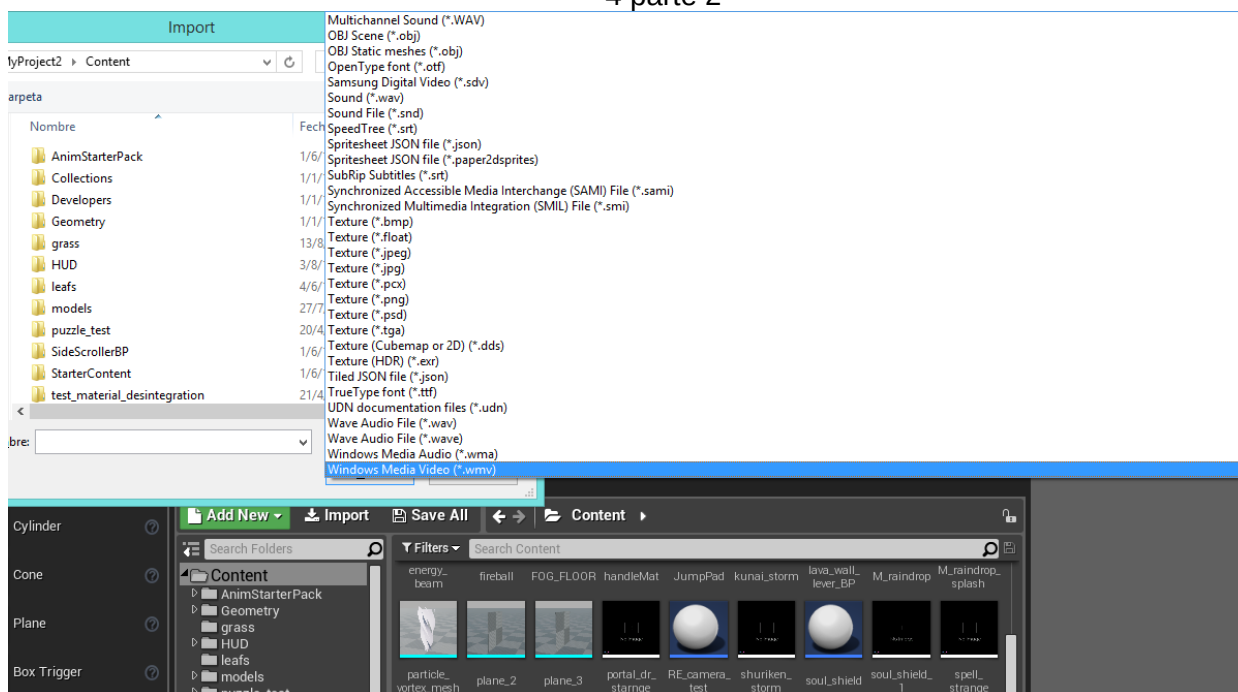


Figura 8. Lista de formatos multimedia que pueden ser exportados en Unreal Engine 4 parte 3

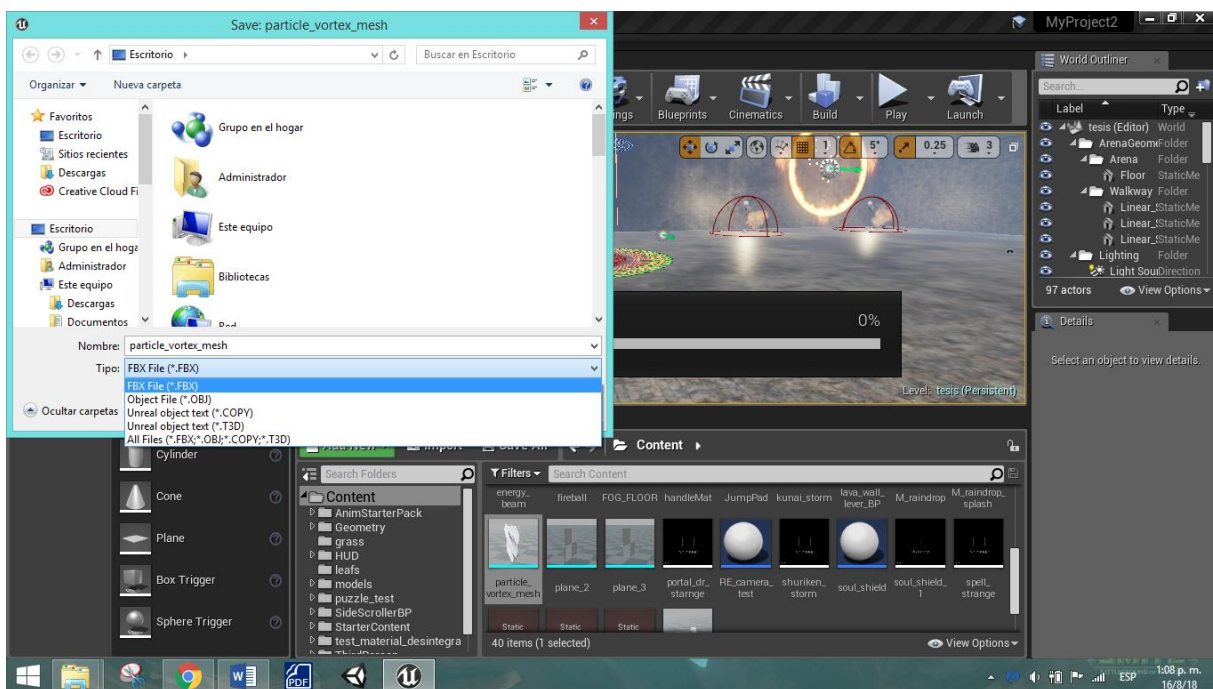


Figura 9. Lista de formatos multimedia que pueden ser importados desde Unreal Engine 4

Tabla 3. Fases de desarrollo del proyecto de investigación

Fases	Títulos	Descripción	Materiales	actividades
I	Estado del arte	Descripción breve de trabajos similares previos.	<i>Laptop</i> con acceso a internet, artículos científicos, libros.	Revisar los estudios existentes para el desarrollo de la teoría que va a componer el manual a nivel teórico y práctico. Hacer un resumen del material bibliográfico.
II	Diseño del manual y el juego	Diseñar prototipo del manual y del juego	El <i>software</i> editor de texto y el motor de videojuego en una <i>laptop</i> que cumpla con los requisitos para usarlo.	Recopilar e identificar los puntos más esenciales para realizar el manual. Desarrollar el diseño del manual. En base al diseño del manual generar un proyecto en el motor Unreal Engine 4.
III	Desarrollo del manual	Desarrollo del diseño del manual	<i>Laptop</i> con editor de texto, acceso a internet, libros.	Desarrollar el manual para el desarrollo de videojuegos paso a paso en el motor Unreal Engine 4, usando como referencia las bibliografías recolectadas para abarcar desde lo básico a lo más esencial.
IV	Desarrollo del juego		<i>Laptop</i> con los requisitos para ejecutar Unreal Engine 4, Blender, y acceso a la página web de Mixamo y varias páginas con modelos 3D para descargar e	En base al manual desarrollado aplicar su contenido en la generación de un juego en Unreal Engine 4 que abarque gran parte de lo visto en el para mostrar los resultados del manual y sus posibilidades.

			importar al proyecto.	
V	Culminación y presentación de tesis y el juego.			Recopilación y culminación del manual redactado con los pasos para desarrollar un videojuego a nivel teórico y práctico. Juego desarrollado en el motor Unreal Engine 4 para computadoras hecho con los pasos descritos en el manual para ser presentado.

CAPÍTULO 4.0

ANÁLISIS E INTERPRETACIÓN DE LOS RESULTADOS

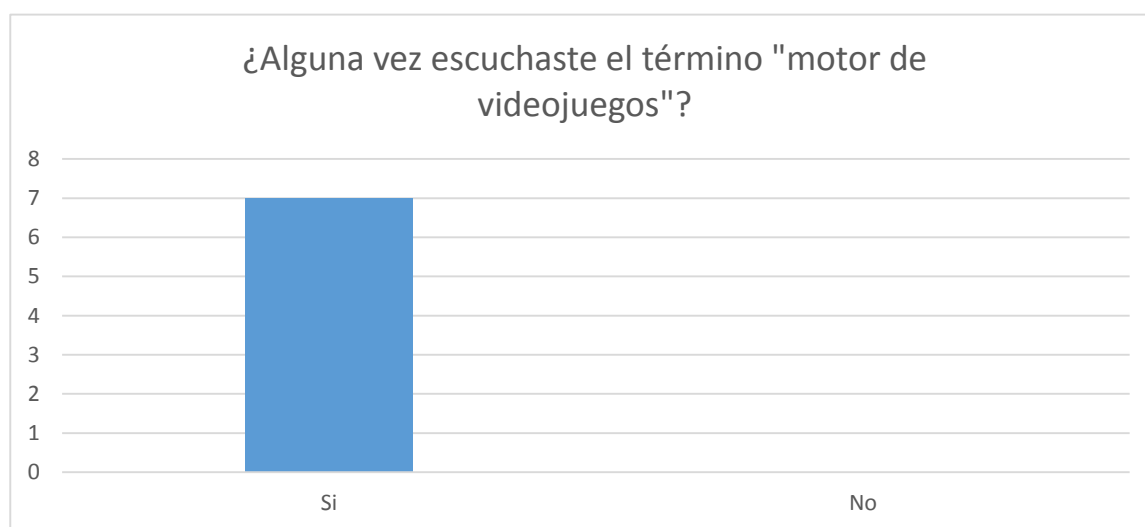
4.1 ANÁLISIS E INTERPRETACIÓN DE LOS RESULTADOS

El instrumento que se utilizó para esta investigación es la encuesta.

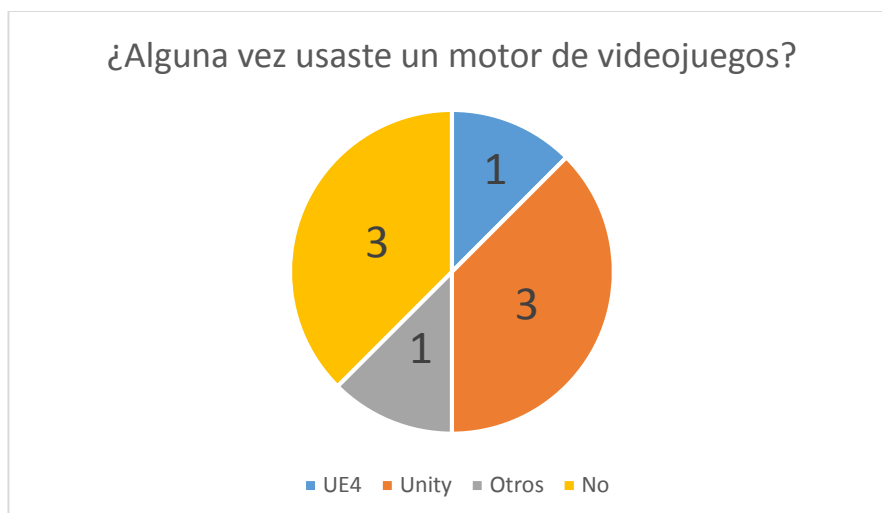
Esta encuesta fue desarrollada en la página www.surveymonkey.com con 10 preguntas en relación con el programa, su composición y la disposición de información de esta en la Universidad Latina de Panamá, luego se envió el enlace de la encuesta a los estudiantes que cumplían con los requisitos de la población de muestra a sus teléfonos inteligentes.

Se estableció un tiempo límite de dos semanas para recibir más encuestas de los participantes de la población, sin embargo hasta el límite de esas dos (2) semanas solo hubo siete (7) que participaron y completaron la encuesta.

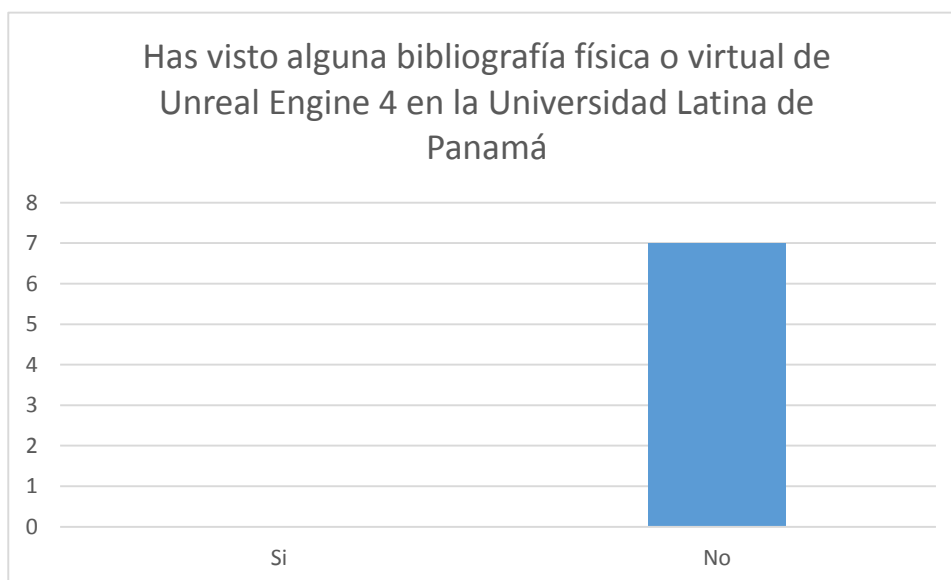
En base a las preguntas se determinó que



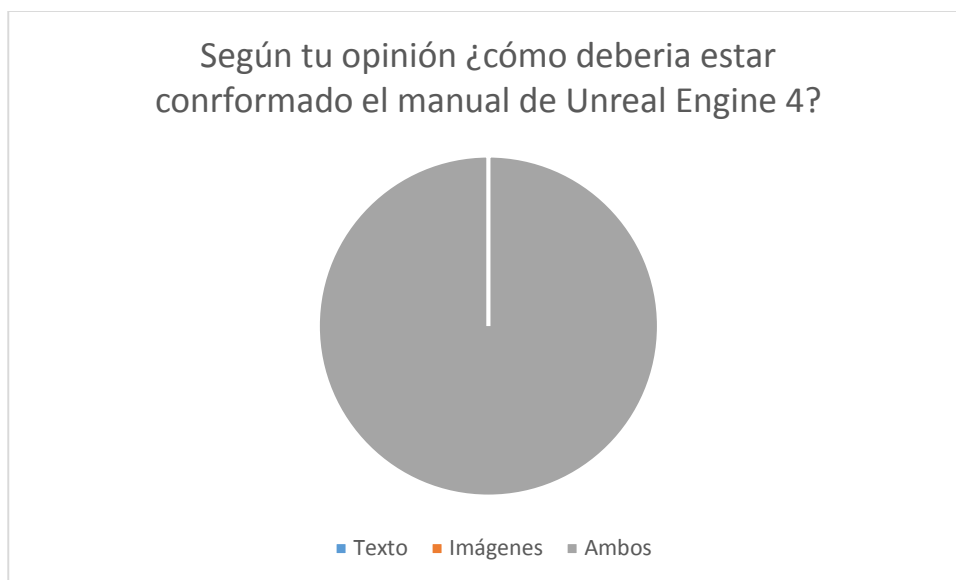
Gráfica 1. Encuesta estudiante 1



Gráfica 2. Encuesta estudiante 2



Gráfica 3. Encuesta estudiante 3



Gráfica 4. Encuesta estudiante 4



Gráfica 4. Encuesta estudiante 4

CAPÍTULO 5.0

PROPUESTA DE LA TESIS

5.1 INTRODUCCIÓN DE LA PROPUESTA

La propuesta consiste en un manual de usuario para el motor Unreal Engine 4 que ha tenido un uso sin precedentes en el desarrollo de videojuegos, animaciones, efectos especiales, etc. Dicho manual explica en teoría y práctica el desarrollo de videojuegos con detallismo de la interfaz del programa, sus funciones, su novedoso método de programación e inclusive instrucciones paso a paso para el desarrollo de elementos clave dentro de este.

Para mejorar la comprensión de los estudiantes de la Universidad Latina de Panamá en la carrera de Ingeniería de Sistemas Informáticos, Animación u otros interesados en poder usarlo para el desarrollo de diversos proyectos con variados usos sin preocuparse por conceptos como cuota mensual o funciones limitadas.

5.2 JUSTIFICACIÓN DE LA PROPUESTA

El desarrollo del manual asistirá a los estudiantes de la Universidad Latina de Panamá no solo en la carrera de Ingeniería en Sistemas en el desarrollo de videojuegos, también a los de animación u otros interesados en desarrollar contenido en el motor Unreal Engine 4 para realizar cualquier proyecto, ya sea interactivo o no, con múltiples usos y posibilidades, impulsando nuevas ideas que será una realidad con su interfaz, su programación amigable e inclusive sus funciones completamente gratis bajo motivos de desarrollo personal y educativo, sin preocuparse por limitaciones como modelos estándares o modelos pagados en comparación con otros motores con los pagos mensuales al tener limitaciones económicas o una moneda más débil que el dólar.

Por eso se tomó la decisión de desarrollar el proyecto de investigación, cuyo Capítulo I consista en explicar la teoría de los videojuegos, empezando con su historia desde sus inicios como proyectos de estudiantes del MIT a su constante evolución, hasta llegar a ser la industria del entretenimiento que es hoy en día que compite con la televisión, el cine e inclusive con la música. También los géneros existentes de los videojuegos para entender las características necesarias para desarrollar uno entendiendo inclusive las limitaciones o inclusiones de funciones dentro de dicho género. Por último, pero no menos importante se plantea de manera resumida y simple la lógica de algunas funciones básicas de los videojuegos para el desarrollo práctico y programático de este en el motor.

El capítulo II abarca la historia del motor Unreal Engine desde sus inicios en 1998 y su contante evolución con sus futuras versiones, hasta el número 4 ahora vigente, los requisitos, pasos de instalación e inclusive una explicación detallada de su interfaz de usuario con todas sus funciones específicas para la adición y edición del proyecto que puede desarrollar el usuario.

El capítulo III se basó en explicar el método de programación de Unreal Engine 4 denominado *blueprints*, explicando su interfaz de usuario, aspecto de funciones, uso de variables, su lógica y aplicaciones.

El capítulo IV explica el desarrollo de un proyecto paso a paso, desde importar, crear y editar contenido (modelos 3D, animaciones, multimedia, etc) hasta los detalles de programación.

El capítulo V explica la función de efectos visuales en Unreal Engine 4 abarcando la importación, creación y edición de materiales para después ser usados

en la interfaz de Cascade que maneja la creación de estos efectos visuales, también anexando un paso a paso en el desarrollo de uno para entender su funcionamiento.

El capítulo VI explica las configuraciones del proyecto desarrollado, controles, plataforma, nombre e incluyendo la generación del ejecutable para poder ser usada en las variadas plataformas en este caso Windows.

5.3 OBJETIVOS DE LA PROPUESTA

OBJETIVOS GENERALES

Desarrollar un Manual de videojuego del motor en Unreal Engine 4.

OBJETIVOS ESPECÍFICOS:

- Determinar los procedimientos de gestión de la plataforma de videojuegos Unreal Engine 4.
- Diseñar el contenido didáctico que pueda auxiliar a los estudiantes de la Universidad Latina de Panamá a desarrollar las habilidades para el desarrollo de proyectos a través del programa Unreal Engine 4.
- Establecer los componentes y el uso de Unreal Engine 4 en elementos claves para desarrollar un videojuego.

5.4 METAS A ALCANZAR

Desarrollar un manual que explique de manera detallada y concisa, la interfaz del programa Unreal Engine 4, así como sus funciones en el desarrollo de proyectos para mejorar la comprensión de este programa para los estudiantes de la Universidad Latina de Panamá, e impulsar el uso de este programa no solo en el desarrollo de videojuegos, también en el desarrollo de videos, animaciones, efectos especiales, simulaciones, etc.

5.5 BENEFICIOS DE LA PROPUESTA

Tener a disposición material instructivo de las características y funciones del motor de videojuegos Unreal Engine 4, así como información tanto teórica como practica en el desarrollo de proyectos variados para los estudiantes de la Universidad Latina de Panamá, abarcando no solo los de Ingeniería de Sistemas, también a los otros estudiantes de distintas carreras interesados en sus usos para animación, creación de estructuras, creación de modelos 3D, simulaciones, etc.

5.6 CRONOGRAMA DE LA PROPUESTA

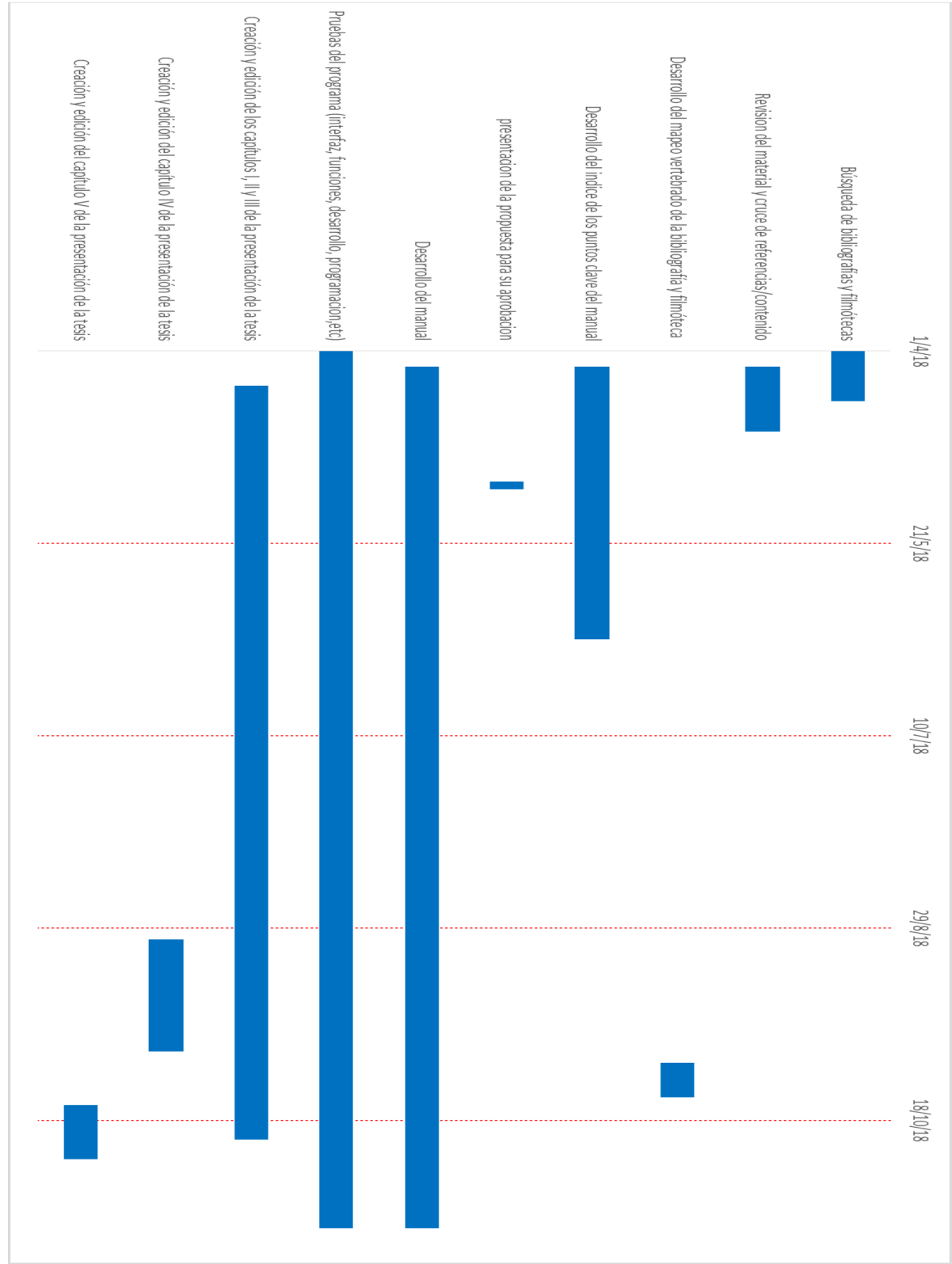


Figura 10. Cronograma del desarrollo del manual

5.7 PRESUPUESTO DE LA PROPUESTA

Debido a que los programas son de licencia gratuita por motivos educativos o personales y tanto los modelos como animaciones fueron obtenidos de páginas que ofrecen descargas gratuitas. No se ha usado ningún presupuesto en el desarrollo del proyecto o del manual.

5.8 DISEÑO DE LA PROPUESTA

Debido a la extensión y diseño del manual del programa Unreal Engine 4 se ha optado por posicionar el contenido del mismo dentro de los anexos para mantener el formato de escritura y el índice original.

ANEXOS

Tabla de contenido

INTRODUCCIÓN	1
GLOSARIO	3
LOS VIDEOJUEGOS.....	9
Historia de los videojuegos	9
Géneros y subgéneros de los videojuegos	15
Lógica esencial de elementos en los videojuegos.....	25
MOTOR DE VIDEOJUEGO	30
HISTORIA DE UNREAL ENGINE	31
INSTALACIÓN DE UNREAL ENGINE 4	34
INTERFAZ DE UNREAL ENGINE 4	38
HERRAMIENTAS DEL EDITOR.....	47
Barra de pestañas.....	47
Barra de menús	48
File (Archivo)	49
Cargar y guardar	49
Proyecto	50
Actores	51
Aplicación	51
Edit (Editar)	52
Historia	52
Editar	52
Configuración	53
Window (Ventana)	53
Editor de nivel	53
General	55
Experimental	55
Diseño.....	56
Help (Ayuda).....	56
Explorar	56
Tutoriales.....	57
Online	57
Barra de herramientas	58

Controles	61
Modos (<i>Modes</i>)	63
Colocar (<i>Place</i>).....	63
Pintar (<i>Paint</i>)	65
Paisaje (<i>Landscape</i>)	66
Follaje (<i>Foliage</i>)	67
Edición de geometría (<i>Geometry Editing</i>).....	67
Trazado de línea mundial (<i>World Outliner</i>).....	69
Navegador de contenido (<i>Content Browser</i>)	70
Detalles (<i>Details</i>)	71
PROGRAMACIÓN POR NODOS	72
Tipos de nodos	73
Entrada y salida de nodos	78
DESARROLLO DE UN JUEGO EN UNREAL ENGINE 4	82
Selección de proyecto	82
Importación de contenido.....	83
Creación de <i>blueprints</i>	84
CREACIÓN DE EFECTOS VISUALES EN UNREAL ENGINE 4	96
CONFIGURACIÓN Y CREACIÓN DEL PROYECTO.....	106
RECOMENDACIONES	110
CONCLUSIONES	111

INTRODUCCIÓN

La funcionalidad de un motor de videojuegos es proporcionar renderización para gráficos 2D y 3D simulaciones físicas, efectos, audios, videos, animaciones, *scripts*, inteligencia artificial, conexiones de redes, administración de recursos para ejecutarlo, etc. El desarrollo del videojuego varía por la reutilización o adaptación del mismo motor en la generación de otros juegos y dependiendo de los recursos y herramientas del motor se puede agilizar la producción y creación del videojuego.

Esto y mucho más es lo que compone este manual para instruir a los estudiantes interesados en el desarrollo de videojuegos a través del motor Unreal Engine 4, que ha tenido un impacto sin precedentes por su interfaz, funciones, método de programación por nodos, desarrollo de efectos, visuales y disposición tanto inmediata como gratuita en aspectos de desarrollo educativo o personal en diferentes proyectos.

La estructura de esta investigación se basa en seis partes; en la primera sección se hace un breve repaso de la historia de los videojuegos para después conocer definiciones de los géneros existentes y un análisis de la lógica que abarcan tanto aspectos básicos como complejos para la planificación y aplicación de ideas en el proyecto que pueden realizarse en el motor.

La segunda sección abarca la descripción de lo que es un motor de videojuegos, la historia resumida del motor Unreal Engine hasta su versión actual, así como los requisitos e instrucciones de instalación del programa para empezar las pruebas y desarrollo de videojuegos en el equipo.

La tercera sección describe de manera resumida y detallada los elementos de la ventana del editor del motor Unreal Engine 4 y sus funciones en el desarrollo de proyectos a nivel estético, así como también el programático e inclusive las múltiples funciones para poder manipular los elementos internos y externos del nivel.

La cuarta sección explica detalladamente la programación de *script* visual denominada *blueprints* o planos que caracteriza el motor por encima de los otros motores por su facilidad de escritura, edición y corrección de errores en comparación con la programación por líneas de código, tomando en consideración las definiciones, características, tipos de operaciones, funciones y lógica de los mismos planos.

La quinta sección describe el desarrollo de proyectos en el motor Unreal Engine 4, abarcando desde la selección de plantillas, importación de contenido multimedia incluyendo modelos 3D, crear variados planos, creación y edición de materiales y por último pero no menos importante, la creación de partículas y sus características para efectos especiales en el proyecto.

La sexta sección describe las características, funciones y pasos para configurar el proyecto a nivel de controles, así como la generación del ejecutable del proyecto para las plataformas vigentes al momento de escribir el manual.

GLOSARIO

AAA: Clasificación informal para videojuegos producidos y distribuidos por una editorial mediana o mayor que generalmente tiene mayores presupuestos de desarrollo y *marketing*. El desarrollo de juegos AAA se asocian con un alto riesgo económico con requisitos de altos niveles de ventas para alcanzar la rentabilidad tanto del producto como la editora que los creo.

AI: Inteligencia Artificial (Artificial Intelligence) (AI), un agente racional flexible que percibe su entorno y lleva a cabo acciones que maximicen sus posibilidades de éxito en algún objetivo o tarea.

Assets: Traducido como activos, son los elementos que serán introducidos al videojuego. Estos elementos incluyen Modelos 3D, personajes, texturas, materiales, animaciones, scripts, sonidos, y algunos elementos específicos de cada motor. Cada motor trabaja de una manera distinta a otros lo cual puede aceptar assets que otros motores no pueden manejar, cuando son importados en un formato predeterminado es convertido a uno que el motor pueda reconocer y manejar fácilmente y en ciertos motores puede importarse en formatos variados.

API: Interfaz de Programa de Aplicaciones (Application Programming Interface), es un conjunto de subrutinas, funciones y procedimientos programación orientada a objetos que ofrece cierta biblioteca para ser utilizado por otro *software* como una capa de abstracción usadas generalmente en las bibliotecas de programación.

Blueprints: Es un sistema de *script* visual de juego completo basado en usar una interfaz basada en nodos para crear juegos dentro del editor, usado para definir clases orientadas a objetos u objetos en el editor, a medida que se use UE4 se

encontrarán objetos definidos como planos que serán denominados coloquialmente como *blueprints*.

Buff: Efecto positivo que mejora alguna de las características de un personaje o enemigo, generalmente de manera temporal, como aumentar la velocidad, la salud, la fuerza de ataque, etc.

Cameo: Aparición de manera inesperada de un personaje especialmente conocido entre el público, representando un papel totalmente secundario e irrelevante, pueden ser de referencia a televisión, historia, noticias, etc.

CPU: Unidad Central de Procesamiento (Central Processing Unit) (CPU), *hardware* dentro de un ordenador u otros dispositivos programables, que interpreta las instrucciones de un programa informático mediante la realización de las operaciones básicas aritméticas, lógicas y de entrada/salida del sistema.

Debuff: Efecto negativo que empeora alguna de las características de un personaje o enemigo, generalmente de manera temporal, como reducir la salud, velocidad, la fuerza de ataque, etc.

DirectX: Colección de API desarrolladas para facilitar las complejas tareas relacionadas con multimedia, especialmente programación de juegos y vídeos, en la plataforma Microsoft Windows.

ESRB: Tabla de Clasificación de *Software* de Entretenimiento (Entertainment Software Rating Board) (ESRB), un sistema norteamericano para clasificar el contenido de los videojuegos, productos y películas, y asignarle una categoría dependiendo de su contenido, en Europa se le conoce como PEGI (Pan European Game Information).

Gamepad: Periférico de entrada que permite controlar el menú y los movimientos de los actores de un videojuego. Generalmente son pequeños (cabén en las manos), poseen botones, palancas o cruces para que puedan ser usados por los dedos.

GOTY: Edición Juego del Año (Game Of the Year Edition) (GOTY), significa que esta versión de su juego pretende ser aplicable a un premio GOTY, que puede significar cualquier cosa, desde una actualización completa con todos los parches, gráficos mejorados, copias agrupadas de juegos anteriores en la serie, DLC incluido y expansiones, físico de caja mercancía, o cualquier otra cosa que el desarrollador y el editor puedan pensar para hacer que se destaque y gane la nominación.

GPU: Unidad de Procesamiento Gráfico (Graphics Processing Unit) (GPU), coprocesador dedicado al procesamiento de gráficos u operaciones de coma flotante, para aligerar la carga de trabajo del procesador central en aplicaciones como los videojuegos o aplicaciones 3D interactivas. De esta forma, mientras gran parte de lo relacionado con los gráficos se procesa en la GPU, la unidad central de procesamiento (CPU) puede dedicarse a otro tipo de cálculos (como la inteligencia artificial o los cálculos mecánicos en el caso de los videojuegos).

Hardware: Son los componentes físicos de la computadora (Pantalla, teclado, etc.).

HUD: Presentación de la Información (Heads-Up Display) (HUD), conjunto de iconos, números, mapas, etc. que nos dan información sobre el estado de nuestra partida y/o nuestro personaje, como por ejemplo vida restante, ubicación, munición, objetos en uso, etc. También se le denomina como Interfaz de Usuario (User Interface) (UI).

Indie: Es un término inglés que significa independiente o independencencia.

Joystick: El *joystick* o palanca de mando es un artefacto compuesto por una palanca de control que consta de tres ejes, el cual es utilizado para controlar o manejar un videojuego.

Juego Indie: Término usado para los videojuegos de desarrollo independiente.

Nodo: Es un punto de intersección, conexión o unión de varios elementos que confluyen en el mismo lugar.

NPC: Personaje No Jugable (Non Player Character) (NPC), son parte de la programación del juego incontrolable por una persona, que cuenta con cierto nivel de interacción y a veces inteligencia artificial moderada.

O.S: Sistema Operativo (Operating System) (OS), *software* principal o conjunto de programas de un sistema informático que gestiona los recursos de *hardware* y provee servicios a los programas de aplicación de *software*, ejecutándose en modo privilegiado respecto de los restantes (aunque puede que parte de él se ejecute en espacio de usuario).

PC: Personaje Jugable (Player Character) (PC), personaje controlado por un jugador durante una sesión de juego.

PVE: Jugador Contra Entorno (Player Versus Environment) (PVE), modalidad en la que la única interacción que existe entre los jugadores es la colaboración y todos los enfrentamientos se realizan contra los NPC.

PVP: Jugador Contra Jugador (Player Versus Player) (PVP), muy usado en los videojuegos de rol para designar una modalidad en la que dos jugadores se enfrentan entre ellos.

QTE: Eventos de Tiempo Rápido (Quick Time Events) (QTE), en que se inicia una secuencia en la que se debe presionar un conjunto de botones en el orden que

aparece y que pueden ser aleatorios la aparición de los mismos, para completar una acción; en caso contrario se debe repetir la secuencia y en ciertos casos implica la muerte del jugador dependiendo de cómo se desarrolle el mismo.

RAM: Memoria de Acceso Aleatorio (Random Access Memories) (RAM), memoria de trabajo de computadoras para el sistema operativo, los programas y la mayor parte del *software*.

Remake: Reedición de un videojuego ya publicado anteriormente, generalmente para adaptarlo a los medios técnicos, sobre todo a nivel gráfico, y gustos del público actuales, pero manteniendo la estructura fundamental del juego en cuanto a jugabilidad, historia, personajes, etc.

Renderización: Es el proceso de la computadora de mostrar en pantalla el aspecto visual del juego. Se encarga de mostrar al jugador todo el poder gráfico que el desarrollador haya configurado en el motor, la renderización muestra todo lo que es el terreno o BSP, modelos, animaciones, texturas y materiales. La renderización contribuye todo el aspecto visual del juego.

Scripts: Es un lenguaje de programación que ejecuta diversas funciones en el interior de un programa de computador, el lenguaje puede variar.

SDK: Kit de Desarrollo de Software (Software Development Kit) (SDK), que es un conjunto de herramientas que le permite al programador o desarrollador de *software* crear una aplicación informática para un sistema concreto, por ejemplo *frameworks*, ciertos paquetes de *software*, plataforma de *hardware*, computadoras, videoconsolas, sistemas operativos, etc.

Software: Son los programas que realizan tareas específicas en la computadora (explorador de archivos, internet, etc.).

Tutorial: Es un curso breve y de escasa profundidad, que enseña los fundamentos principales para poder jugar y entender mejor la interacción del jugador y el mundo a su alrededor.

UE4: Unreal Engine 4 (UE4), motor de videojuegos y animación creada por la empresa Epic Games.

VRAM: Memoria Gráfica de Acceso Aleatorio (Video Random Access Memories) (VRAM), es una memoria RAM que maneja toda la información visual que le manda la CPU del sistema.

1UP: Diminutivo usado para referirse a vidas extras en un juego.

LOS VIDEOJUEGOS

Historia de los videojuegos

De acuerdo con Gonzalo Frasca en el (2001) los videojuegos son “cualquier forma de *software* de entretenimiento por computadora, usando cualquier plataforma electrónica, y la participación de uno o varios jugadores en un entorno físico o de red”.

Los videojuegos se remontan al año 1958, inicialmente por el “Tennis para Dos”, que surgió de la mente de William Higinbotham, físico que trabajaba en los laboratorios Brookhaven National como diseñador de circuitos electrónicos para el Proyecto Manhattan y gran aficionado al Pinball. Los laboratorios celebraban un día anual del visitante, y se decidió realizar una especie de juego basado en un programa de cálculo de trayectorias, utilizado por el ejército americano. La pantalla era circular porque Higinbotham conectó un osciloscopio.

El resultado final fue "Tennis for Two". Consistía en una línea horizontal que representaba el suelo del campo de tenis y de una pequeña línea vertical en la mitad del campo que representaba la red. Los jugadores debían elegir el ángulo en que debía salir la bola y golpearla. A diferencia de casi todos sus sucesores, no hacía uso de una perspectiva aérea para mostrar el desarrollo del juego. Los dos jugadores que podían participar en cada partida disponían de un controlador compuesto por un pulsador que servía para golpear la pelota virtual y un mando analógico con el que se controlaba la dirección de la bola. Este juego actuaba de manera bastante más realista que el conocido Pong (aparecido 15 años después) y muchos lo consideran el primer videojuego de la historia.

El juego tuvo un éxito arrollador y los visitantes hacían unas colas gigantescas para jugar. A pesar de que “Tennis for Two” se convirtió en el foco de atención del público, Higinbotham nunca se planteó patentar su creación ya que no la consideró de importancia. De haberlo hecho, se hubiera convertido en una de las personas más ricas del mundo, al cabo de dos años la maquinaria fue desmantelada y sus partes fueron redistribuidas para otras funciones. Todo era demasiado caro como para dedicarlo a un juego de tenis de mesa.

En 1962 Steve Russell, un estudiante del MIT, dedicó seis meses a crear un juego para computadora usando gráficos vectoriales: *Spacewar!*. En este juego, dos jugadores controlaban la dirección y velocidad de dos naves espaciales que luchaban entre ellas.

El videojuego funcionaba sobre un PDP y fue el primero en tener cierto éxito aunque apenas fue conocido fuera del ámbito universitario.

En 1966 Ralph Baer en ese momento diseñador jefe de Sanders Associates, una empresa que trabajaba para el ejército reconsideró una idea que había abandonado unos años antes, un dispositivo que conectado a un simple televisor, permitiese jugar al espectador con su aparato.

Aprovechándose de su situación en la empresa, Baer comenzó a diseñar su aparato en secreto en los laboratorios de la misma por miedo a que su idea pudiese ser considerada como poco seria por sus superiores. Debido a que las negociaciones nunca terminaron no se concretó en nada dejándola en el olvido.

En 1971 Bill Pitts, un estudiante de la Universidad de Stanford fascinado por *Spacewar!* tuvo la idea de hacer una versión del juego que funcionase con monedas para su explotación en los salones recreativos. Junto con su socio Hugh

Tuck la empresa Computer Recreations, Inc., con el propósito de construir una versión operada con monedas de *Spacewar!*; Pitts se hizo cargo de la programación y Tuck, ingeniero mecánico, construyó la cabina.

Los prototipos iniciales funcionaban pero el problema era la fabricación de múltiples unidades y debido a los altos precios de los componentes vender las partidas a 10 centavos en ese tiempo era demasiado y por lo tanto no era rentable, la solución de ellos fue crear un solo computador que pudiera hacerse cargo de hasta 8 consolas simultáneamente amortizando los gastos.

En enero de 1968 Ralph Baer no había desistido en sus empeños y se había hecho con la primera patente por sus conceptos sobre videojuegos. En julio de 1968 Bill Enders, un exdirectivo de RCA que trabajaba para Magnavox y que había quedado impresionado con las primeras demostraciones de Baer, convenció a otros directivos de la empresa para que dieran una oportunidad a Baer. En abril de 1972 luego de hacer los acuerdos y planificación la firma presentó la nueva máquina a la prensa y a sus distribuidores.

Al mismo tiempo se presentó el primer accesorio de la máquina, un rifle de plástico de buena apariencia, y diez juegos adicionales, todos ellos vendidos por separado. Los *flyers* de la época mostraban ya una consola de videojuegos exactamente tal y como la conocemos hoy, pero Magnavox cometió errores de *marketing* que jugaron en su contra: por un lado el anuncio de televisión daba la impresión de que la consola solo se podría usar con un aparato de televisión de la misma marca, lo que no era cierto; por otro lado, la distribución se limitó a las franquicias de Magnavox, lo que limitaba considerablemente el número de clientes potenciales. Sin embargo para ese entonces lograron vender 130.000.000

unidades en su campaña navideña, llegando a llamar la atención de varios entre ellos a Nolan Bushnell quien al probar el juego de ping-pong decidió contratar a Alan Alcorn un ingeniero de la compañía Ampex para desarrollar una versión en máquina de “Arcade”; con el nombre de Pong no suponía grandes innovaciones respecto al título de Baer, pero sí contaba con mejoras (rutina de movimientos mejorada, puntuación en pantalla, efectos de sonido, entre otras) que hacían presagiar el éxito que lograría en los salones. Con esta idea nació la famosa empresa Atari que intentó vender sus máquinas a distintas empresas sin éxito y Nolan Bushnell se convenció de que su empresa Atari fuera la que fabricara y distribuyera sus máquinas, Al final fue un éxito, se desbordaron de pedidos llegando hasta 150 máquinas de “Pong” entre estas y otras copias de la misma por parte de otras empresas provenientes de Japón, Italia, Francia. Ya en 1974 había 100.000 máquinas de “Arcade” en Estados Unidos generando un ingreso de doscientos millones de dólares anualmente; gracias a esto nacieron los videojuegos como una industria.

Gracias a la introducción del primer microprocesador por parte de Intel su modelo 4004 cambió para siempre el desarrollo de los videojuegos pues ampliaba las ediciones a nivel de programación y gráficos sin alterar el *hardware* a diferencia de los modelos hechos con circuitos TTL que llevaron al desarrollo de las computadoras personales de Apple y sus primeros juegos, entre ellos “Adventure” que fue uno de los más influyentes para la plataforma Apple II un juego de aventura conversacional, más adelante Atari también aprovechó y desarrolló su primera consola en usar la tecnología de microprocesador con la posibilidad de jugar diferentes juegos a través de cartuchos intercambiables, de ahí surge la famosa e

icónica Atari 2600. En 1976 Warner Bros. Inc. compra Atari por veintiocho millones de dólares y en 1977 lanzan al mercado la Atari 2600.

En 1978, tras presentar una demanda a Atari por haberse basado en ping-pong de 1972 para su éxito Pong, Magnavox introdujo su Odyssey II para hacer competencia a la 2600 de Atari. Bushnell, se alejaba cada vez más de la dirección de su compañía, fue finalmente sustituido por Ray Cassar, un ejecutivo sin experiencia en videojuegos que cambió radicalmente la filosofía de la compañía, centrándose en el mercado de los ordenadores personales y poniendo fin a la época dorada de Atari.

Mientras tanto, en Japón Toshihiro Nishikado adoptó la nueva tecnología de microprocesadores y desarrolló un juego de disparo para la empresa Taito que influenciado por el éxito de la película “Star Wars” le dio su forma de batalla espacial, así nació el juego de “Space Invaders” que impulsó la fiebre de los videojuegos y marcó una era dorada en la industria de los mismos. Luego aumentó considerablemente las ideas y creaciones de estas cuando llegó la irrupción del color.

Otra empresa que marcó esta época dorada fue Namco con títulos como “Frogger” o “Centipede”, el más emblemático y conocido hasta la actualidad en muchas generaciones es “Pac-Man” siendo jugado tanto por hombre como mujeres por sus gráficos simples y llamativos como su jugabilidad simplificada, logrando convertirlo en un éxito inmediato y con recaudaciones masivas de sus máquinas de Arcade.

A medida que pasaba el tiempo en la década de 1980 con la llegada de los microprocesadores y el auge de la industria de los videojuegos aparecieron más

empresas fundadas por desarrolladores, comerciantes y entusiastas llegando a desarrollar una amplia gama de juegos, así como consolas y computadoras dedicadas a estos y con la llegada de los 8 bits se crearon para explotar al máximo los videojuegos y multimedia computadoras domésticas de distintas marcas, que fueron innovadoras para la época como la Commodore 64, el Armstrad CPC 464, el MSX; en lo que respecta a consolas de 8 bits la primera y más popular en ese entonces fue la “Nintendo Entertainment System” o “NES” para abreviar y ahí hizo debut una franquicia que está vigente hasta la actualidad que es conocida tanto por la juventud como las generaciones anteriores proveniente de la década de 1980 “Super Mario Bros” y muchos otros grandes éxitos que les han sacado varias versiones e historias a lo largo de los años hasta la actualidad como “The Legend of Zelda”, “Donkey Kong”, “Super Metroid”, “Kirby”. Entre muchas otras sagas provenientes de desarrolladores de otras empresas que deseaban lanzar sus juegos pero no contaba con el diseño ni el presupuesto para sus propias consolas como Namco con “Pac-Man”, Konami con “Castlevania” o Capcom con “Street Fighter”.

Géneros y subgéneros de los videojuegos

A la hora de planificar y desarrollar un videojuego se debe elegir el género en el que se va a crear, pues sin importar todas las ideas creativas, los elementos estéticos, audiovisuales ni la historia en la que se va a estar basada, no se puede proceder sin establecer cómo será la mecánica del mismo incluyendo sus limitaciones o para quien va dirigido el juego que estos aspectos ayudan en la toma de decisiones del diseño, planificación y culminación del mismo juego, de no ser así será más difícil de desarrollar pues se tendrá que ser más creativo y original, un reto poco probable pero no imposible de lograr.

Entre los géneros existentes tenemos:

- **Acción.** Es un videojuego en el que el jugador debe usar su velocidad, destreza y tiempo de reacción.
- **Arcade.** Término genérico de las máquinas recreativas de videojuegos disponibles en lugares públicos de diversión, centros comerciales, restaurantes, bares o salones recreativos especializados. Son similares a los *pinballs* y a las tragamonedas de los casinos, pero generalmente sin las limitaciones legales que implican.
- **ARPG.** Juego de rol de acción (Action Role Playing Game) (ARPG). Es un género de videojuegos que comparte muchas características con los RPG, pero que ofrecen combates en tiempo real. Su sistema de combate es parecido a los de tipo *Hack and Slash*.
- **Aventura.** Es un género de videojuegos, caracterizado por la investigación y exploración, la solución de rompecabezas o rompecabezas, la interacción

con personajes del videojuego, y un enfoque en el relato en vez de desafíos basados en reflejos.

- **Aventura conversacional.** Es un género de videojuegos en que la descripción de la situación en la que se encuentra el jugador proviene principalmente de un texto. El jugador debe teclear la acción a realizar y normalmente se interpreta la entrada en lenguaje natural. A veces existen gráficos en estos juegos, que sin embargo solo son situacionales o que ofrecen ayuda complementaria (al estilo de ilustraciones de libros).
- **Aventura gráfica.** Es un subgénero de los juegos de aventura concebido como una evolución de las aventuras conversacionales. La dinámica consiste en ir avanzando por el juego a través de la resolución de diversos rompecabezas, planteados como situaciones que suceden en la historia, interactuando con personajes y objetos.
- **Beat'em up.** También conocido como *brawler*. Es un género de videojuegos en el que destaca el combate cuerpo a cuerpo entre el protagonista y un gran número de antagonistas. Éste género está separado de los videojuegos de lucha puesto que ocurren sobre un nivel largo, con desplazamiento de pantalla cuando el jugador se mueve por el escenario, y con menor cantidad de ataques pero con un esquema de control simple.
- **Bullethell.** Literalmente infierno de balas, son videojuegos que presentan cantidades abrumadoras de proyectiles enemigos, cuyos patrones deben memorizarse para poder evitarlos.

- **Clicker.** Es un género de videojuegos de corte casual donde el jugador tiene que hacer *click* repetidamente (u otra acción equivalente) para obtener algún tipo de recurso que luego podrá invertir, ya sea en mejorar un personaje o cualquier otro tipo de acción dentro de la temática del juego.
- **Creación musical.** Es un subgénero de los juegos musicales donde el jugador dispone de diversas herramientas para la composición musical, y que se centra en un aspecto creativo.
- **Endless runner.** Es un género de videojuegos donde el jugador debe avanzar de manera irremediable en una misma dirección, generalmente escapando de algún enemigo o peligro, y cuyo objetivo es avanzar lo máximo posible antes de morir.
- **Estrategia.** Es un videojuego que requiere que el jugador ponga en práctica sus habilidades de planeamiento y pensamiento para gestionar recursos de diversos tipos (materiales, humanos, militares, etc.) y conseguir la victoria.
- **FPS.** Tirador en Primer Persona (First-Person Shooter) (FPS) o también conocido como videojuego de disparos en primera persona. Es un subgénero de videojuegos de disparo en los que el jugador observa el mundo desde la perspectiva del personaje protagonista.
- **Gestión.** También conocidos como videojuegos de construcción y gestión. Son un subgénero de los videojuegos de simulación donde los jugadores construyen expanden o gestionan comunidades ficticias o proyectos con recursos limitados. Algunos juegos de estrategia tienen

aspectos de este tipo sin embargo el objetivo del género de gestión no es derrotar a un enemigo.

- **God game.** Término en inglés de juego de dios. Es un subgénero de la estrategia donde el jugador observa el mundo desde una perspectiva superior, como desde el cielo, y donde cuenta con la capacidad de influir en los eventos que suceden en el mundo donde se ambienta el juego, como si se tratase de un dios.
- **Hack and Slash.** Es un género de videojuegos basado en los combates. El estilo es como el de los *beat'em up* con la diferencia de que el personaje suele llevar algún tipo de arma cuerpo a cuerpo, normalmente de filo o contundente.
- **Incremental.** Es un género de videojuegos de corte casual donde el jugador tiene que realizar de manera repetida una acción (como hacer *click* o tocar la pantalla) para obtener algún tipo de recurso y luego invertirlo en todo tipo de acciones que en definitiva servirán para obtener más recursos de manera incremental.
- **JRPG.** Juego de Rol Japonés (Japanese Role Playing Game) (JRPG). Término para referirse a los RPG o juegos de rol desarrollados en Japón (y países limítrofes) dado que cumplen con una serie de premisa estéticas y formales muy similares, como aspecto desenfadado y muy colorista, personajes muy jóvenes y andrógenos, estética manga/anime, historias lineales o combate por turnos.

- **Lógica.** También conocidos como videojuegos de inteligencia o videojuegos de puzzle. Son un género de videojuegos que se caracterizan por exigir agilidad mental al jugador. Pueden involucrar problemas de lógica, matemáticas, estrategia, reconocimiento de patrones, completar palabras o azar. Incluye juegos de lógica rápida, juegos de física, juegos de objetos ocultos y de acertijos.
- **Lucha.** Es un subgénero de los videojuegos de acción que se basa en manejar un personaje peleador, ya sea dando golpes, usando poderes o aplicando llaves, existen desde dos hasta cuatro jugadores localmente, mientras que los *online* pueden llegar a ser hasta el doble.
- **Lucha 1 vs 1.** Es un subgénero de los videojuegos de lucha. Generalmente consiste en pelear contra otro luchador, manejado por el CPU o por otro jugador, cuya finalidad es derrotarlo o evitar que derroten al nuestro, dándole golpes o cualquier otro tipo de ataque para debilitarlo y vencerlo, así como esquivar y contraatacar cualquier ataque recibido.
- **Lucha Free-for All.** Es un subgénero de los videojuegos de lucha. Free-for All (“Todos contra todos”) contiene la esencia del juego de lucha 1 vs 1 pero el enfrentamiento puede ser también entre 3 o 4 personas (o personajes controlados por el CPU) simultáneamente.
- **Musical.** Son videojuegos de música que inducen a la interacción del jugador con la música, cuyo objetivo es seguir los patrones de una canción en el videojuego ejecutando los paneles, botones o simuladores de instrumentos

para completar sonidos y/o entonaciones que concuerdan con la música o con motivos de ejecutarlos al ritmo de la canción para completar el set.

- **MMOG.** Videojuego Multijugador Masivo en Línea (Massively Multiplayer Online Game) (MMOG). Es un videojuego en donde pueden participar, e interactuar en un mundo virtual, un gran número de jugadores simultáneamente conectados a través de la red, normalmente internet.
- **MMORPG.** Videojuego de Rol Multijugador Masivos en Línea (Massively Multiplayer Online Role-Playing Game) (MMORPG). Es un videojuego de rol que permite a miles de jugadores introducirse en un mundo virtual de forma simultánea a través de internet e interactuar entre ellos. Consisten en la creación de un personaje e ir aumentando los niveles de éste y experiencia en peleas, a veces pueden tener modos PvP, PvE, etc.
- **MMORTS.** Estrategia en Tiempo Real Multijugador Masivos en Línea (Massively Multiplayer Online Real-Time Strategy) MMORTS. Es un videojuego de estrategia en tiempo real que permite a miles de jugadores introducirse en un mundo virtual de forma simultánea a través de internet, e interactuar entre ellos en tiempo real, a veces pueden tener modos PVP, PVE, etc.
- **MOBA.** Arena de Combate Multijugador Online (Multiplayer Online Battle Arena) (MOBA), también conocido como Estrategia de Acción en Tiempo Real (Action Real-Time Strategy) (ARTS). Es un género de videojuegos derivado de la estrategia en tiempo real (RTS), en el que a menudo dos equipos de jugadores compiten entre sí, con cada jugador controlando un

solo personaje a través de una interfaz de estilo RTS, establece cooperatividad en equipo, a veces pueden tener modos PvP, PvE, etc.

- **Party game.** Es un género multijugador en el que los jugadores deben desplazarse por turnos en el tablero virtual, y al finalizar todos los turnos participan en distintos minijuegos competitivos individuales o por equipos, con el fin de obtener la mejor puntuación o tiempo.
- **Plataforma.** Es un subgénero de los videojuegos de acción que se caracteriza por tener que caminar, correr, saltar o escalar sobre una serie de plataformas, con enemigos, mientras se recogen objetos para poder completar el juego.
- **Point and click.** Son aquellos juegos donde se usa exclusivamente el ratón del PC para interactuar con el juego; es necesario hacer *click* en distintas zonas del escenario para realizar las acciones.
- **Point and Shoot.** Literalmente apuntar y disparar. Es un subgénero de los juegos de disparo con vista en primera persona, en los que el apuntado y el disparo se realizan al pulsar sobre la pantalla táctil. Por lo que está diseñado para dispositivos móviles como *smartphones* o *tablets*.
- **Ritmo.** Es un tipo de videojuego de acción musical. Los videojuegos de este tipo se concentran en el baile o simular la ejecución de instrumentos musicales. Los jugadores deben apretar una serie de botones en momentos precisos correspondiendo a una secuencia indicada por el juego.
- **Roguelike.** Es un género de videojuegos que habitualmente participa un jugador; pueden tener un sistema de turnos o microturnos, una parte de

exploración, contenido aleatorio, muerte permanente, dificultad elevada (con una curva de dificultad pronunciada que estimula el aprendizaje de los pormenores del juego para avanzar), requiere perseverancia del jugador, muy poca narrativa. No necesariamente debe presentar todas las características anteriores.

- **RPG.** Juego de Interpretación de Roles (*Role-Playing Game*) (RPG) y también conocido como juego de rol. Es un género de videojuegos en el que uno o más jugadores desempeñan un determinado rol, papel o personalidad a lo largo de una trama. Cada personaje no sigue un guion prefijado sino que la historia se va creando en el transcurso de la partida.
- **RTS.** Estrategia en Tiempo Real (*Real-Time Strategy*) (RTS). Es un videojuego de estrategia en el que no hay turnos sino que el tiempo transcurre continuamente para los jugadores. Los videojuegos de estrategia en tiempo real están pensados para jugadores de forma dinámica y rápidamente. No precisan un planteamiento tan pausado de las decisiones, se centran a menudo en la acción militar y la recolección de recursos simples.
- **Sandbox.** En este género de videojuegos el jugador cuenta con cierta libertad para moverse por un escenario con varias rutas y afrontar las distintas misiones principales o secundarias en el momento y orden que él decida. Usualmente se da cierta libertad para completar una misión de diferentes maneras.
- **Shooters.** También conocidos como videojuegos de disparos. Es un género que engloba un amplio número de subgéneros que tienen la característica

común de permitir controlar un personaje que, generalmente, dispone de un arma (mayoritariamente de fuego) que puede ser disparada a voluntad. Pertenece al género acción.

- **Simulación.** Son videojuegos que intentan recrear situaciones de la vida real. Pretenden reproducir sensaciones físicas (velocidad, aceleración, percepción del entorno) y una de sus funciones es dar una experiencia real de algo que no está sucediendo, para no poner en riesgo la vida de alguien.
- **Survival horror.** Es un subgénero de videojuegos enfocados en atemorizar al jugador, al que se pretende provocar inquietud, desasosiego e incluso miedo. Estos videojuegos están encuadrados en el género de acción-aventura.
- **TCG.** Juego de Cartas Coleccionables (Trading Card Game) (TCG). Es un tipo de juego de cartas no predefinidas y existentes en gran cantidad y de varios tipos y características, que otorgan individualidad a cada carta y con las cuales puede construirse una baraja o mazo de acuerdo con las reglas de cada juego.
- **TBS.** Estrategia Basada en Turnos (Turn-Based Strategy) (TBS). Es un videojuego en el cual el flujo se divide en partes bien definidas y visibles llamadas turnos o rondas. Por ejemplo, cuando la unidad de flujo de juego es el tiempo, los turnos representan unidades de tiempo, como años, meses, semanas, o días.

- **TPS.** Juego de Disparos en Tercera Persona (Third- Person Shooter) (TPS). Es un subgénero de los videojuegos de disparos en el cual el personaje es visto desde una perspectiva en tercera persona.
- **Wargame.** Es un juego de guerra que recrea un enfrentamiento armado de cualquier tipo (táctico, operacional, estratégico o global) con reglas que implementan cierta simulación de la tecnología, estrategia y organización militar, ubicada en cualquier contexto (histórico, hipotético o fantástico). No implica en ningún momento el uso de violencia física entre los jugadores.
- **4X.** *Explore* (exploración del mapa), *Expand* (expansión territorial), *Exploit* (explotación de recursos) and *Exterminate* (exterminación del enemigo) y también conocidos como videojuegos de construcción de imperios. Subgénero de los videojuegos de estrategia. Tienen como objetivo la creación y mantenimiento de un imperio, dando prioridad a aspectos como la economía, diplomacia e investigación.

(Bolívar, 2017)

Lógica esencial de elementos en los videojuegos

En esta parte del manual se plantea desde el punto de vista lógico programático, cómo funcionan los elementos más usados en los videojuegos para entender que es lo que se necesita para que cumplan una o varias tareas y sirva como base al momento de desarrollar nuevas ideas derivadas que pueden llegar a compartir ciertas características:

- **Trampa.** Se entiende por trampa un elemento del juego para obstaculizar el avance del jugador más adelante de un nivel, escena, mundo, etc.

La lógica implica que pueden ser dependientes del jugador que el jugador cumpla con condiciones específicas para que realicen una o más acciones basados en esta; mientras que las independientes hacen sus tareas asignadas sin la necesidad de que el jugador esté cerca o interactúe con la trampa; las tareas pueden abarcar desde obstaculizar, disminuir puntos de salud y hasta la eliminación inmediata del jugador.

- **Enemigo.** Implica un obstáculo al jugador. Habitualmente funciones con IA (inteligencia artificial) y cumple con tareas específicas, desde patrullar un área hasta obstaculizar de manera directa o indirecta a jugador, en ciertos casos causando daño. También pueden tener variación de estética y dificultad para mejorar la experiencia del jugador, en algunos casos se diferencian de menor a mayor dificultad con base en los puntos de salud y daño ocasionado al jugador pueden compartir el mismo diseño pero teniendo distintos colores, se les puede añadir más características a criterio del desarrollador.

- **Jefes.** Otro tipo de enemigo que puede llegar a tener una IA semejante a los enemigos promedios, pero pueden tener puntos de salud y daño mucho más altos así como funciones y habilidades que hagan competencia contra el jugador. Su uso frecuentemente es obstaculizar una fase esencial en el avance del jugador del siguiente nivel, adquisición de mejoras o habilidades, cualquier otra característica queda a discreción del desarrollador.
- **Inventario.** el nombre mismo indica un sistema que permite almacenar contenidos del juego para poder ser usados más adelante, usualmente los objetos almacenados sirven para recuperar salud, maná, llaves de puertas, etc.
- **La función de salto.** Realizando una acción (botón, asignación, evaluación, etc.), se aplica una velocidad vertical positiva al jugador con un valor de preferencia flotante y que al mismo tiempo con los valores de simulación de física y gravedad contrarresta poco a poco el ascenso y es llevado hacia abajo hasta que toque el suelo.
- **La función de ataque.** Al presionarse un botón o realizarse una acción se cambia el valor de una variable numérica o *booleana* y a partir de ahí se ejecuta la animación de ataque correspondiente a partir del tiempo de animación, habilitar la colisión y cálculo de daño para luego desactivarlo evitando errores.
- **Mejoras.** Elementos del juego que pueden llegar a mejorar permanente o temporalmente características o atributos del jugador para asistirlo en fases futuras o presentes en los niveles, usualmente al colisionarlos o usarlos se

cambian los valores de variables, la mayoría de tipo numérica por un nuevo valor en el que se destaquen los cambios en la jugabilidad.

- **Equipos.** Comparten la misma lógica de las mejoras pero se diferencian por el hecho de que el jugador decide si usarlos o retirarlos voluntariamente. Es usual devolverlos al inventario, sino simplemente se despojan.
- **Llaves y puertas.** Se entiende por puertas a un obstáculo en el avance del jugador hasta que se cumpla una condición para retirarla, usualmente verificando un valor numérico o *booleano* y pueden ser varios valores de verificación. La llave sirve para alterar el valor de dicha variable para entonces poder verificar en la puerta.
- **Lógica de los combos.** Se busca en la evaluación de variables booleanas y numéricas en el que dependiendo del número del combo del ataque y el tipo de ataque (golpe, patada, etc) realiza una secuencia específica de animación en el motor de UE4 sería un *blueprint* con funciones *branch* anidadas con las salidas, dependiendo de los valores ya sean verdadero o falso.
- **Lógica de los acertijos.** En muchos niveles existen estos obstáculos como reto hacia la capacidad cognitiva del jugador; la estética y lógica queda a criterio del desarrollador; pero el diseño de programación se asemeja al de las puertas, pero con más evaluaciones de variables con rangos y procedimientos específicos e incluso exhaustivos que al cumplirse las condiciones se hace la tarea correspondiente, ya sea para desbloquear obstáculos, ofrecer mejoras, o materiales de inventario, etc.

- **HUD.** Pantalla de Usuario Principal (Head User Display) (HUD), muestra al jugador elementos importantes en la pantalla relacionados con los videojuegos como puntos de vida, dinero, etc. En el caso del motor UE4 se usa un *widget blueprint* y se crean desde el texto a las barras de vida enlazadas a las variables deseadas y se editan estéticamente; luego se procede a enlazar las variables de interés con los elementos del *widget blueprint* a través de funciones
- **Ataques especiales.** También es posible anexar animaciones 2d y 3d dentro de este. El diseño estético queda a decisión del desarrollador.
- **Cinemáticas.** Conocidas en inglés como *cutscenes* son videos de variados formatos multimedia (también se puede hacer uso de la función *matinee* para hacer secuencias en el mismo juego), que son usados desde introducción, explicación de eventos o personajes y hasta finales del juego. Estos también pueden ser usados en estas imágenes, el uso es tan amplio como variado además de que el modo de reproducirlos en el juego comparte la lógica de las trampas por parte de la activación y el cómo o cuando usarlo queda a decisión del desarrollador.
- **Barreras.** Ampliamente usados en los niveles y escenarios; desde el punto de vista de la lógica y la programación debe existir límites en el juego, especialmente en los mencionados para su uso eficiente de procesamiento y memoria, también para evitar errores como caer por debajo del suelo o terminar más allá de los muros del nivel, quedando atrapado en la misma, atravesar elementos que no se deben, etc. En UE4 basta colocar objetos

físicos con la configuración de colisión habilitada, pero en caso de que quiera un límite con un escenario detallado también sirve poner paredes con la configuración de colisión de bloquear todo lo dinámico y alternar la visibilidad de tal forma que se haga invisible.

- **Portales.** Comparten ciertas características de las puertas y llaves con respecto a cómo abrirlas y que son un obstáculo para avanzar, pero en este caso sirven para la transición de un lugar a otro; pueden ser entre niveles, escenarios, mundos, etc.

Estéticamente varían los diseños, pero comparten la misma lógica en la que realizan la tarea al colisionar, interactuar con el jugador una vez se cumplan las condiciones establecidas por el desarrollador.

- **NPC.** Pueden tener o no IA para movilizarlos por los escenarios. No implica tener IA con dañar al jugador. Su función abarca desde informar al jugador hasta asistirlo para avanzar desde escenarios a niveles. Pueden tener o no puntos de vista como de daño, cualquier otra característica queda a decisión del desarrollador.
- **Proyectiles.** Su lógica se basa en objetos que se mueven con una velocidad y trayectoria, ya sea constante o dinámica (a decisión del desarrollador) sus usos abarcan estética, trampa, ataque especial, cinemática, etc.

MOTOR DE VIDEOJUEGO

Es una serie de rutinas de programación que permite la creación, edición de diseño y representación de un videojuego, tanto en consolas como computadoras con variados Sistemas Operativos y hasta plataformas móviles.

Su funcionalidad es proporcionar renderización para gráficos 2D y 3D simulaciones físicas, efectos, audios, videos, animaciones, *scripts*, inteligencia artificial, conexiones de redes, administración de recursos para ejecutarlo, etc. El desarrollo del videojuego varía por la reutilización o adaptación del mismo motor en la generación de otros juegos y dependiendo de los recursos y herramientas del motor se puede agilizar la producción y creación del videojuego.

Cada motor existente varía sus funciones, versatilidades, complejidad de su interfaz como implementación de funciones y sin agregar los recursos informáticos, pues dependiendo del motor varía desde el espacio en el disco duro hasta los requisitos mínimos de memoria RAM y el procesador para su correcto funcionamiento.

Algunos motores son gratuitos con versiones de pruebas o versiones básicas y ofrecen versiones más completas con funciones extras a cambio de pagos en cuotas en un periodo de tiempo determinado o pago completo, existen otras que son de uso exclusivo para las empresas que las desarrollaron y que muy rara vez la prestan a terceros.

Entre los más conocidos tenemos: Unity, CryEngine, Unreal Development Kit, MT Framework, Samaritan, Titanium, Rockstar Advanced Game Engine, Frostbite, Havok, Anvil Engine, 4A Engine, mercury engine y Unreal Engine 4.

HISTORIA DE UNREAL ENGINE

Es un motor de juego de PC y consolas creado por la compañía Epic Games, demostrado inicialmente en el *shooter* en primera persona Unreal en 1998, y es la base de juegos como Unreal Tournament, Deus Ex, Turok, Tom Clancy's Rainbow Six: Vegas, Gears of War, la saga de Bioshock, Star Wars Republic Commando, Batman: Arkham Asylum, SMITE, Mass Effect o Paragon. Ha sido utilizado en géneros como el rol y juegos de perspectiva en tercera persona. Hasta la fecha han salido cuatro versiones y sus sucesores han superado sus antecesores en funciones y gráficos.

Unreal Engine 1. Hizo su primera aparición en 1998, La primera generación Unreal Engine integraba renderizado, detección de colisiones, IA, visibilidad, opciones para redes y manipulación de archivos de sistema en un motor bastante completo. Epic usó este motor para los títulos Unreal y Unreal Tournament.

Unreal Engine 2. Esta versión salió en el 2002 pasando por una reescritura completa del código del núcleo y del motor de renderizado, además de integrar el nuevo UnrealEd 3. También incluyó el SDK de Karma physics. Muchos otros elementos fueron actualizados, con mejoras y agregando soporte para la PlayStation 2, GameCube y Xbox, y estos últimos fueron las consolas de mayor demanda en ese entonces.

En la versión Unreal Engine 2.5 fue mejorado el rendimiento y agregadas físicas para vehículos, editor de sistema de partículas para el UnrealEd y soporte de 64-bit en Unreal Tournament 2004. Además, en la versión especializada de Unreal Engine 2.5 llamada UE2X, se optimizaron características para la Xbox, agregando soporte

de efectos de sonido EAX 3.0, que fue usado, por ejemplo, para el juego Unreal Championship 2.

Unreal Engine 3. El Unreal Engine de tercera generación aparece en 2006, diseñado para PC con soporte DirectX 9/10, Xbox 360 y PlayStation 3. Su motor reescrito soporta técnicas avanzadas como HDRR, Mapeo Normal, y sombras dinámicas. Incluyendo componentes para herramientas complementarias al igual que las anteriores versiones del motor. Se sustituye a Karma por PhysX de Ageia (posteriormente adquirido por NVIDIA), y FaceFX se incluye además para generar animaciones faciales. Epic utilizó esta versión del motor para el videojuego Gears of War y Unreal Tournament 3, posteriormente utilizando una versión mejorada para Gears of War 2.

En el E3 de 2007 Sony anunció que se asociaba con Epic para optimizar el motor para el *hardware* del PlayStation 3, cambios que utilizan actualmente los desarrolladores de juegos de esta plataforma. Asimismo surge el anuncio del desarrollo de una modificación del Unreal Engine para la Wii. En la GDC de 2008, Epic revela numerosas mejoras de diseño, entre las que se incluyen: renderizado para mayor número de objetos simultáneos, físicas más realistas para efectos de agua, físicas de texturas corporales, mayor destructibilidad para los entornos, IA mejorada y efectos mejorados en luces y sombras con rutinas avanzadas para los shaders, el Unreal Engine 3, además se aplica en sectores no relacionados con los videojuegos como simulación de construcciones, simuladores de conducción, previsualización de películas y generación de terrenos.

El 5 de noviembre del 2009 Epic Games publicó una versión gratuita del Unreal Engine 3, el “Unreal Development Kit” para permitir a grupos de desarrolladores principiantes realizar juegos con el Unreal Engine 3.

Unreal Engine 4. Desde el 2 de marzo de 2015 está disponible para todo aquel que lo desee de forma gratuita, al igual que todas las actualizaciones que se lancen de él. Según se puede leer en el comunicado en la página oficial: "Puedes descargar el motor gráfico y usarlo para cualquier cosa en el desarrollo de videojuegos, educación, arquitectura, visualización de realidad virtual, cine y animación. Si cualquiera de estos proyectos se comercializa de forma oficial Epic Games obtendría el 5 % de los beneficios de la obra cada trimestre cuando este producto supere sus primeros 3000 dólares”.

Hasta la fecha ha sido popular por usuarios regulares y empresas prestigiosas en el desarrollo de videojuegos y animación, pues la disponibilidad gratuita e interfaz de usuario ha facilitado el desarrollo de los mismos y su mayor atractivo debe ser la creación de scripts visual a través de nodos facilitando la creación de contenido a nivel lógico y estético para no programadores y programadores; sin embargo, los últimos tendrán una ventaja de desarrollo en el uso de las variables a evaluar para mejorar la experiencia de los jugadores con detalles que serán colocados dentro del juego.

INSTALACIÓN DE UNREAL ENGINE 4

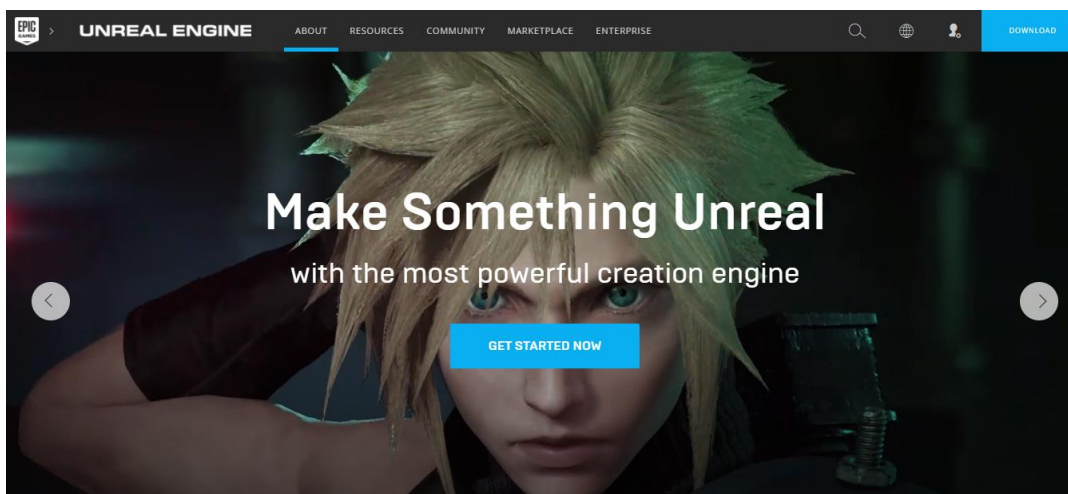
Antes de proceder se debe considerar los requisitos mínimos con el *hardware* para poder usar sin problemas el motor UE4

- S.O: Windows (7/8 64 bits) y Mac (macOS 10.XX) versión de 64 bits.
- CPU: Intel o AMD de 4 núcleos 2.5 GHz o superior.
- GPU: NVIDIA GeForce GTX 470 o AMD Radeon 6870 HD series, Metal 1.2 o tarjetas gráficas compatibles, equivalentes a 1GB VRAM.
- RAM: 8 GB en caso de Linux 16GB.

Para empezar el desarrollo práctico de videojuegos se debe proceder a la instalación del programa UE4; para eso se accede a través del explorador de internet de preferencia a la dirección.

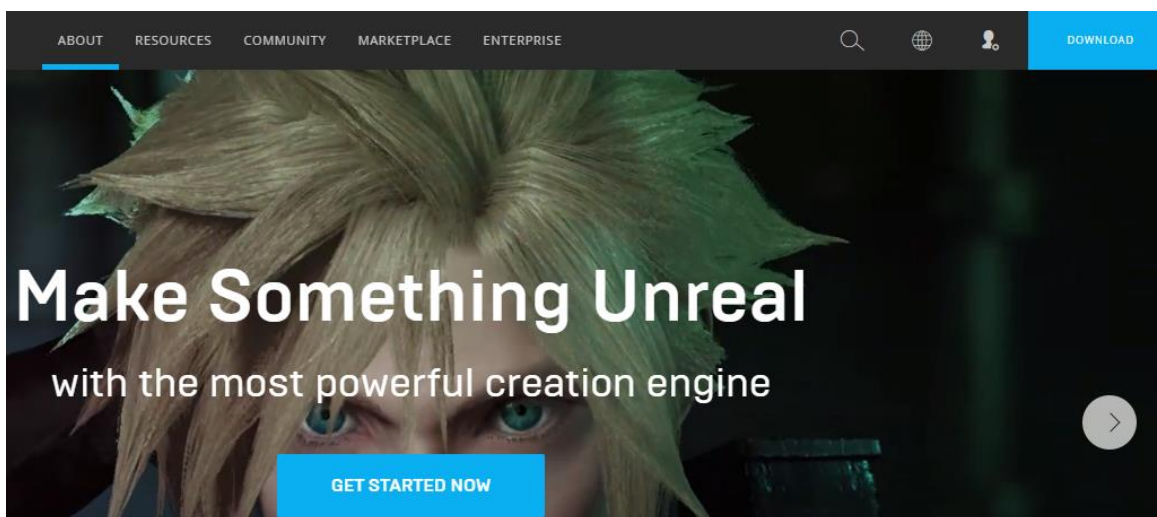
<https://www.unrealengine.com/en-US/what-is-unreal-engine-4>

Abrirá la página principal y mostrará la siguiente imagen que aparece a continuación.



(Games, 2017)

La página tiene a su disposición enlaces para entender mejor el motor y sus características, recursos como documentación detallada de sus características, funciones o video tutoriales, comunidad de usuarios para dudas y soluciones particulares de desarrollo, mercado para la compra y adquisición de *assets* y empresa para conocer a los desarrolladores del motor y noticias relacionadas con los mismos. Se procederá a ir al enlace de descarga al presionar el botón “Download”.



(Games, 2017)

Para proceder se debe crear un usuario de Epic Games, completando los datos o registrando de manera automática a través de la red social Facebook o Google+ y aceptando los términos y condiciones del motor, en caso de tenerlo ya creado se debe registrar.



Create your Epic Account

 SIGN IN

 SIGN IN

OR

PANAMA ▼

*FIRST NAME

*LAST NAME

*DISPLAY NAME ⓘ

*EMAIL

*PASSWORD

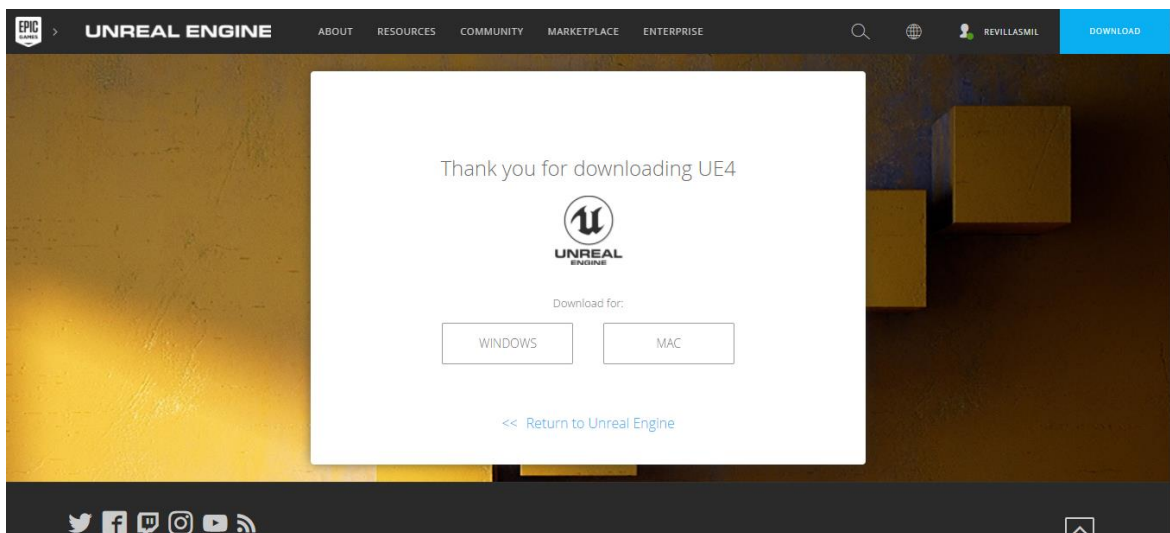
☐ I have read and agree to the [terms of service](#).

CREATE ACCOUNT

Have an Epic Games account? [Sign In](#)

(Games, 2017)

Al completarse el registro o autenticación de usuario aparecerá la página de descarga que dará la opción del asistente de instalación en el que se debe elegir dependiendo del sistema operativo, que hasta ahora solo hay disponible para Windows y MAC no existe hasta la fecha una versión para Linux.



(Games, 2017)

Thank you for downloading UE4



Download for:

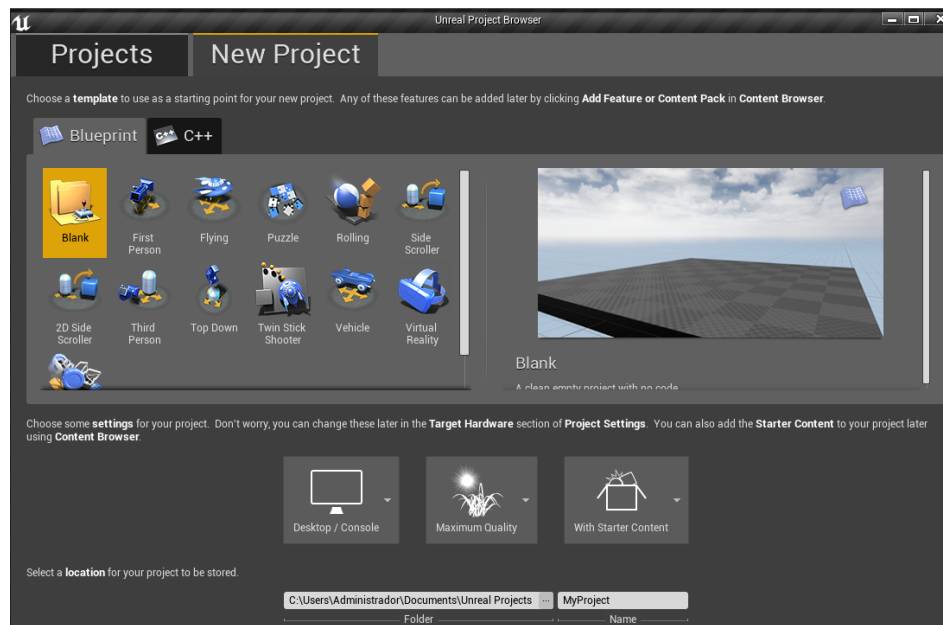
WINDOWS	MAC
---------	-----

<< [Return to Unreal Engine](#)

(Games, 2017)

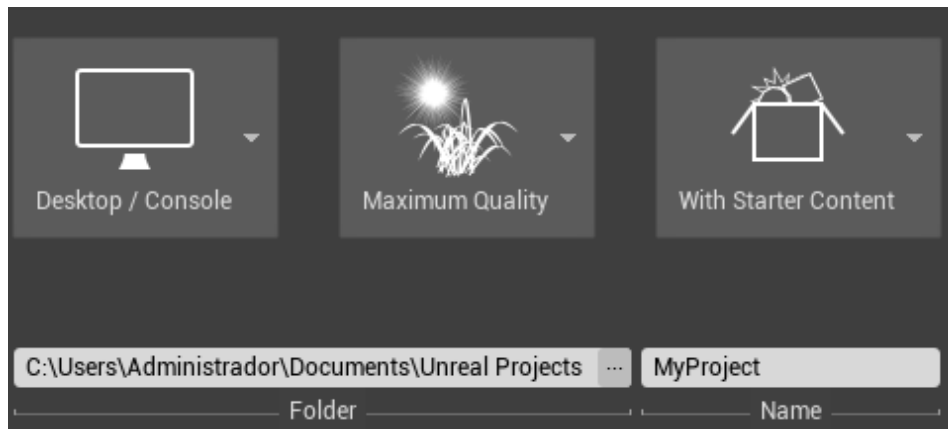
INTERFAZ DE UNREAL ENGINE 4

Al ejecutar UE4 están a disposición dos pestañas, una es para abrir los proyectos existentes y la otra es para crear proyectos como plantillas predeterminadas, cada una con características específicas en que se cubren los géneros más básicos de los videojuegos para desarrollarlos más adelante y agregar detalles que deseen los desarrolladores.



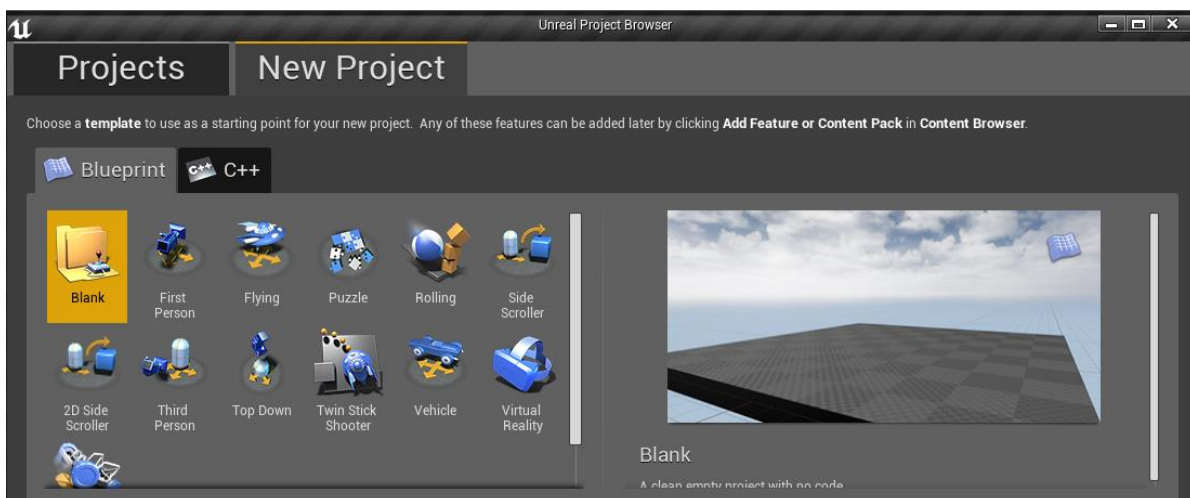
(Villasmil, 2018)

Más abajo se muestran las opciones de creación del proyecto; de izquierda a derecha la plataforma en la que va a estar destinada, la calidad de la imagen, el contenido de inicio (figuras, texturas, figuras y materiales de básicos que se incluye), debajo de este la ubicación y el nombre del proyecto que puede ser editado.



(Villasmil, 2018)

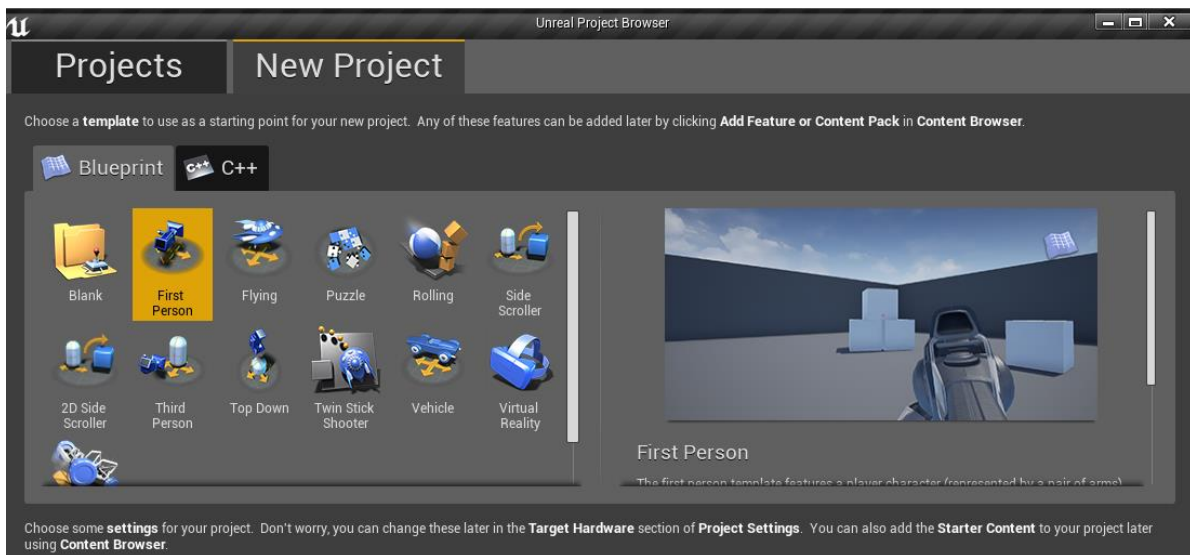
El proyecto en blanco está ausente de cualquier elemento, excepto un escenario básico y la iluminación del escenario, más allá de eso no contiene modelos 3d predeterminados para el jugador o estructuras simples como escaleras, o rampas.



(Villasmil, 2018)

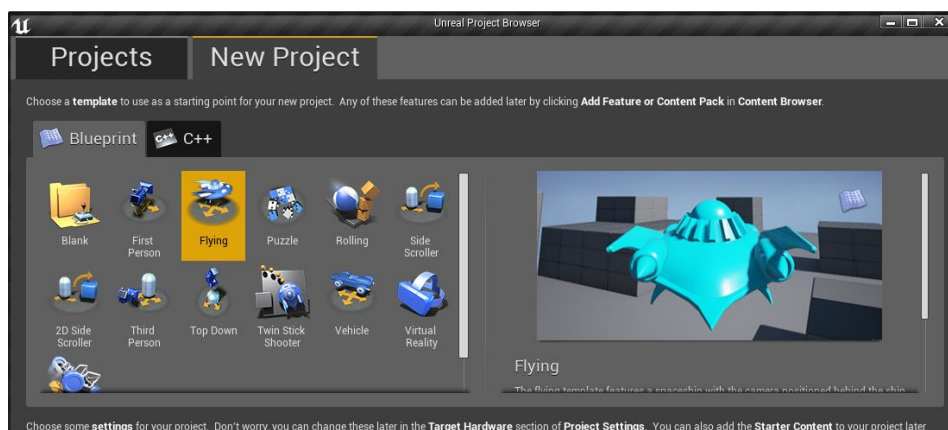
El proyecto de FPS tiene de manera predeterminada un escenario con paredes, cubos que poseen la capacidad de recibir colisión y simulación de física (inercia, gravedad, etc.), además de un personaje jugable con controles

establecidos a través del teclado con un modelo 3d de un arma que dispara pelotas como proyectiles que igualmente poseen simulación física.



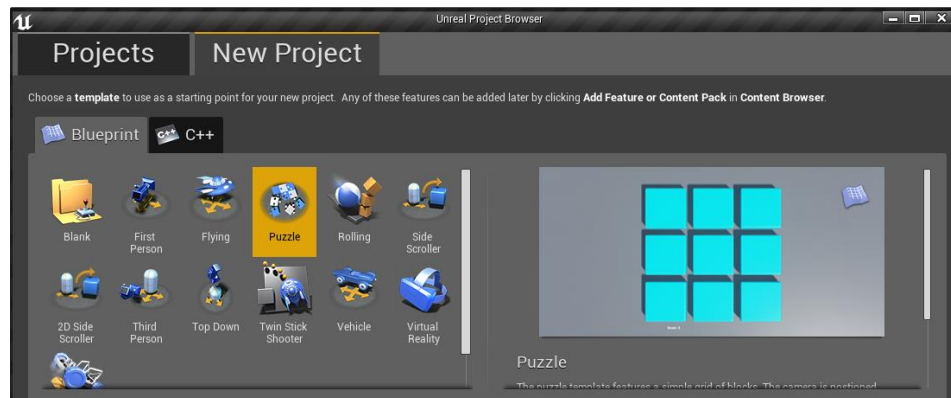
(Villasmil, 2018)

El proyecto de vuelo tiene de manera predeterminada un escenario básico con cubos como obstáculos y un modelo 3d de un vehículo predeterminado por el motor que además ya tiene asignado los controles por el teclado, y estará configurado para simular el estado de vuelo.



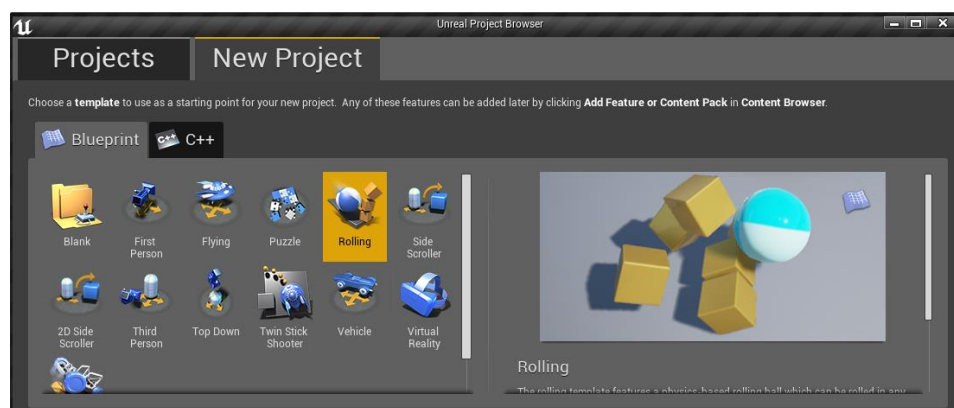
(Villasmil, 2018)

El proyecto de *puzzle* o rompecabezas o acertijo es parecido al proyecto de tercera persona, pero incluye unos cubos con lógica integrada que al contacto con el jugador cambiarán de color y el acertijo es que todos tengan el mismo valor *booleano* que implica una condición que debe llegar a ser verdadero.



(Villasmil, 2018)

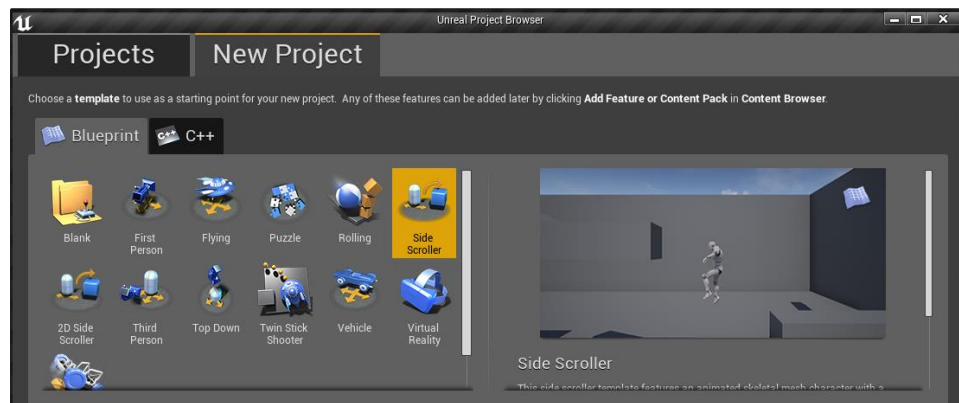
El proyecto de rotación tiene de manera predeterminada como jugador una pelota que se puede controlar a través del teclado y que simula física y los elementos dentro del área de juego con el que interactúe la pelota.



(Villasmil, 2018)

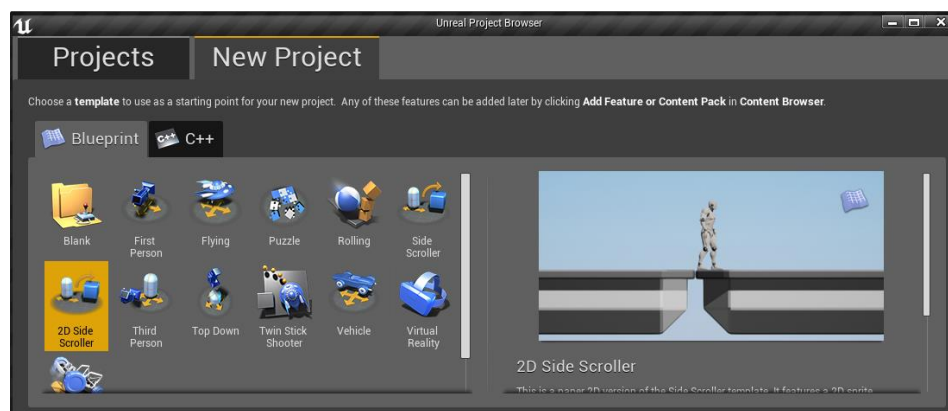
El proyecto de desplazamiento lateral o *side scroller* tiene de manera predeterminada una visión de cámara en un ángulo de lado a lado y el jugador que

se puede controlar solo se mueve de izquierda a derecha, así como saltar hacia arriba y descender, sirve perfectamente para el desarrollo de juegos en 2.5D.



(Villasmil, 2018)

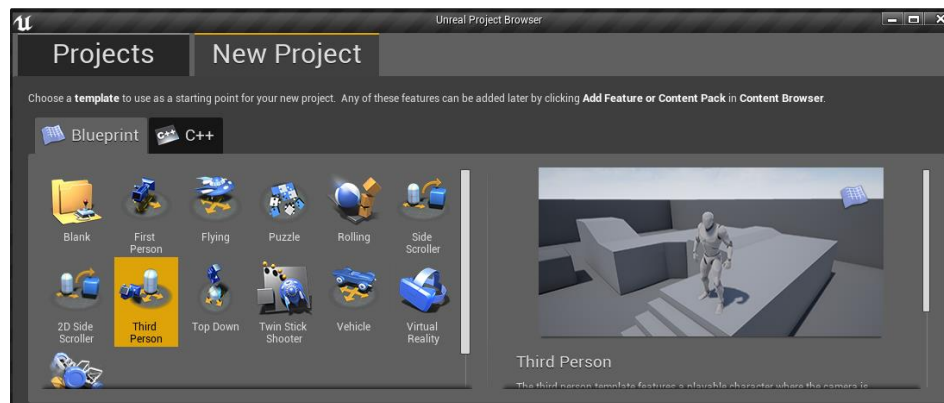
El proyecto de desplazamiento lateral 2D o 2D *side scroller* tiene de manera predeterminada una visión de cámara en un ángulo de lado a lado y el jugador que se puede controlar solo se mueve de izquierda a derecha, así como saltar hacia arriba y descender, se parece a la anterior pero está limitada por gráficos en dos dimensiones y el uso de *sprites*.



(Villasmil, 2018)

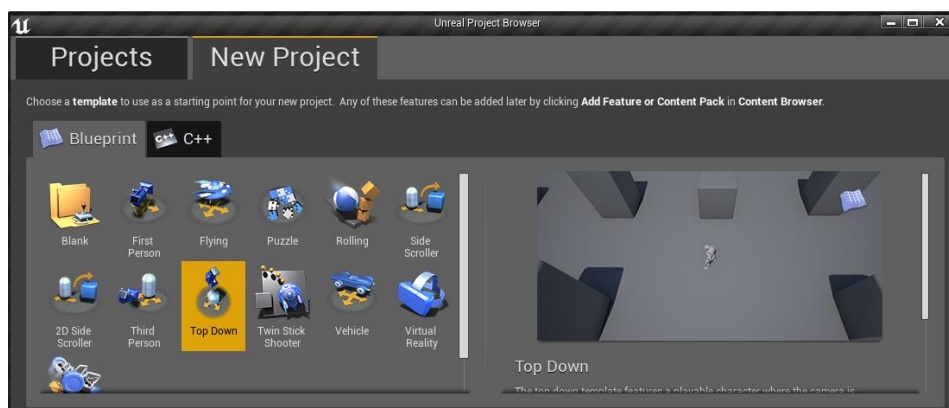
El proyecto de tercera persona consiste de controlar un personaje con una visión de una cámara que se puede mover alrededor, pero siempre enfocando al

personaje en una visión de tercera persona y este puede desplazarse en las tres dimensiones (x, y, z) y es capaz de saltar.



(Villasmil, 2018)

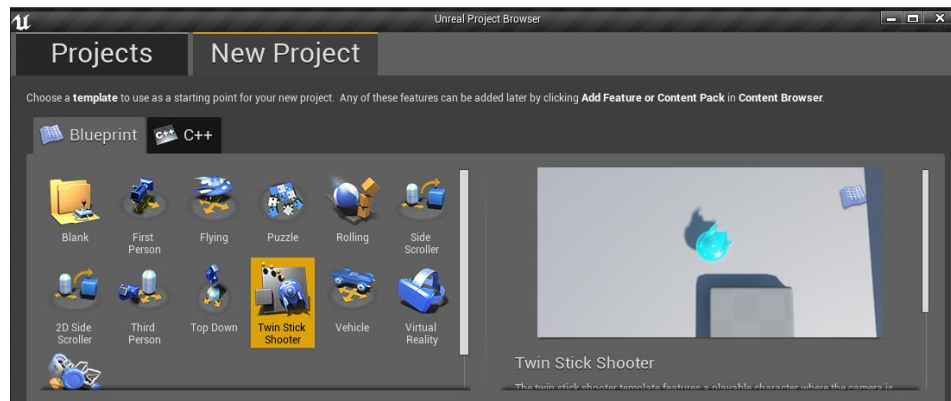
El proyecto de arriba hacia abajo comparte las características del de tercera persona a nivel de diseño y animación; sin embargo la cámara está en visión isométrica desde arriba del personaje y solo se puede mover automáticamente en la ubicación en la que hagas *click* a través del ratón de la computadora.



(Villasmil, 2018)

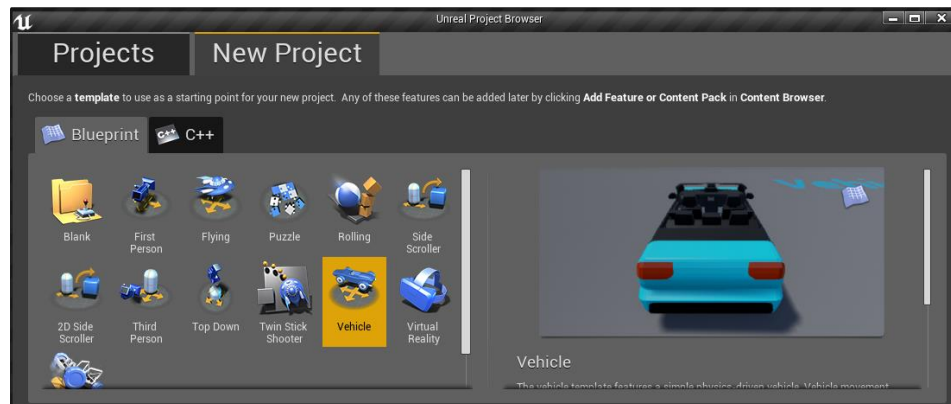
El proyecto de tirador con doble *stick* comparte la visión isométrica del anterior pero el personaje será el modelo de la nave que será controlado en tres partes el *joystick* izquierdo, la movilidad, el derecho para rotar el frente de la nave

para apuntar y la tercera es un botón designado para expulsar proyectiles en la dirección que se apunte.

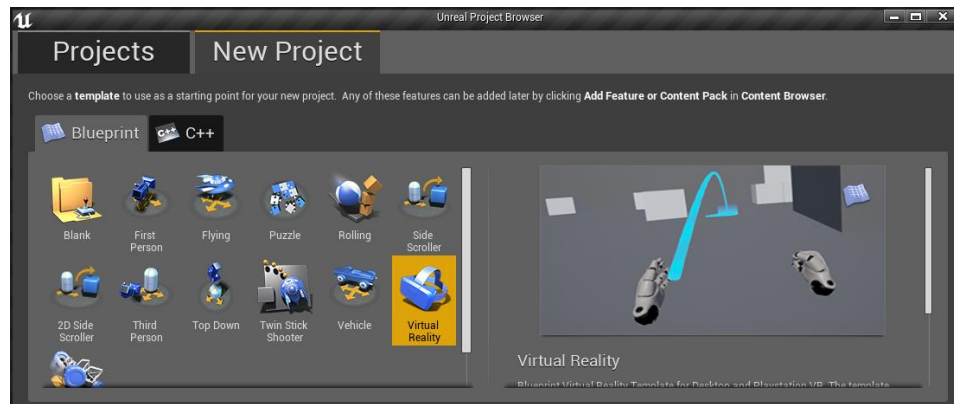


(Villasmil, 2018)

El proyecto de vehículo comparte los mismos elementos y características que el proyecto de tercera persona, sin embargo el jugador controlar un vehículo predeterminado con sus controles y movimientos ya asignados.

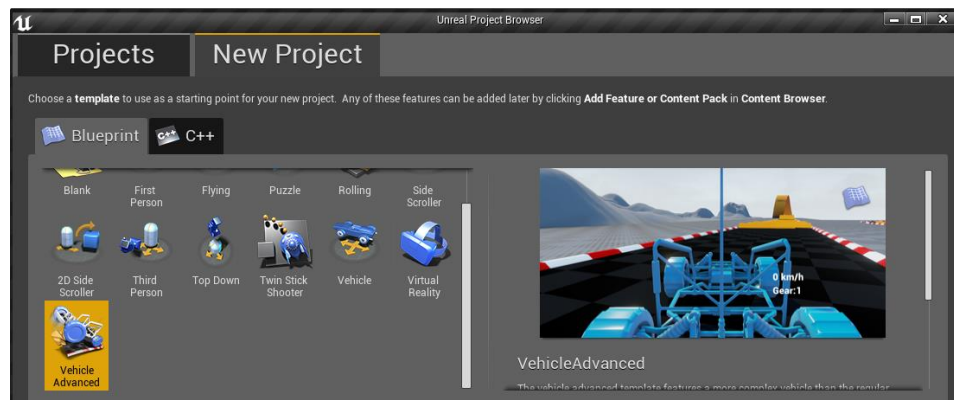


(Villasmil, 2018)



(Villasmil, 2018)

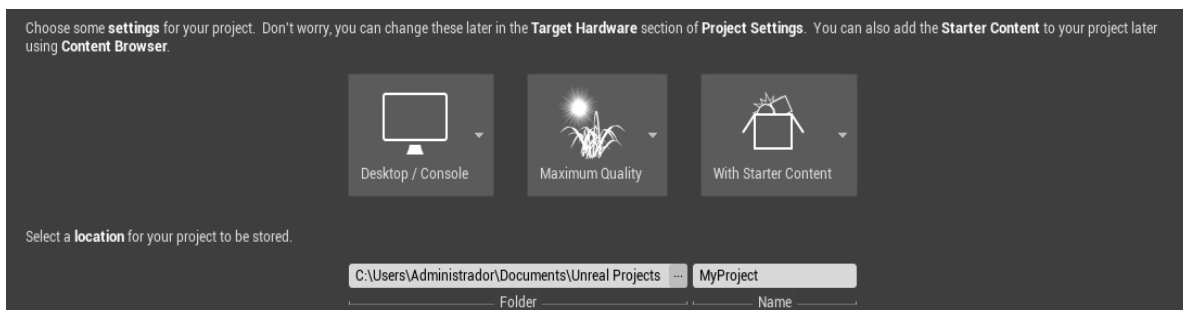
El proyecto de realidad virtual está con diseños y controles predeterminados para usar con las plataformas más populares de realidad virtual (Oculus Rift, HTC Vive, etc.), en el que incluye el ingreso de comandos a través de los periféricos de preferencia.



(Villasmil, 2018)

El proyecto de vehículo avanzado funciona casi igual al de vehículo, pero tiene una perspectiva en primera persona y más controles con respecto al manejo del vehículo, así como simulación de física para la interacción del vehículo con el entorno y los obstáculos.

Más abajo en la ventana del nuevo proyecto aparecen cinco elementos para el desarrollo del proyecto y tienen que ver con el *hardware* objetivo del proyecto que puede ser escritorio/consola o móvil/Tablet, la calidad gráfica que puede ser máxima calidad o escalable 3D/2D, usar o no el contenido inicial que consiste en activos del motor predeterminados para el desarrollo del proyecto que incluye una variedad de figuras, texturas, materiales, partículas y muchos otros más activos ya desarrollados por los desarrolladores de Unreal Engine 4 listos para aplicarse, la ubicación de destino de la carpeta del proyecto dentro de la computadora y que de manera predeterminada se ubica en la carpeta de proyectos de Unreal en Documentos pero que puede ser cambiada a decisión del usuario y por último p,ero no menos importante, el nombre que tendrá nuestro proyecto a libre elección del usuario.



(Villasmil, 2018)

HERRAMIENTAS DEL EDITOR

El editor cuenta con una interfaz en parte gráfica y otra textual.



(Value Coders, 2011)

Barra de pestañas

El Editor de niveles tiene una pestaña en la parte superior que proporciona el nombre del nivel que se está editando actualmente. Las pestañas de otras ventanas del editor se pueden acoplar junto con esta pestaña para una navegación rápida y fácil, similar a un navegador web.



(Games, Level Editor, 2014)

El nombre de la pestaña reflejará el nivel que se está editando actualmente. Este es un patrón consistente en todo el editor: las pestañas del editor se nombrarán después de editar el activo actual. A la derecha de la barra de pestañas se encuentra el nombre del proyecto actual.

Barra de menús

La barra de menú en el editor debería ser familiar para cualquiera que haya usado previamente aplicaciones de Windows. Proporciona acceso a herramientas y comandos generales que se utilizan al trabajar con niveles en el editor.



(Games, Level Editor, 2014)

File (Archivo)

Cargar y guardar

Comando	Descripción
Nuevo nivel	Crea un nuevo nivel, o escoge una plantilla de nivel para empezar.
Abrir un nivel	Carga un nivel existente.
Guardar	Guarda el nivel actual en el disco duro.
Guardar como	Guarda el mapa persistente actual usando el nombre dado.
Guardar todos los niveles	Guarda todos los niveles cargados no guardados, incluidos los niveles de transmisión.
Abrir activo	Invoca a un selector de activos.
Guardar todo	Guarda todos los niveles y activos no guardados.
Elija archivos para guardar	Abre un cuadro de diálogo con opciones de guardado para niveles y contenido.
Presentar al control de la fuente	Abre un diálogo con opciones de registro de contenido y niveles.

Proyecto

Comando	Descripción
Nuevo proyecto	Abre un diálogo para crear un nuevo proyecto de juego.
Abrir Proyecto	Abre un cuadro de diálogo para elegir un proyecto de juego para abrir.
Agregar código al proyecto	Agrega código C ++ al proyecto. El código solo puede compilarse si tiene instalado Visual Studio.
Cocinar Contenido	Cocina el contenido de su proyecto para una plataforma específica. Esto es útil si ejecuta un proyecto basado en código desde Visual Studio.
Empaquetar Proyecto	Compila, cocina y empaca el proyecto y su contenido para su distribución.
Abrir Visual Studio	Abre el código de C ++ en Visual Studio.
Actualizar el proyecto de Visual Studio	Actualiza el proyecto de código C ++ en Visual Studio.

Actores

Comando	Descripción
Importar al nivel	Importa objetos y actores desde un formato fbx y T3D al nivel actual.
Exportar todo	Exporta todo el nivel a un archivo en el disco (se admiten múltiples formatos).
Exportar seleccionados	Exporta los objetos seleccionados actualmente a un archivo en el disco (se admiten múltiples formatos).

Aplicación

Comando	Descripción
Agregar nivel a los favoritos	Establece si el nivel cargado actualmente aparecerá en la lista de niveles favoritos.
Niveles recientes	Selecciona un nivel para cargar.
proyectos recientes	Muestra una selección de un proyecto antes visto en orden de apertura para abrirlo
Salida	Cierra la aplicación

Edit (Editar)**Historia**

Comando	Descripción
Deshacer	Deshace la última acción completada.
Rehacer	Rehace la última acción que decidió deshacer.
Historial de deshacer	Muestra el listado de elementos desechados del proyecto.

Editar

Comando	Descripción
Cortar	Cortar selección.
Copiar	Copiar selección.
Pegar	Pegar contenido del portapapeles.
Duplicar	Selección duplicada.
Borrar	Eliminar la selección actual.

Configuración

Comando	Descripción
Preferencias de editor	Configura el comportamiento y las funciones del Editor de Unreal.
Configuraciones de proyecto	Cambia las configuraciones del proyecto actual.
Extensiones	Abre la pestaña de exploración de extensiones.

Window (Ventana)

Editor de nivel

Comando	Descripción
Detalles	Muestra el panel Detalles, que proporciona acceso a varias propiedades para objetos seleccionados. Puede mostrar hasta 4 paneles de detalles simultáneamente.
Puertos de vista	Muestra la ventana gráfica, que es su vista en el mundo 3D de su juego. Puede mostrar hasta 4 ventanas simultáneamente.

Lineado externo jerárquico LOD	Esto mostrará la configuración de HLOD y permitirá la generación de “clusters”.
Capas	Abre el panel Capas, donde puede dividir su nivel en una serie de capas de visibilidad.
Modos	Muestra el panel Modos, que accede a las herramientas de edición de niveles especiales, como Paisaje y Follaje.
Lineado externo Mundial	Abre el Lineado externo Mundial, un desglose jerárquico de todos los actores actualmente en el nivel.
Estadísticas	Muestra el panel Estadísticas, que proporciona acceso a una variedad de estadísticas de rendimiento para el nivel.
Barra de herramientas	Muestra la barra de herramientas, que contiene una variedad de funciones comunes que realizará durante la edición de niveles.
Configuración mundial	Muestra el panel Configuración mundial, donde puede encontrar

	configuraciones específicas para el nivel que está editando actualmente.
--	--

General

Comando	Descripción
Navegador de contenido	Abre cualquiera de hasta 4 navegadores de contenido para trabajar con activos de contenido.
Herramientas de desarrollo	Proporciona acceso a varias herramientas de desarrollo que son útiles para los programadores que trabajan con el motor y el editor.
Encontrar en planos	Encuentra referencia a funciones, eventos y variables en todos los planos.
Lanzador de proyecto	Provee flujos de trabajos avanzados para empaquetar, desplegar y lanzar los proyectos.

Experimental

Comando	Descripción
Panel de localización	Abre el panel de localización

Diseño

Comando	Descripción
Restablecer diseño	Realiza una copia de seguridad de la configuración de usuario y restablece las personalizaciones de diseño.
Guardar diseño	Guarda el diseño de la interfaz de usuario actual.
Habilitar pantalla completa	Alterna el editor entre la pantalla completa y los modos normales.

Help (Ayuda)

Explorar

Comando	Descripción
Documentación	Abre la página principal de documentación.
Referencia de API	Abre la documentación de referencia de la API.
Variables de consola	Abre la ventana gráfica de controles de ventana gráfica.
Controles de vista	Abre la ventana de controles de la hoja de trucos.

Tutoriales

Comando	Descripción
Tutoriales	Tutoriales introductorios que cubren los conceptos básicos del motor Unreal Engine 4.

Online

Comando	Descripción
Soporte	Navega hacia la página principal de soporte de Unreal Engine 4.
Foros	Te lleva a los foros de Unreal Engine 4 para ver anuncios y participar en discusiones con otros desarrolladores.
Centro de respuestas	Lleva al centro de respuestas para realizar preguntas, buscar respuestas existentes y compartir conocimiento con otros desarrolladores de Unreal Engine 4.
Wiki	Lleva a la página <i>wiki</i> de Unreal Engine 4 para ver los recursos creados por la comunidad o crear tus propios recursos.
Visita UnrealEngine.com	Navega a UnrealEngine.com para aprender más de la tecnología del motor.

Créditos	Despliega los créditos de la aplicación.
Sobre el editor Unreal	Despliega los créditos de la aplicación e información de derechos de autor.

Barra de herramientas

El panel de la barra de herramientas, como en la mayoría de las aplicaciones, es un grupo de comandos que proporciona acceso rápido a herramientas y operaciones de uso común.



(Villasmil, 2018)

Comando	Descripción
Guardar (<i>Save</i>)	Guarda el nivel actual.
Control de fuente (<i>Source Control</i>)	Le permite conectarse o asignar su solución de control de fuente.
Contenido (<i>Content</i>)	Abre el Navegador de contenido que muestra todos los contenidos en su juego. Aquí es donde vas a crear, importar y editar todo el contenido.
Mercado (<i>Marketplace</i>)	Abre el Unreal Engine Launcher en la sección Marketplace.
Configuración (<i>Settings</i>)	Muestra el menú de configuración que proporciona un acceso fácil a las

	opciones de uso común que controlan los aspectos de selección, edición y vista previa del Editor de niveles.
Planos (<i>Blueprints</i>)	Proporciona acceso para crear o editar cualquier <i>blueprints</i> en el mundo, incluida la apertura del <i>blueprint</i> de nivel para el nivel actual en <i>blueprint</i> Editor. Este menú le brinda acceso rápido a la configuración del marco para su juego (reglas del juego, tipo de jugador, HUD, etc.) desde el editor.
Cinemáticas (<i>Cinematics</i>)	Le permite crear una nueva secuencia de <i>matinee</i> o editar cualquier secuencia de <i>matinee</i> existente en el nivel.
Construir (<i>Build</i>)	Realiza una operación de compilación en todos los niveles (persistente y de transmisión) abiertos en el editor. La construcción precalcula tantos datos como sea posible relacionados con varios aspectos del nivel. Por ejemplo, la iluminación estática (mapas de luz, sombras, iluminación global) y la geometría se calculan durante este

	proceso. Al hacer <i>click</i> en la flecha, aparece el menú Opciones de compilación.
Reproducir (<i>Play</i>)	Inicia el juego en el modo de juego normal. Al hacer <i>click</i> en la flecha, aparece el menú Opciones de reproducción. Vea la sección Reproducir en el Editor para más información.
Lanzar (<i>Launch</i>)	Inicia el mapa actual en cualquiera de las plataformas compatibles. Los dispositivos conectados se enumeran en el menú al que se accede haciendo <i>click</i> en la flecha.

Controles

En el motor de videojuegos Unreal Engine existen controles con comportamientos predeterminados para desarrollar y navegar los proyectos así como sus componentes de manera fácil e intuitiva.

Leyenda: BMI= Botón Ratón Izquierdo, BMD= Botón Ratón Derecho

Controles	Acciones
Controles de Perspectiva	
BMI + arrastrar ratón	Mueve la cámara adelante, atrás y rota hacia la izquierda y derecha.
BMD + arrastrar ratón	Rota la cámara de vista en la dirección que sea arrastrado el ratón.
BMI + BMD + arrastra ratón	Desplazarse arriba y abajo.
Ortográfica (superior, frontal, lateral)	
BMI + arrastrar ratón	Crea un cuadro de selección de marquesina.
BMD + arrastrar ratón	Desplaza la cámara de la ventana gráfica.

BMI + BMD + arrastra ratón	Acerca la cámara de la ventana gráfica dentro y fuera.
Enfoque	
F	Enfoca la cámara en el objeto seleccionado. Esto es esencial para aprovechar al máximo el desplazamiento de la cámara.

Controles a través de teclado WASD

W Numpad8 Up	Mueve la cámara hacia adelante.
S Numpad2 Down	Mueve la cámara hacia atrás.
A Numpad4 Left	Mueve la cámara hacia la izquierda.
D Numpad6 Right	Mueve la cámara hacia la derecha.
E Numpad9 Page Up	Mueve la cámara hacia arriba.
Q Numpad7 Page Dn	Mueve la cámara hacia abajo.
Z Numpad1	Enfoca la cámara hacia afuera (aumenta el campo de visión).
C Numpad3	Enfoca la cámara hacia adentro (disminuye el campo de visión).

Para cambiar a una vista ortográfica, haga *click* en el menú desplegable Perspectiva y luego seleccione un modo de vista ortográfica.



(Villasmil, 2018)

Modos (*Modes*)



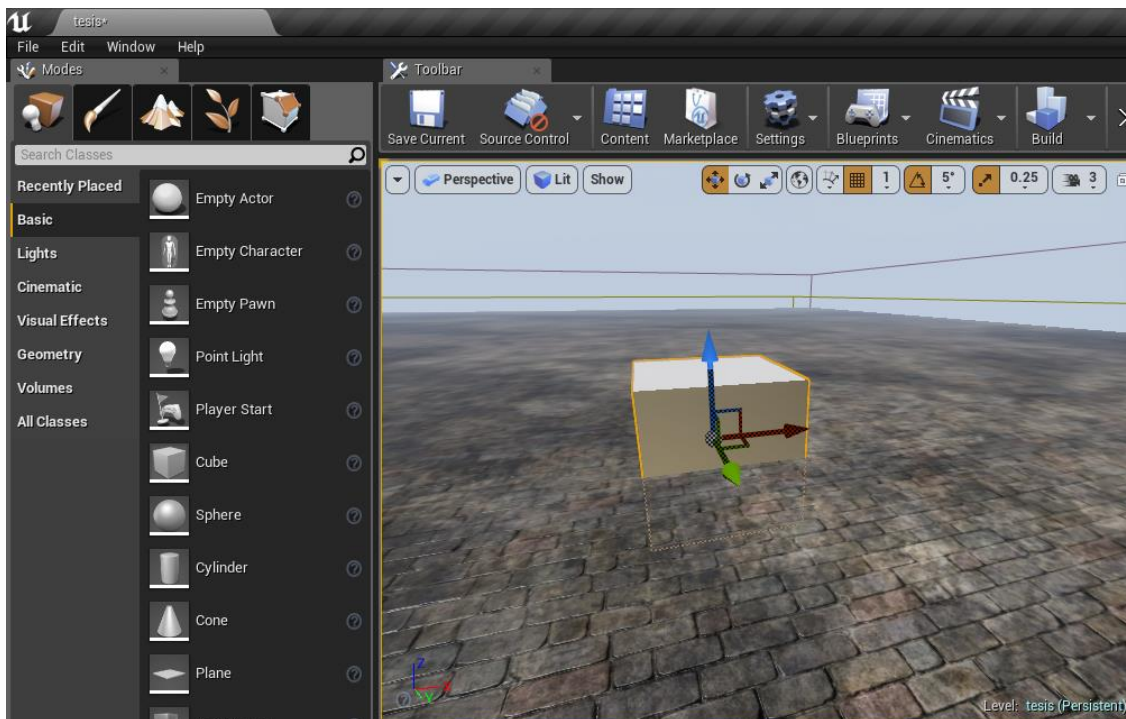
(Villasmil, 2018)

En esta sección de la interfaz se controla todo lo que tenga que ver con manipulación de posicionamiento de todas las clases que existen en el motor, abarcando desde los modelos 3D hasta los variados planos y se divide en cinco funciones:

Colocar (*Place*)

El modo de colocar es una herramienta especializada para acelerar y simplificar la creación de entornos en Unreal Engine 4. De forma similar al

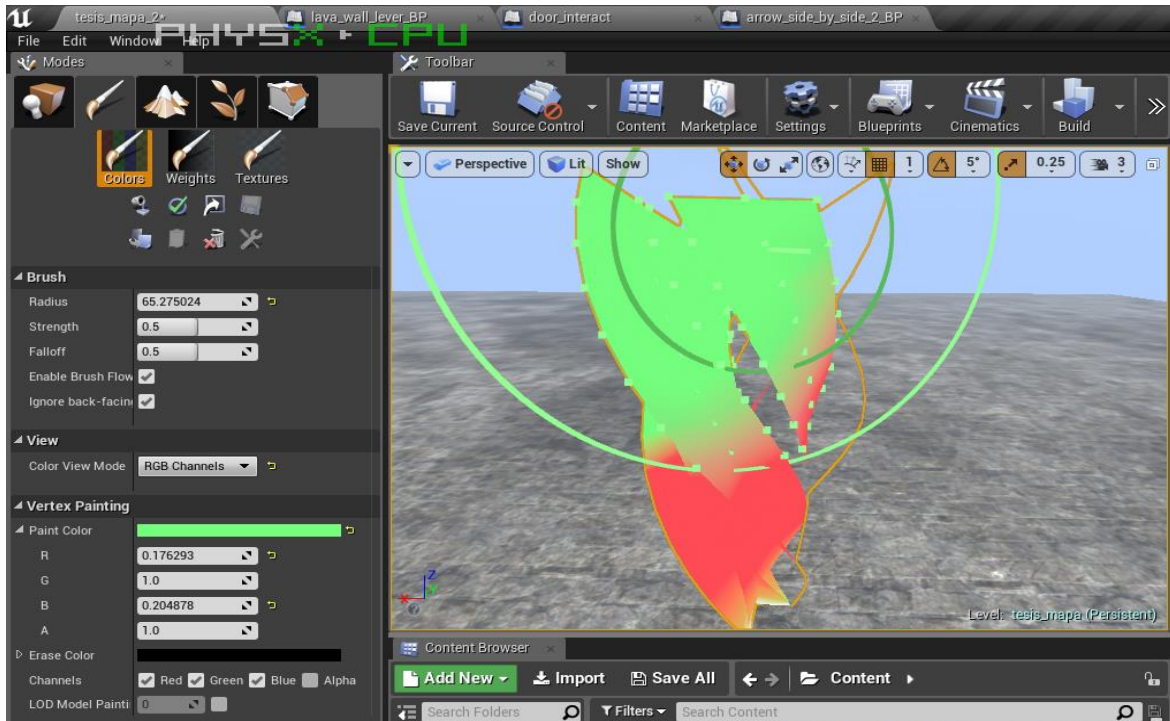
Navegador de contenido, el Modo de colocar se centra únicamente en los activos que se pueden colocar directamente dentro de su nivel (es decir, Actores). Mientras está activo, este navegador le brinda acceso inmediato a todos los objetos que pueden ubicarse dentro de su proyecto, sin tener que navegar a carpetas de proyecto específicas como lo haría con el Navegador de contenido.



(Villasmil, 2018)

Pintar (Paint)

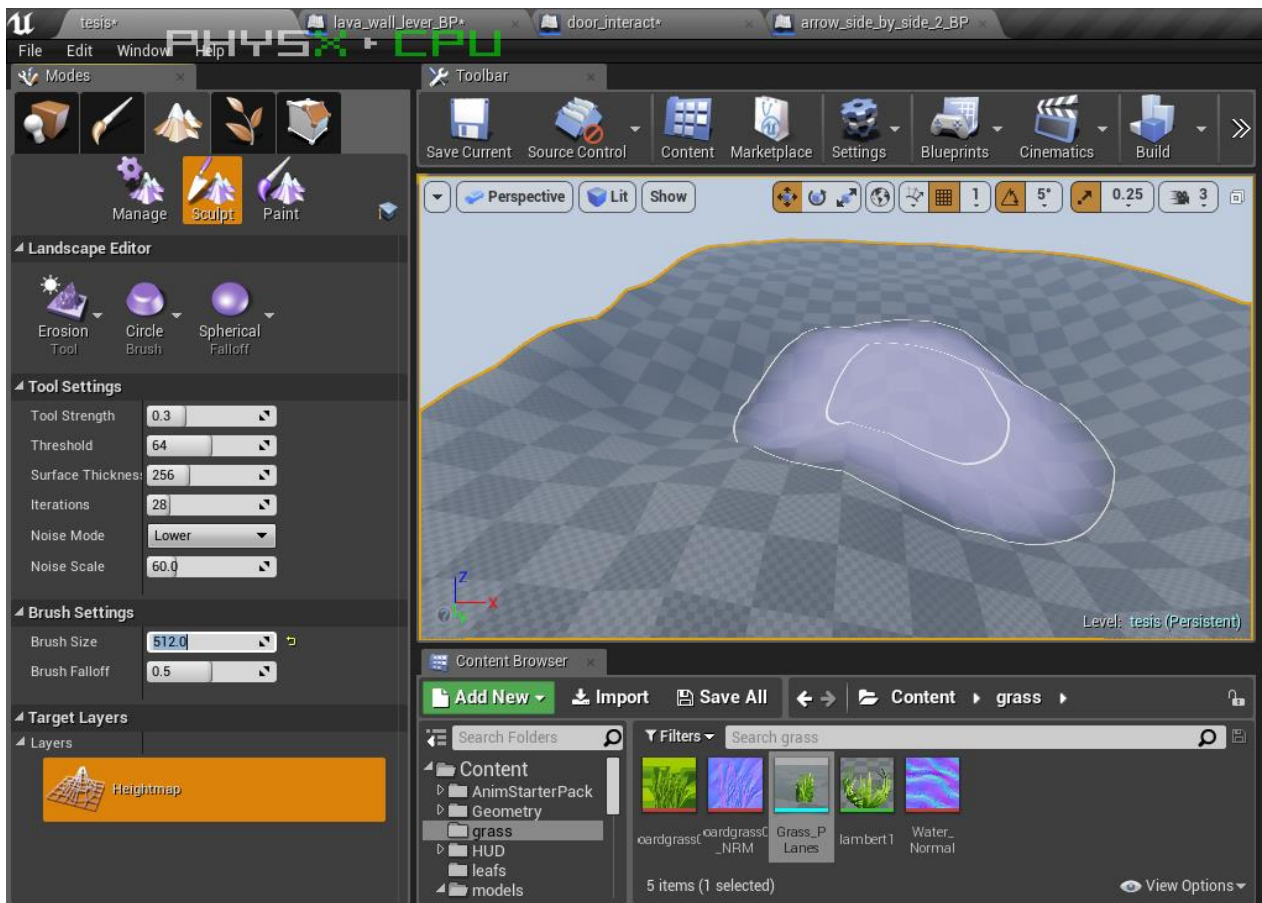
El modo de malla de pintura proporciona una manera rápida y fácil de ajustar el color y la textura en sus mallas. Las siguientes secciones cubren todas las habilidades críticas que necesitará para sacar el máximo provecho de esta poderosa herramienta.



(Villasmil, 2018)

Paisaje (*Landscape*)

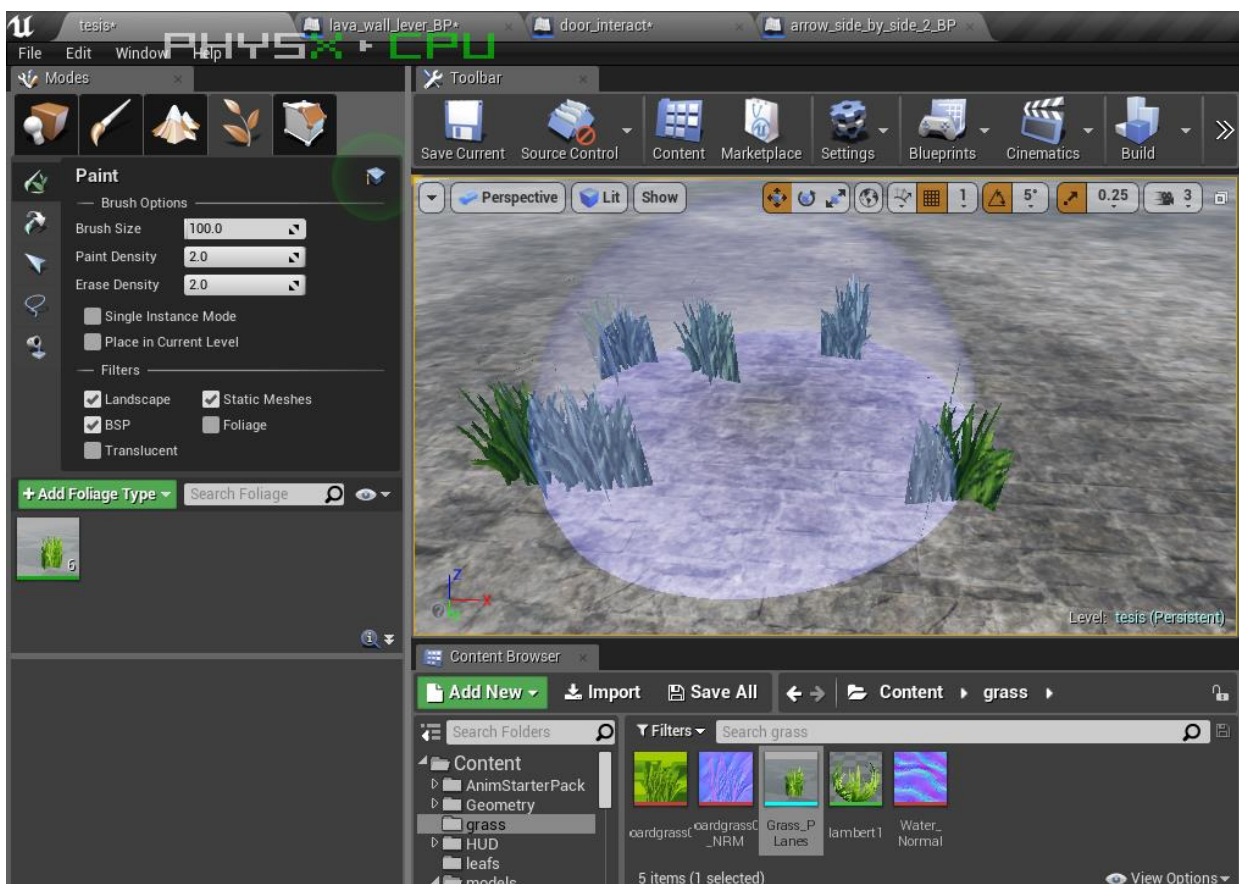
Alterna el modo Paisaje para editar terrenos de paisajes en el escenario. El sistema de paisaje le permite crear un terreno para su mundo (montañas, valles, terreno desigual o inclinado, incluso aberturas para cuevas) y modificar fácilmente tanto su forma como su apariencia mediante el uso de una gama de herramientas.



(Villasmil, 2018)

Follaje (*Foliage*)

Alterna el modo Follaje para pintar follaje de instancias. El sistema de mallaa de follaje instanciado le permite pintar o borrar rápidamente conjuntos de mallas estáticas en actores de paisaje, otros agentes de malla estática y geometría BSP. Estas mallas se agrupan automáticamente en lotes que se representan utilizando instancias de *hardware*, lo que significa que muchas instancias se pueden representar con una sola llamada a un solo dibujo.

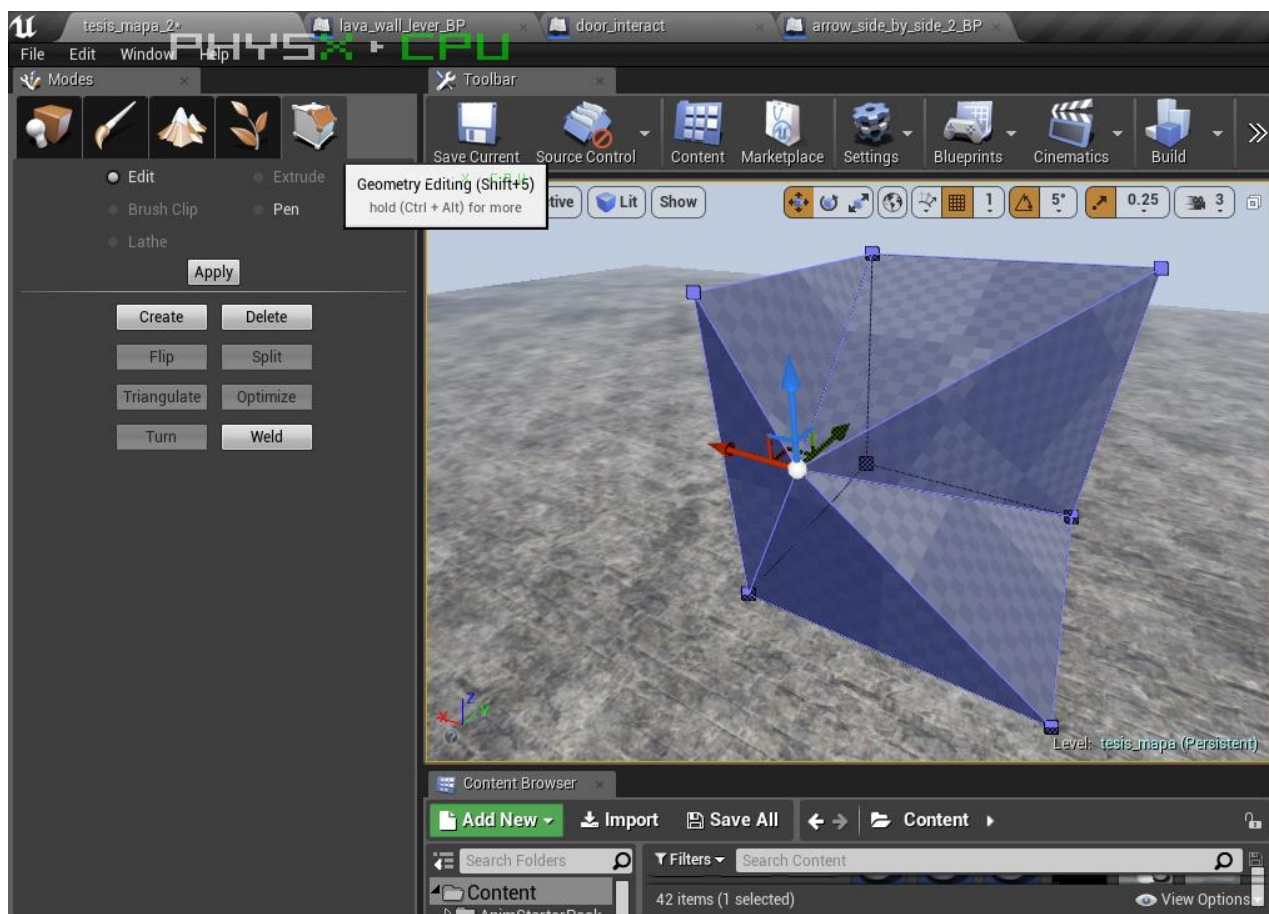


(Villasmil, 2018)

Edición de geometría (*Geometry Editing*)

La mejor forma de cambiar la forma real de un Brush es usar el Modo de Edición de Geometría. Este modo de editor permite la manipulación directa de los

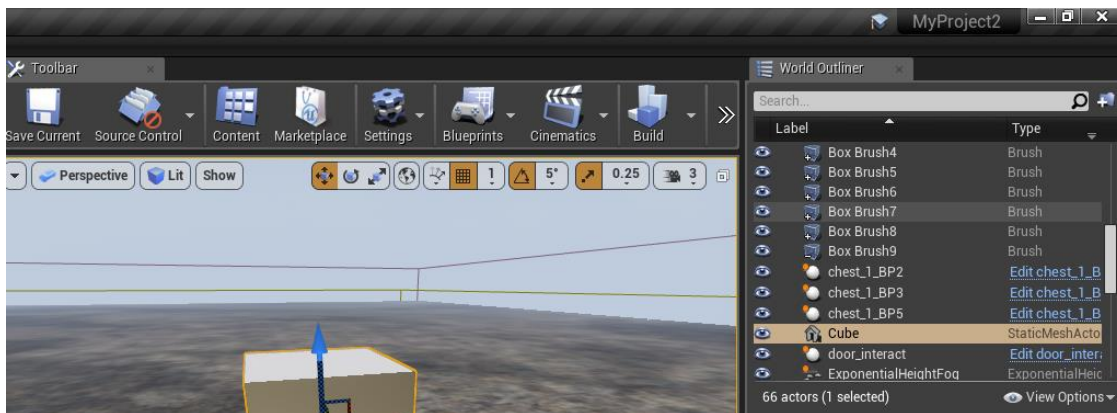
vértices, los bordes y las caras de este. Es muy similar a trabajar en una aplicación de modelado 3D muy simplificada.



(Villasmil, 2018)

Trazado de línea mundial (*World Outliner*)

Este muestra todos los actores dentro de la escena en una vista de árbol jerárquica. Los actores se pueden seleccionar y modificar directamente desde su panel. También puede usar el menú desplegable de Información para activar una columna adicional que muestre niveles, capas o nombres de ID.



(Villasmil, 2018)

Navegador de contenido (*Content Browser*)

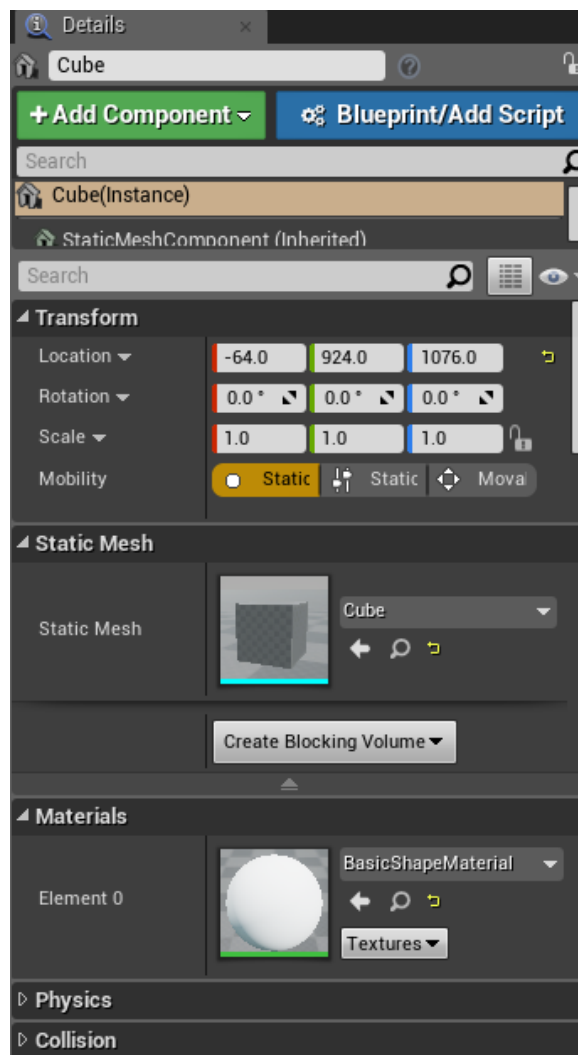
El Navegador de contenido es el área principal de Unreal Editor para crear, importar, organizar, ver y modificar contenidos dentro de Unreal Editor. También proporciona la capacidad de administrar carpetas de contenido y realizar otras operaciones útiles en activos, como cambiar el nombre, mover, copiar y ver referencias. El Navegador de contenido puede buscar e interactuar con todos los recursos del juego.



(Villasmil, 2018)

Detalles (*Details*)

El panel Detalles contiene información, utilidades y funciones específicas de la selección actual en la ventana gráfica. Contiene cuadros de edición de transformación para actores en movimiento, rotación y escala, muestra todas las propiedades editables para los actores seleccionados y proporciona acceso rápido a la funcionalidad de edición adicional según el tipo de Actor (es) seleccionado en la ventana gráfica.



(Villasmil, 2018)

PROGRAMACIÓN POR NODOS

Muchos motores hasta la fecha requieren de un conocimiento intermedio hasta avanzado en programación, tanto por la lógica como el lenguaje de programación en el que esté compuesto como C#, C++, Javascript e incluso html.

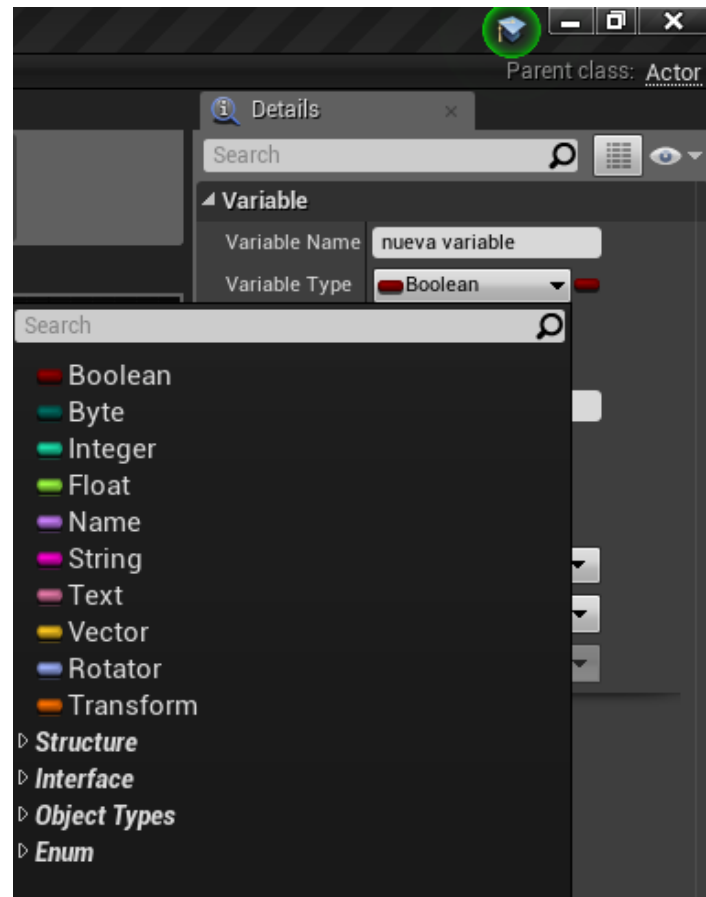
La programación de controles de eventos y diseño del juego a través de programación línea por línea puede llegar a ser largo y tedioso para un equipo de desarrolladores y aún más para un solo integrante sin embargo, Epic Games en su motor Unreal Engine 4 añadieron la características de los “Blueprints”(Planos), que consiste en programar a través de script visual usando nodos gráficos que permiten diseñar más rápido la programación de la que estará compuesta el proyecto sin la necesidad de escribir el código ayudando no solo a los que no tienen conocimiento de programación, también a los programadores a escribir, editar y corregir sus ideas en el proyecto, de tal manera que se ahorra mucho tiempo en comparación con la programación tradicional. También está la ventaja de que los *blueprints* son implementados en lenguaje C++ y pueden ser visualizados en línea de código.

Tipos de nodos

Los nodos están definidos en muchas variables y funciones prediseñadas que ayudan en el desarrollo y análisis en tiempo real de las ideas aplicadas en los *blueprints* dentro del proyecto. Las variables se diferencian entre ellas tanto por color como uso. En la siguiente tabla se especifica sus usos:

Tipo de variable	Color	Usos
Boolean (Booleano)	Rojo	Las variables rojas representan datos <i>booleanos</i> (verdadero / falso).
Entero (Integer)	Cian	Las variables azul celeste representan datos enteros, o números sin decimales como 0, 152 y -226.
Flotante (Float)	Verde	Las variables verdes representan datos flotantes, o números con decimales, tales como 0.0553, 101.2887 y -78.322.
Cadena (String)	Magenta	Las variables magentas representan datos de cadena de texto, o un grupo de caracteres alfanuméricos como “Hello World 1@3”.
Texto (Text)	Rosa	Las variables de color rosa representan los datos de texto mostrados, especialmente cuando ese texto es consciente de la localización.
Vector	Oro	Las variables de oro representan datos vectoriales, o números que constan de

		números flotantes para tres elementos o ejes, como información XYZ o RGB.
Rotador (Rotator)	Púrpura	Las variables púrpuras representan datos de Rotator, que es un grupo de números que definen la rotación en el espacio 3D.
Transformación (Transform)	Naranja	Las variables de color naranja representan datos de transformación, que combina la traducción (posición 3D), la rotación y la escala.
Objeto (Object)	Azul	Las variables azules representan objetos, incluidas luces, actores, muros estáticos, cámaras y canales de sonido.

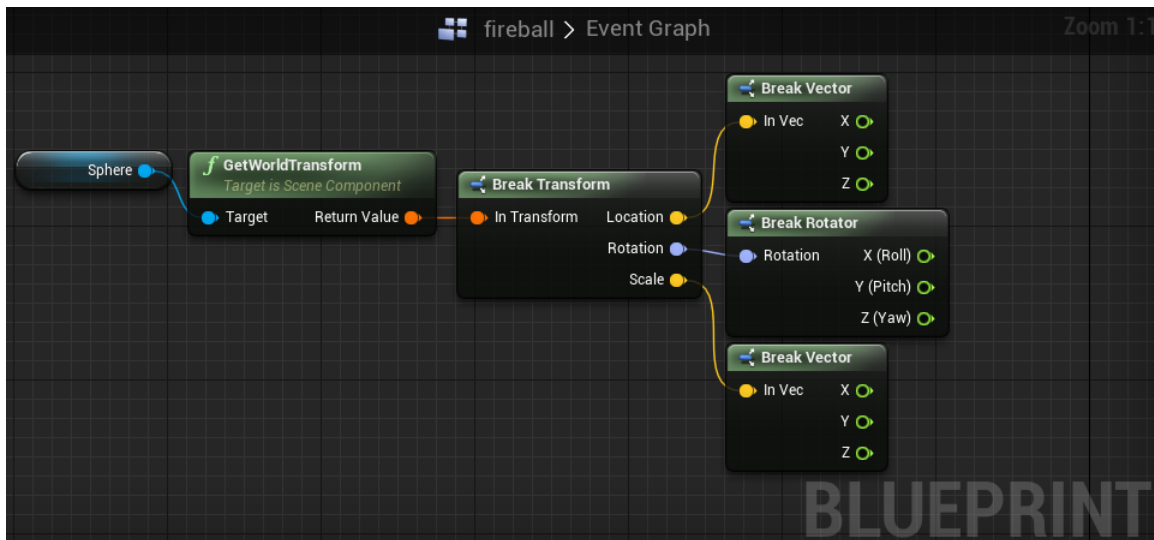


(Villasmil, 2018)

Además de las variables existe una amplia gama de funciones prediseñadas que cumplen tareas específicas y que en ciertos casos solo funcionan con ciertos tipos de variables.

A partir de este punto se notifica al lector que el diseño y programación de nodos fueron desarrollados en su mayoría en inglés por el hecho de que es lenguaje más hablado en el mundo y porque en los amplios foros de dudas provenientes de la comunidad del Motor Unreal Engine así como el soporte técnico de Epic Games, lo trabajan en ese lenguaje y que de esta manera entienda mejor lo que desee buscar o como crear con base en estos.

Un ejemplo es la función Get world Transform que en los *blueprints* recopila los datos de ubicación, rotación y escala de un componente o actor para poderlo usar más adelante, así como la función break vector y break rotation que descomponen estos resultados en sus 3 ejes vectoriales en este caso X,Y,Z.



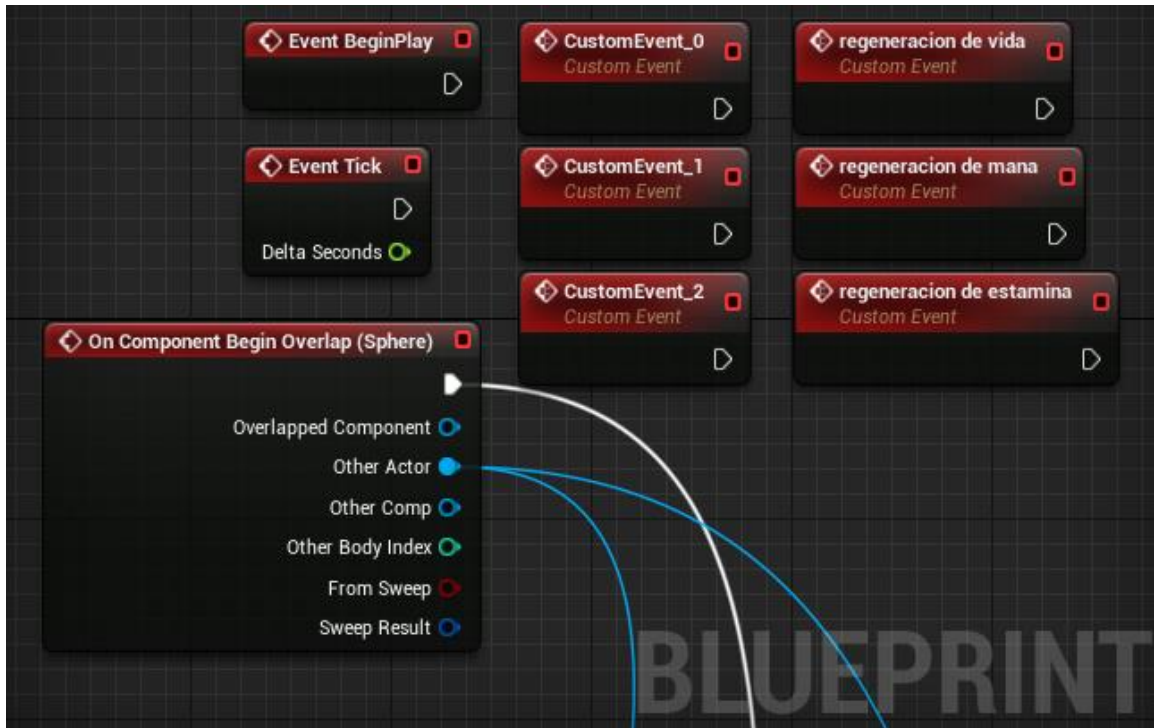
(Villasmil, 2018)

Otras funciones que aparecen de manera predeterminada al crear ciertos *blueprints* son Event Begin Play que realiza las tareas al comenzar el juego o nivel como asignar valores o reproducir multimedia, Event Tick que realiza los eventos cada cuadro del juego que pueden llegar a ser una décima de segundo, pero puede variar el desempeño tanto por hardware como elementos dentro del proyecto que al ser demasiados afectan el desempeño de procesamiento.

Otro que es muy común y muy usado es Begin Overlap dependiendo del elemento del *blueprint* puede ser un actor o componente, esto evalúa si se realizó una colisión con dichos elementos para después realizar una acción, muy útil al momento de evaluar situaciones o tomar acciones con base en estas. También existe la opción de crear funciones personalizadas a través de Custom Event, se

coloca las tareas deseadas que haga y se puede llamar a través de su nombre en otra función, se recomienda cambiarle el nombre teniendo un límite máximo de 255 caracteres para hacer más fácil la documentación y las tareas de escritura o modificación del mismo, pues el motor de crearse múltiples Custom Event solo anexa “_x” al final del nombre donde x será el próximo número que corresponda a la creación del Custom Event y puede llegar a ser confuso al momento de buscar o editarlos. También existen nodos para las funciones matemáticas tanto para valores enteros como flotantes; e inclusive otra importante es Branch que es usada para evaluar comparaciones con un resultado determinando si es verdadero o falso.

Existe otra función, la Delay que al ser llamado hace uso del reloj del equipo y crea un temporizador que se puede editar cuanto dure para que haga cierta tarea, por ejemplo, un temporizador para que termine un nivel en cinco minutos o menos y que cuando finalice el tiempo haga las tareas que correspondan (que perdió el usuario).

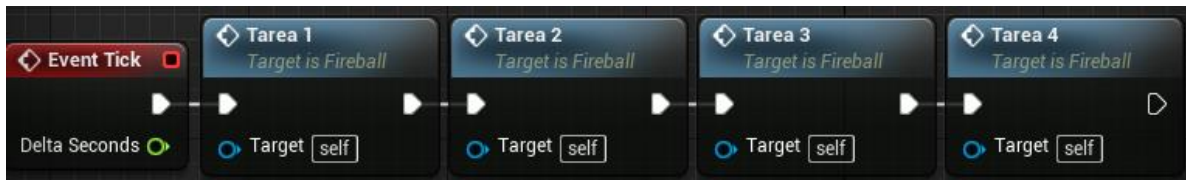


(Villasmil, 2018)

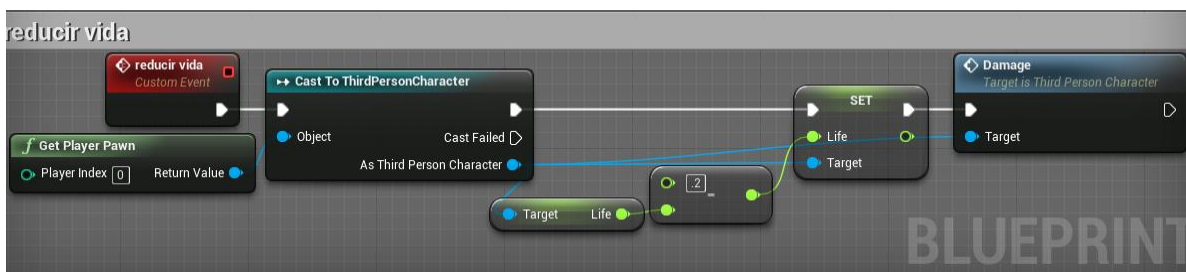
Entrada y salida de nodos

Tanto en las funciones como en las variables tienden a recibir valores de entrada así como sacar resultados para ser usados en tareas y análisis más adelante. Al momento de programar los proyectos por nodos estos siguen un orden de izquierda a derecha y al momento de enlazar las tareas se debe hacer *click* en un extremo y arrastrarlo al siguiente nodo siempre y cuando tenga una entrada para conectar la salida del nodo previo, a este usualmente las entradas y salidas de valores deben ser del mismo tipo a menos que se recurran a funciones de conversión de valores a otros tipos, pero no funciona con todos, por ejemplo es posible convertir un valor entero a flotante, pero no es posible convertir una cadena de texto en valor entero pero sí es posible convertir un valor flotante o entero a cadena de texto. Los orificios de distintos colores que corresponden a los tipos de

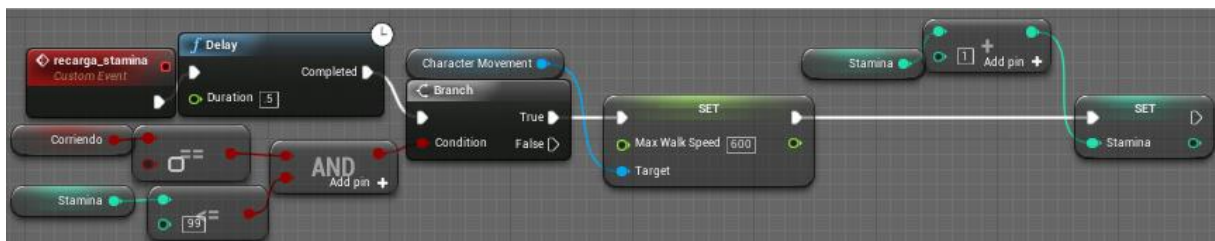
variables pueden estar como entrada o salida de distintos nodos y funciones para realizar operaciones matemáticas, relacionales, evaluaciones y otras tareas que quedan a decisión e imaginación del usuario.



(Villasmil, 2018)



(Villasmil, 2018)

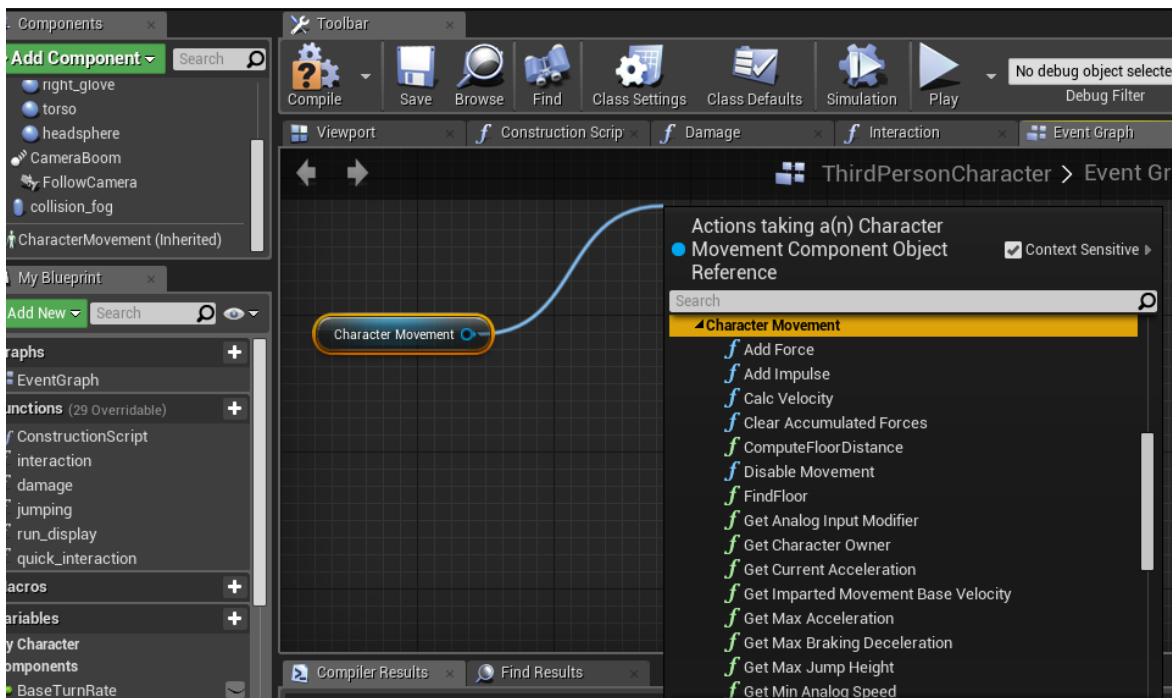


(Villasmil, 2018)



(Villasmil, 2018)

desarrollo. Estos componentes de ser necesario su uso dentro del plano solo basta con hacer *click* y mantener pulsado en el seleccionado de la jerarquía y arrastrar dentro del plano y saldrá una referencia a este componente en forma de nodo con su salida; arrastrando desde la salida de ese nodo se pueden hacer usos de funciones hechas específicamente para interactuar con esta por ejemplo arrastrando la salida de datos del character movement aparecen



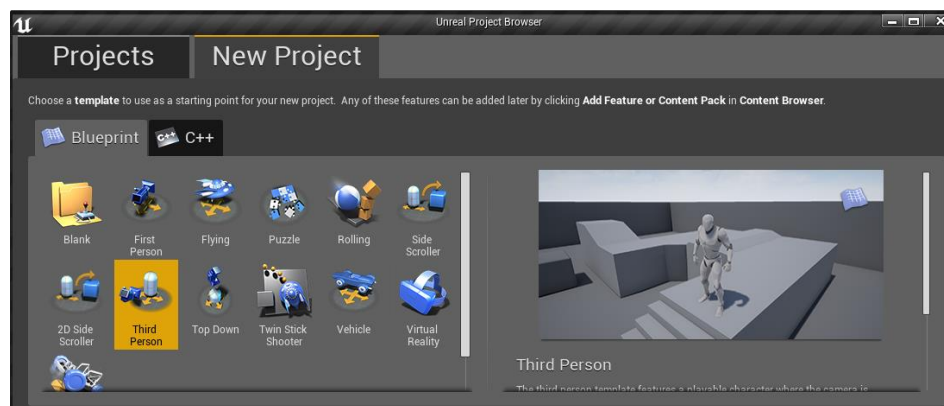
(Villasmil, 2018)

DESARROLLO DE UN JUEGO EN UNREAL ENGINE 4

Selección de proyecto

Luego de abarcar la teoría de los juegos, su lógica y la composición del motor detallando desde su instalación en el equipo de computación, la interfaz del mismo motor, su nuevo *script* de programación visual de los nodos así como su funcionamiento básico. En este capítulo se mostrará el paso a paso para el desarrollo de un proyecto de manera resumida en algunos métodos y pasos ya mencionados anteriormente en los capítulos previos. Detallándose más adelante una serie de pasos no mencionados en los anteriores debido a que su desarrollo no es tan extenso como para tener su propio capítulo en el desarrollo de este manual.

Se seleccionará la plantilla del proyecto de tercera persona que consistirá en controlar un personaje con una visión de una cámara que se puede mover alrededor de él, siempre enfocando al personaje en una visión de tercera persona y este puede desplazarse en las tres dimensiones (x, y, z) y es capaz de saltar.



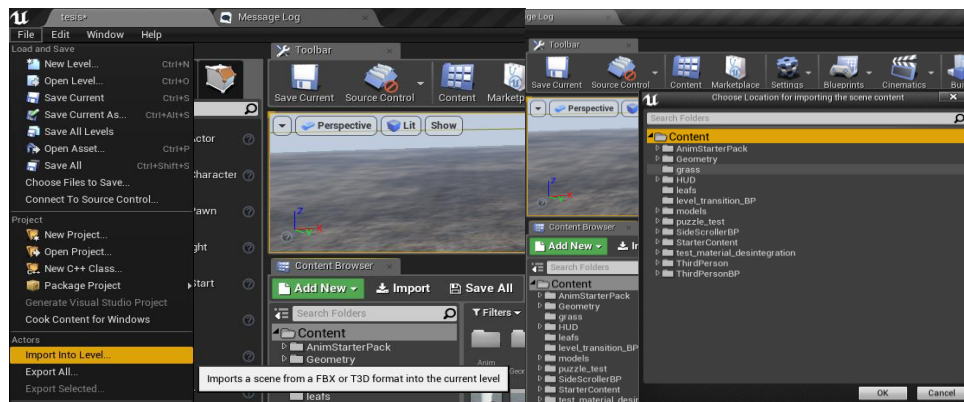
(Villasmil, 2018)

Importación de contenido

De ahí se va a importar el contenido de escenarios, modelos 3D y hasta animaciones, para ampliar y mejorar el proyecto más allá del contenido predeterminado. Esta parte queda a decisión del lector de buscar los elementos que conformaran su contenido, existiendo una amplia variedad de páginas que ofrecen modelos 3d gratuitos para uso sin fines de lucro e inclusive educativos, entre los recomendados por experiencia del lector están las siguientes páginas web:

- www.free3d.com
- www.cadnav.com
- www.sketchfab.com

En cuanto a animación de modelos humanoides existe la página web de www.mixamo.com que puede ayudar con animaciones básicas de un modelo de su preferencia siempre y cuando esté en posición A o posición T para reducir tiempo y esfuerzo en las animaciones de personajes dentro del proyecto. Al momento de importar el contenido dentro del proyecto existen dos opciones; la primera es usar la opción de importar modelo en el nivel, buscar en el explorador de archivos el modelos y seleccionar su ubicación en la jerarquía de archivos.



(Villasmil, 2018)

La segunda opción es más fácil y con pocos pasos mientras se tiene seleccionado el modelo 3D en el explorador de archivos se arrastra hasta el programa por la barra de tareas hasta que se expanda y de ahí se procede a posicionarlo en alguna carpeta del proyecto y después soltarlo, a partir de ahí se importará automáticamente el modelo en el proyecto y funciona también para imágenes, audio y hasta video, este método es tan fácil como intuitivo.

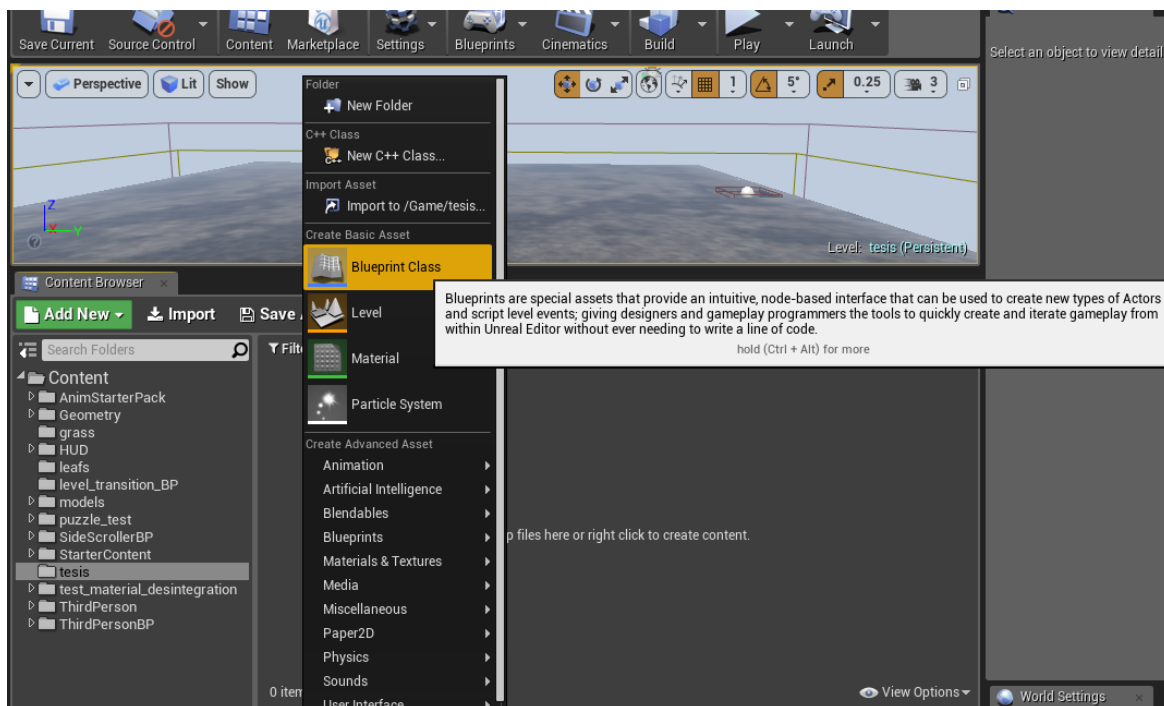
Creación de *blueprints*

La principal característica del motor son los *blueprints* o planos en los que están integrados la programación por nodos del motor y que se clasifican en diferentes tipos los cuales son:

- **Actor.** Es un objeto que puede ser posicionado o generado en el mundo o nivel del proyecto. Este tipo de “Blueprint” son usados para los objetos comunes que van a conformar parte del juego pueden abarcar desde muebles, hasta objetos con movimiento mecánico inclusive escenarios y elementos en uno solo.
- **Peón (*pawn*).** Es un actor que puede ser poseído (definición por los desarrolladores del motor para controlar) y recibir entradas de un controlador (teclado, *joystick*, etc.).
- **Personaje (*character*).** Comparte las mismas características que un peón pero incluye la habilidad de caminar, correr, saltar y muchas más funciones relacionadas con su movilidad en términos de tipos y medición cuantitativa.
- **Controlador de jugador (*carácter controller*).** Es el actor responsable de controlar al peón usado por el jugador.

- **Modo de juego (*game mode*).** Define como el juego será jugado, sus reglas, puntuación, entre otras características y fases que se pueden agregar al tipo de juego.

La manera de crearlos en el motor es seleccionar un espacio en blanco del explorador de archivos y hacer *click* derecho, a partir de ahí aparecerá un listado de elementos que podemos añadir en la ubicación del explorador.

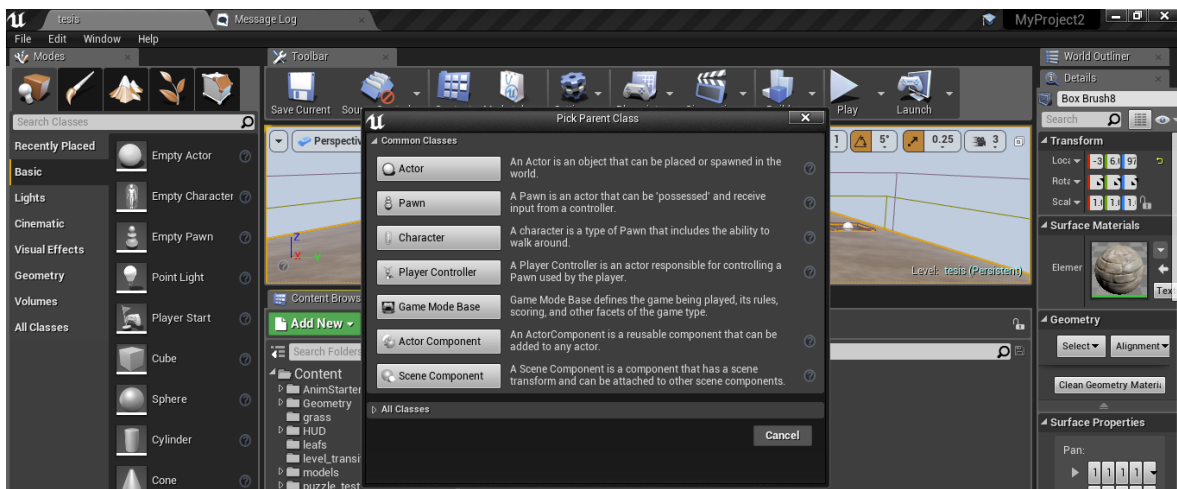


(Villasmil, 2018)

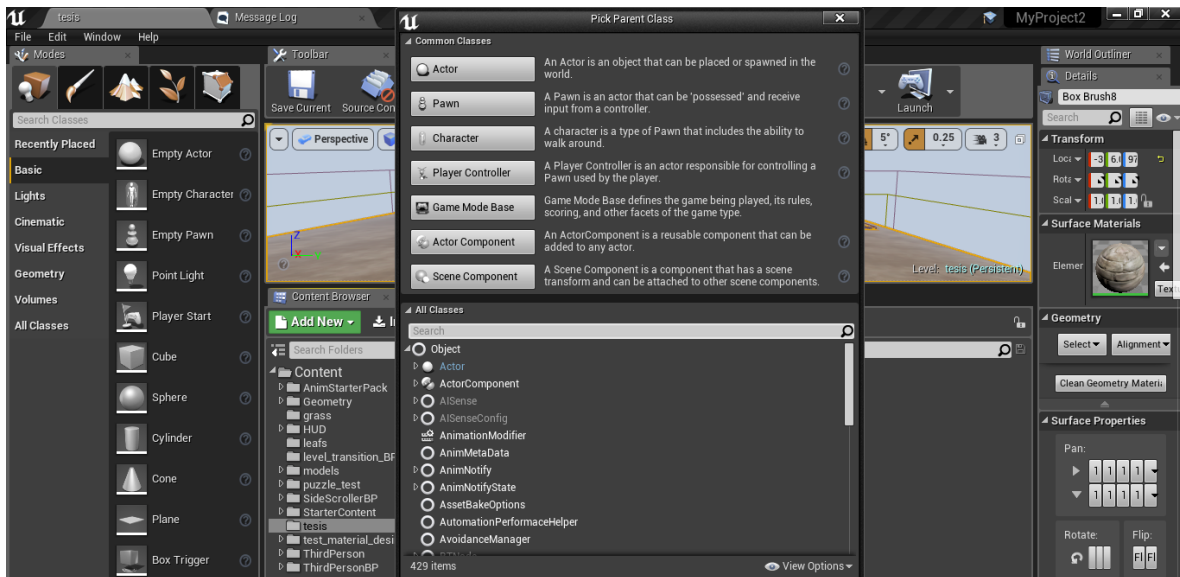
Al hacer *click* en el mostrara los *blueprints* disponibles para empezar el desarrollo de proyectos, junto con la definición que abarca cada uno, y más abajo existe la opción de todas las clases en el que se muestra no solo los *blueprints* predeterminados por la plantilla seleccionada del proyecto, también mostrara todos los *blueprints* creados por los desarrolladores del proyecto en caso de compartir características similares pero con ligeros cambios que deseen aplicar.

Por ejemplo se desea crear un *blueprint* de Inteligencia Artificial llamado Enemigos este tendrá programado que debe atacar al jugador además de ser asignado valores como vida, puntos de ataque y una variable para el color puede ser de vestimenta para enemigos de igual diseño, dificultad, etc.

Luego se procede a crear nuevos *blueprints* para los enemigos básicos hasta los jefes y tendrán en común las funciones del *blueprint* Enemigos, en el cual se van a variar los valores de cada uno para hacer desde los más débiles hasta los más resistentes para mejorar la experiencia y avance del juego.

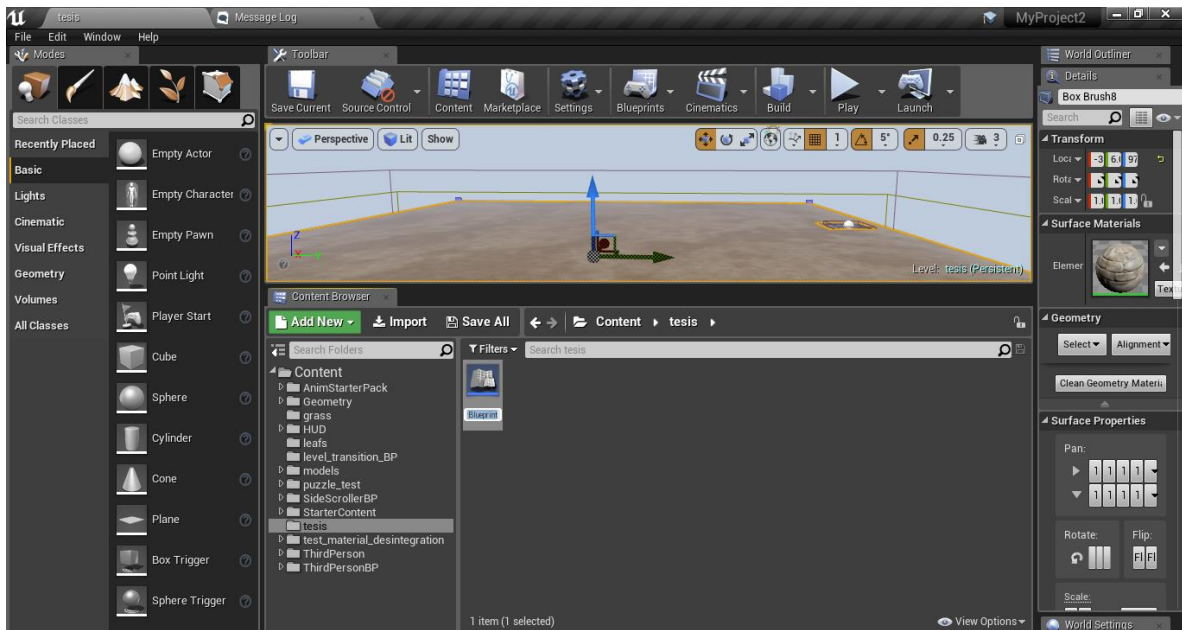


(Villasmil, 2018)



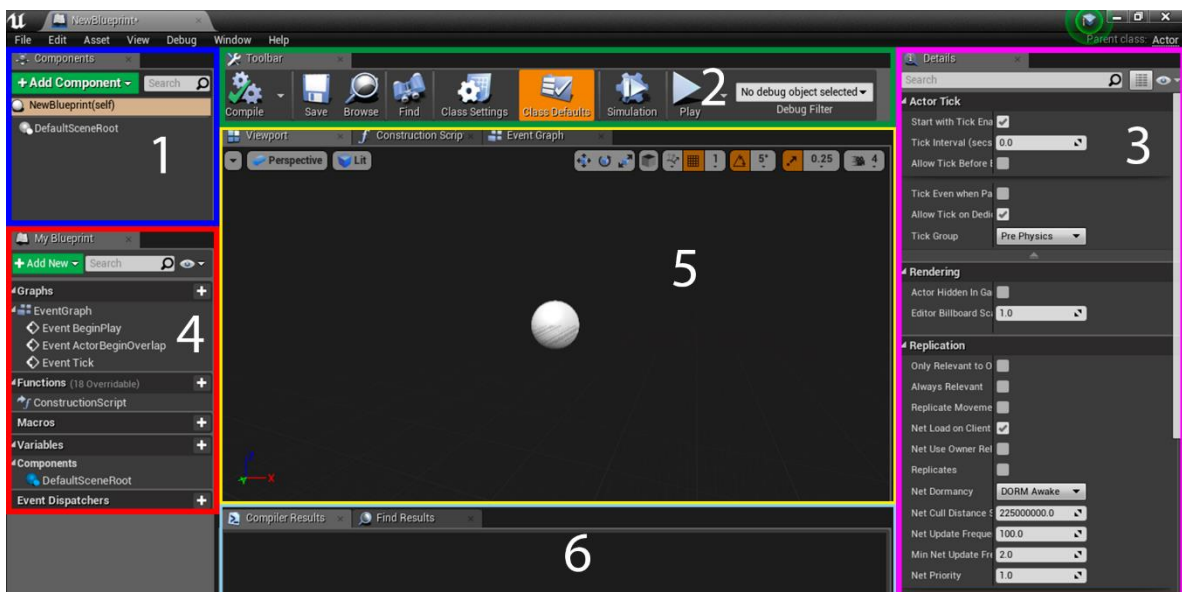
(Villasmil, 2018)

Con el menú de *blueprints* vamos a seleccionar el de tipo actor y vamos a proceder a añadirle elementos básicos, al hacerle *click* será creado en la ubicación del explorador y estará resaltado el nombre para cambiar el nombre del *blueprint*. Se recomienda cambiar el nombre por uno específico, identificativo y hasta único pues en caso de ser replicados con el mismo nombre o variaciones con ligeros cambios van a ser un problema de identificación, modificación y organización. Es posible hacerlo pero queda a decisión del desarrollador y su método de organización.



(Villasmil, 2018)

Una vez creado con todo y nombre se procede a hacer doble *click* o si esta seleccionado solo presionar el botón *Enter*, se visualizara la interfaz para la edición del *blueprint* esta se divide en 6 partes o secciones:



(Villasmil, 2018)

- 1) La sección de componentes permite, añadir, editar y hasta eliminar los variados elementos que pueden ser anexados dentro del *blueprint* por el desarrollador, estos componentes o elementos abarcan desde modelos 3D (predeterminados del motor, personalizados, creados e inclusive personalizados dentro del motor), animaciones 3D, efectos de sonido, efectos visuales (partículas), etc.

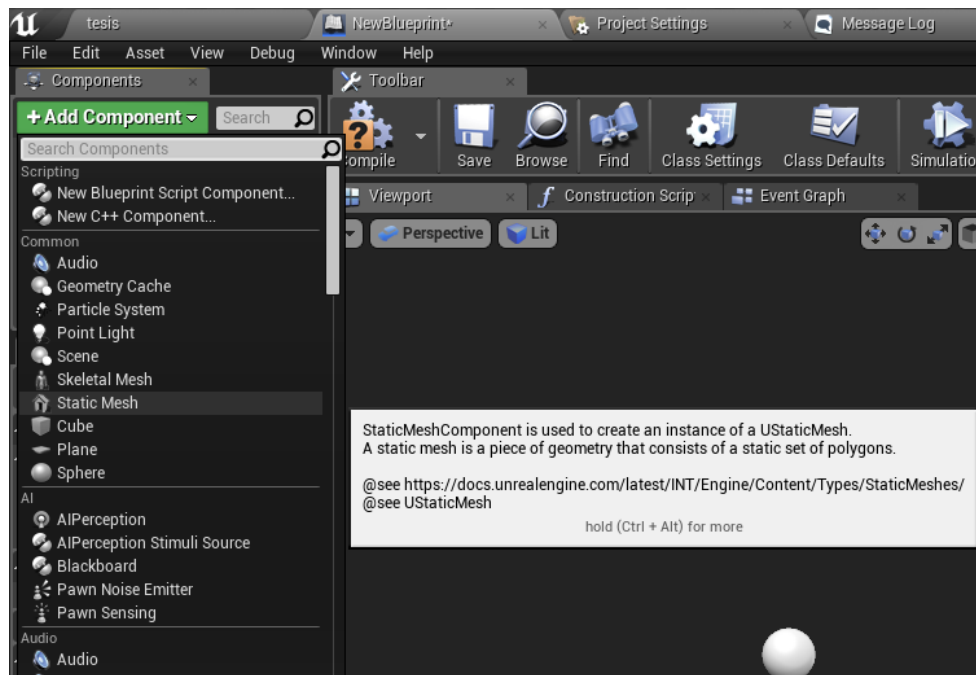
Para anexar nuevos elementos o componentes en el *blueprint* solo se debe hacer *click* en el botón verde *Add component* para luego seleccionar el tipo de objeto a añadir y dependiendo de ese tipo de objeto buscar el elemento compatible dentro del explorador de contenido del proyecto y asignarlo a dicho objeto.

- 2) Esta sección es la interfaz de Unreal Engine 4y sus funciones se encuentran detalladas en el capítulo “Interfaz de Unreal Engine 4”
- 3) Esta sección muestra todos los atributos editables del componente dentro del *blueprint* desde sus dimensiones, ubicación, rotación, materiales, etc.
- 4) Esta sección corresponde a la parte lógica y programática del *blueprint*, en esta yace listada el grafico del *blueprint*, la funciones que pueden ser creadas o editadas, la interfaz, los macros (ejecución de nodos con una entrada de datos y una salida de estos) e incluso la parte más vital a la hora de programar dentro del *blueprint* las variables que pueden ser creadas o editadas para la lógica y programación de las tareas que hagan el *blueprint*.
- 5) Esta sección corresponde a la ventana de visualización del *blueprint* donde mostrara todos los elementos que lo conforman como modelos 3D, multimedia, efectos visuales, efectos de sonido, animaciones, texto y más.

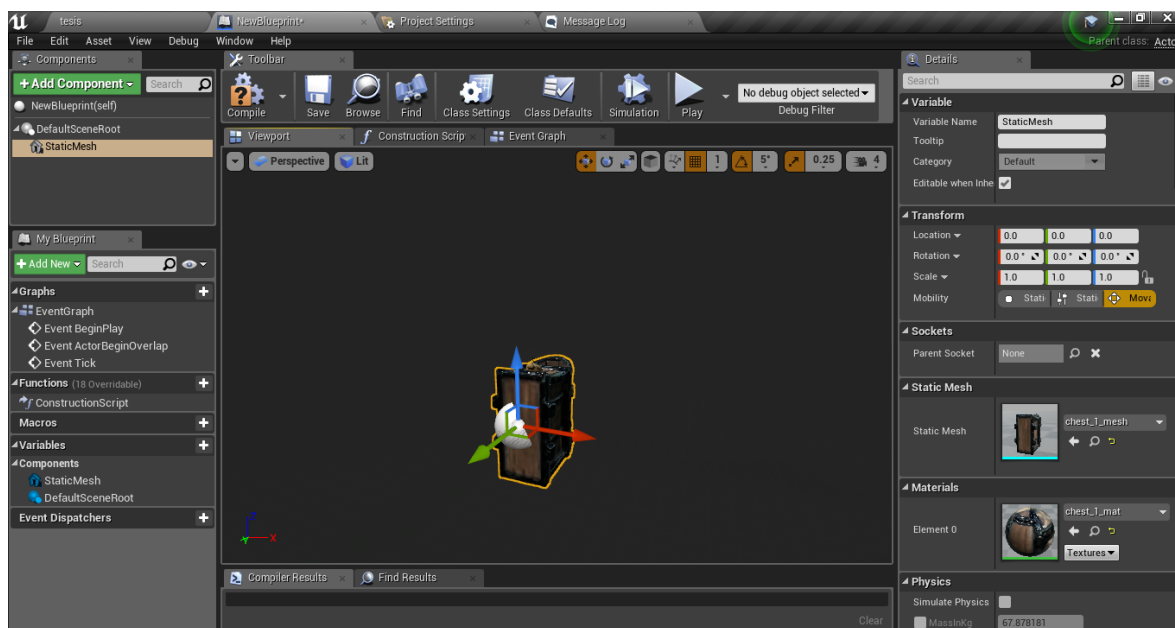
Todo con la finalidad de ver e inclusive de ser necesario editar dichos elementos y sus características a través de la sección de detalles.

- 6) Esta sección muestra los resultados de compilación del *blueprint*, también los resultados de ejecución dentro del proyecto, mostrando no solo los resultados de procesamiento, además muestra cualquier error detectado en el *blueprint* en una lista con la ubicación del elemento causante del error junto con la especificación de dicha falla para su edición y corrección.

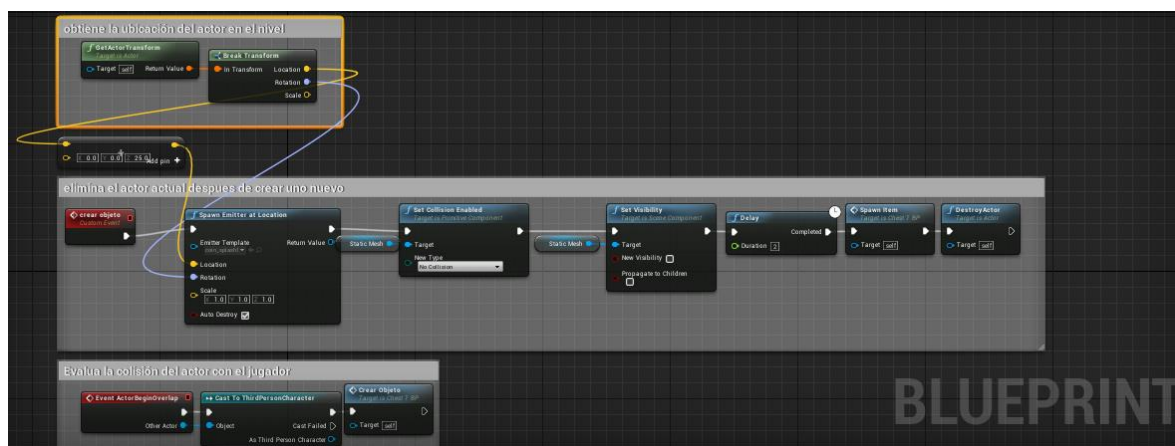
Serán anexados más adelante el uso de estas características con su respectiva captura de pantalla para mejor comprensión de la creación, edición y aplicación de estas.



(Villasmil, 2018)

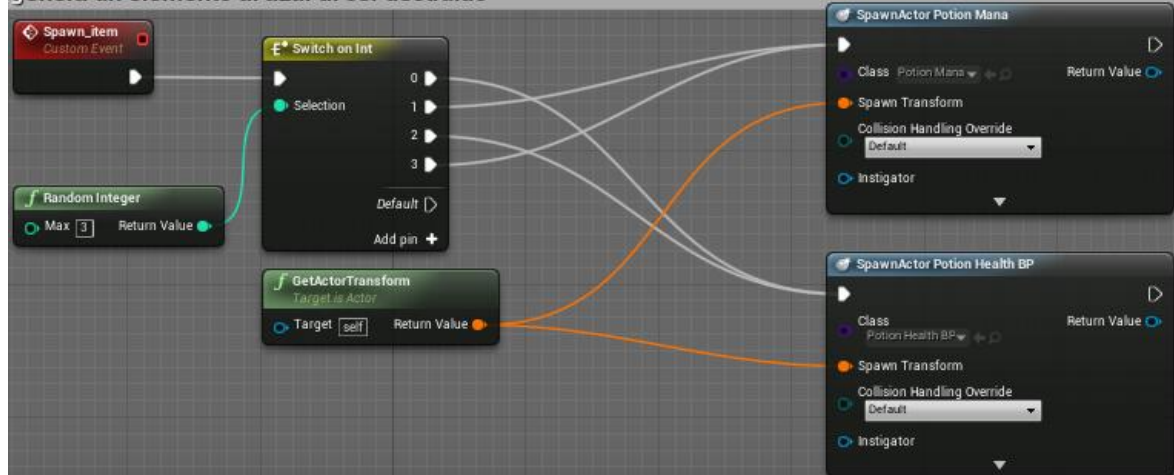


(Villasmil, 2018)

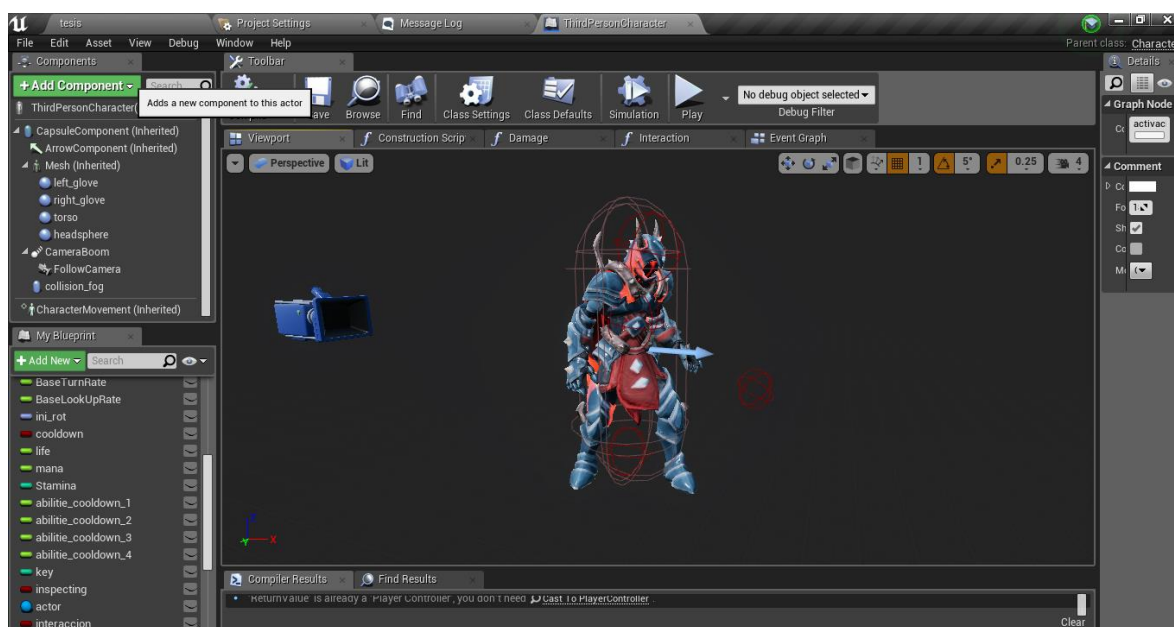


(Villasmil, 2018)

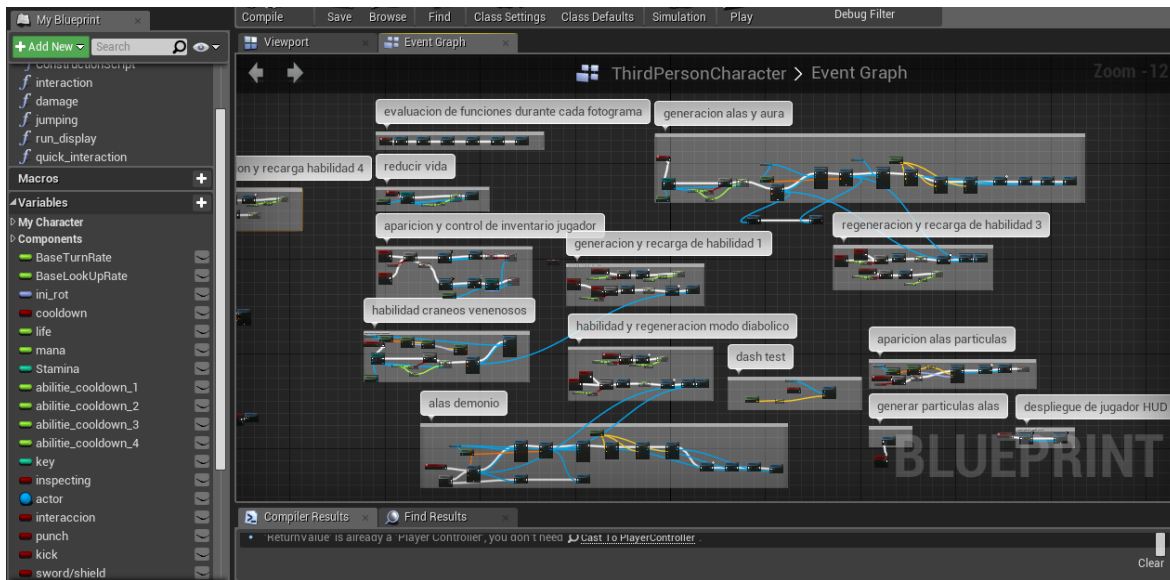
genera un elemento al azar al ser destruido



(Villasmil, 2018)

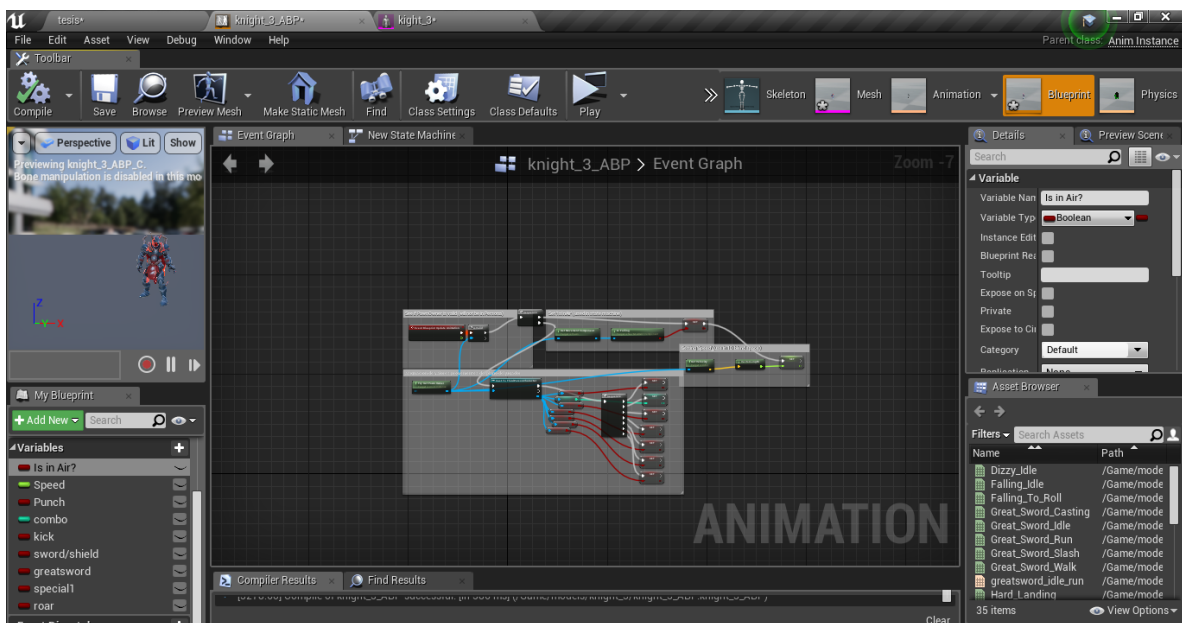


(Villasmil, 2018)

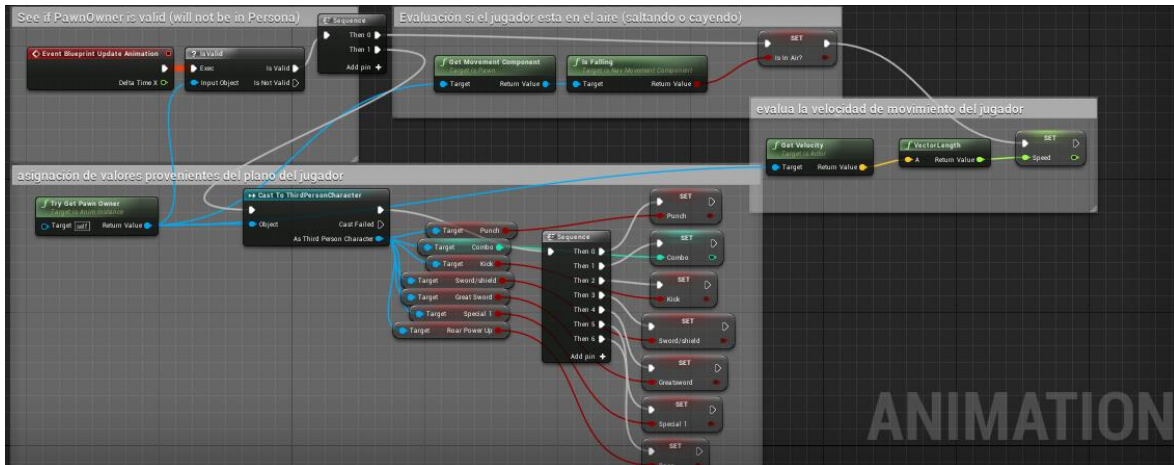


(Villasmil, 2018)

Otro aspecto importante al programar el personaje es el plano de animación, este es responsable de realizar los movimientos, ataques y cualquier secuencia estética necesaria que compone dicho personaje, su diseño de interfaz se asemeja al de los *blueprints*.

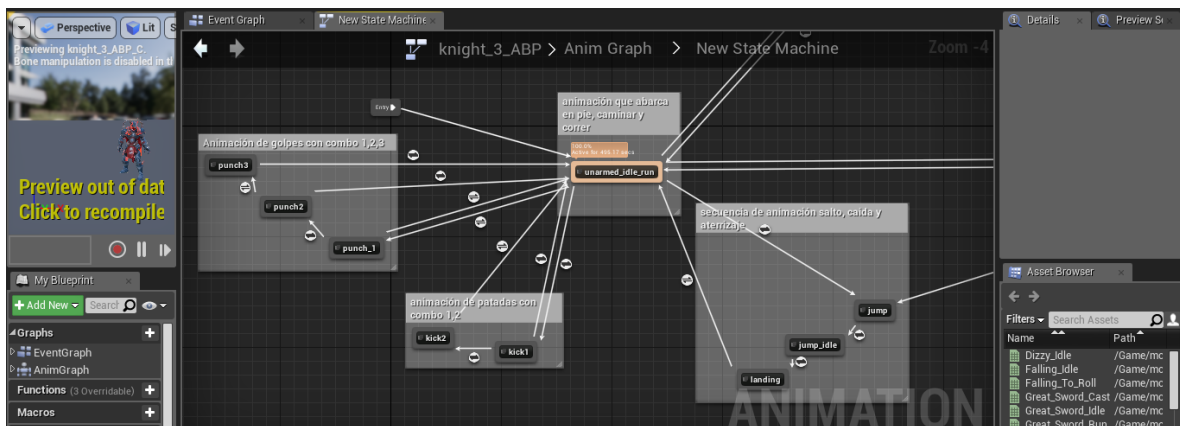


(Villasmil, 2018)

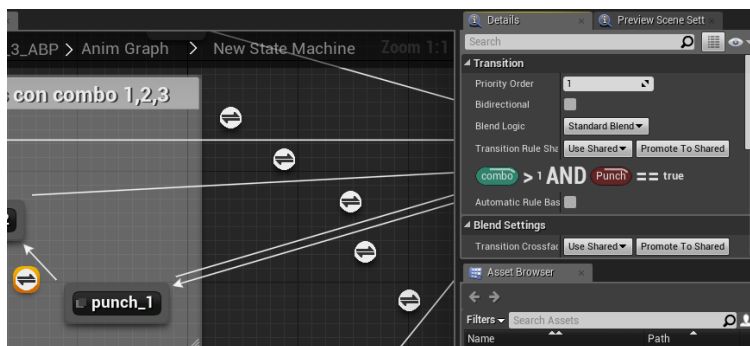


(Villasmil, 2018)

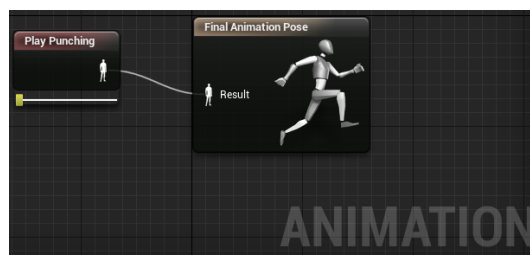
También cuenta con una interfaz alterna que maneja las transiciones de entre distintas animaciones basado en los valores de las variables dentro del mismo plano de animación. El diseño está compuesta por nodos que al hacer doble *click* abre la ventana de reproducción ahí se coloca la animación correspondiente.



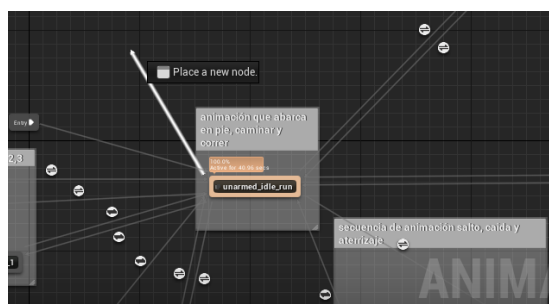
(Villasmil, 2018)



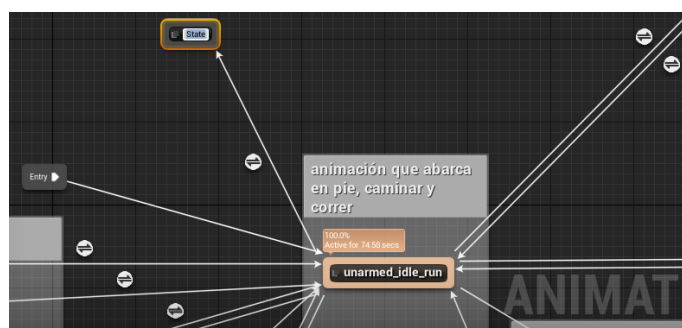
(Villasmil, 2018)



(Villasmil, 2018)



(Villasmil, 2018)



(Villasmil, 2018)

CREACIÓN DE EFECTOS VISUALES EN UNREAL ENGINE 4

Unreal Engine 4 contiene un sistema de partículas totalmente amplio y potente, que permite para no solo desarrolladores también a los artistas crear efectos visuales como humo, fuego, agua hasta ejemplos mucho más complejos como fantasía o ciencia ficción.

Este sistema de Unreal Engine 4 se edita a través de *Cascade*, un editor de efectos de partículas completamente integrado y modular, ofreciendo pre-visualización en tiempo real y edición de efectos modulares, lo que permite la creación rápida y fácil de los efectos más complejos.

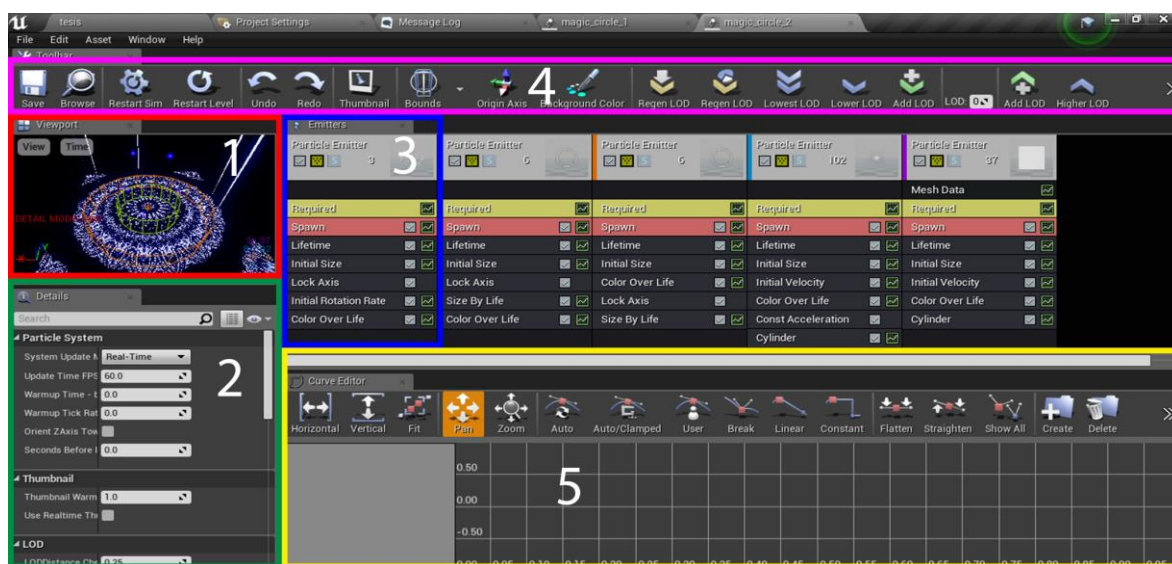
Los sistemas de partículas también están muy relacionados con los diversos materiales y texturas aplicados a cada partícula. Su función principal en sí mismo es controlar el comportamiento de las partículas, mientras que el aspecto y la sensación específica del sistema de partículas en su conjunto a menudo se controlan mediante materiales.

Están las partículas predeterminadas en el motor, también se pueden acceder a unas gratuitas dentro del Marketplace del motor para ver y hasta editar basado en ellas nuevas partículas en las que esté interesado el desarrollador de aplicar dentro de su proyecto. En las próximas páginas se mostrara el desarrollo desde cero de un efecto visual para mejorar el entendimiento de la interfaz y del efecto visual mismo.

Muchos se preguntaran ¿realmente hace falta efectos visuales en un videojuego?, la respuesta depende del desarrollador y lo que quiere mostrar en su proyecto, muchos juegos actualmente muestran uno o varios efectos visuales para mejorar la estética en escenarios, personajes, habilidades, guías, etc.



(Playground, 2018)



(Villasmil, 2018)

- 1) Esta sección corresponde a la ventana de visualización, que brinda una vista previa renderizada del sistema de partículas actual, tal como aparecería cuando se procesa en el juego. Proporciona información en tiempo real de los cambios realizados en el sistema de partículas en *Cascade*. Además de la vista previa completa, el panel de la ventana gráfica también se puede mostrar en los modos de vista sin luz, densidad de textura, sobregiro y alámbrico, y muestra información como los límites actuales del sistema de partículas.
- 2) Esta sección corresponde a los detalles muestra todos los atributos editables del componente dentro del efecto visual desde sus dimensiones, ubicación, rotación, materiales, etc.
- 3) El Panel de Emisores contiene cada emisor de partículas contenido dentro del sistema de partículas actualmente abierto en *Cascade*. Desde aquí puede agregar, seleccionar y trabajar con los diversos módulos de partículas que controlan el aspecto y el comportamiento del sistema de partículas.

La lista de emisores contiene una disposición horizontal de todos los emisores dentro del sistema de partículas actual. Puede haber cualquier número de emisores dentro de un solo sistema de partículas, cada uno generalmente manejando un aspecto diferente del efecto general.

Cada columna representa un solo emisor de partículas, y cada una está formada por un bloque de emisores en la parte superior, seguido de cualquier número de bloques de módulos. El bloque del emisor contiene las propiedades principales del emisor, como el nombre y el tipo del emisor, mientras que los módulos que se encuentran debajo controlan varios aspectos del comportamiento de las partículas. Aunque la interfaz para la lista

de emisores es bastante sencilla, contiene un menú sensible al contexto al que se puede acceder mediante el botón derecho del ratón.

- 4) Esta sección es la interfaz de Unreal Engine 4 y sus funciones se encuentran detalladas en el capítulo Interfaz de Unreal Engine 4.
- 5) Este editor gráfico muestra las propiedades que se modifican en un tiempo relativo o absoluto. A medida que se agregan módulos al editor de gráficos, se anexa más adelante el uso de estas características con su respectiva captura de pantalla para mejor comprensión de la creación, edición y aplicación.

El panel de emisores se conforma de dos secciones



(Villasmil, 2018)

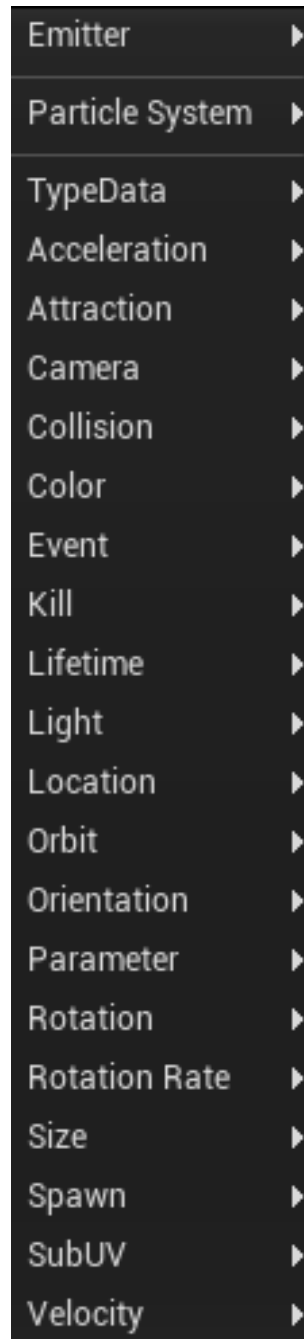
- 1) En esta sección se encuentra el bloque del emisor el cual contiene las propiedades y controles primarios para el propio emisor, como el tipo de emisor, el nombre del emisor, junto con otras propiedades primarias.
- 2) En esta sección se listan todos los módulos que definen el aspecto y el comportamiento de este emisor. Todos los emisores tendrán un módulo requerido, después del cual puede haber cualquier número de módulos para definir mejor el comportamiento.

Al crearse un nuevo emisor se crean de forma predeterminada los módulos de la imagen previa como:

- **Required.** En esta se encuentran los detalles del material del emisor, su ubicación, rotación, alineación, duración, renderizado, etc.
- **Spawn.** Es usado para la generación de uno o varios elementos, así como su distribución, duración y hasta la velocidad de elementos generados.
- **Lifetime.** Se configura la duración del emisor seleccionado dentro del efecto visual.
- **Initial Size.** Su función es configurar el tamaño de dicho emisor.
- **Initial velocity.** A través de este módulo es configurada la traslación del emisor en los ejes de coordenadas (X, Y, Z) tanto con valores positivos como negativos.
- **Color Over Life.** Este módulo es usado para configurar el color del emisor sin alterar el material original.

Es posible añadir aún más módulos dentro del emisor logrando un comportamiento más complejo del efecto visual que se desarrollará, para

esto solo se debe hacer *click* en el botón derecho del ratón en un espacio vacío dentro del cuadro del emisor, en el listado de módulos y se desplegará el siguiente menú para anexar un nuevo módulo.

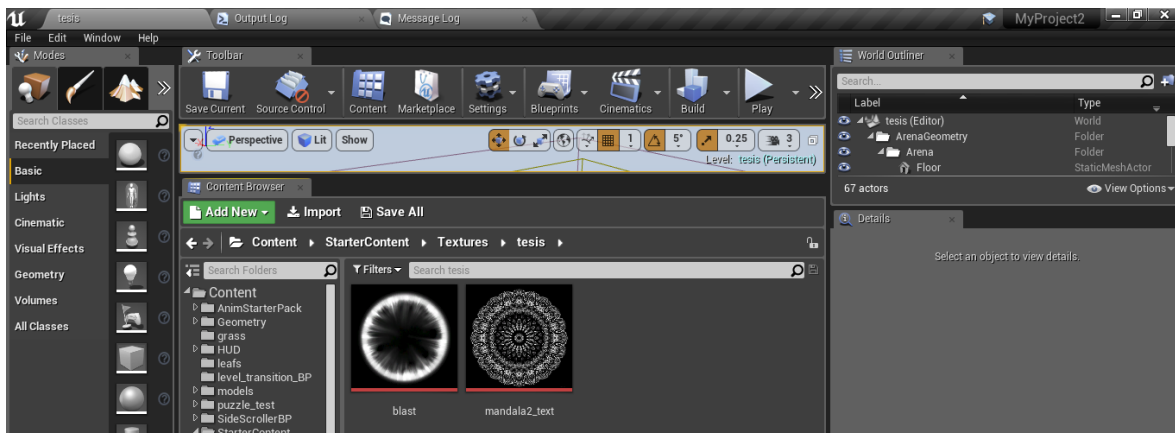


(Villasmil, 2018)

Cada uno de estos módulos listados tiene una función única para editar el comportamiento de dicho emisor dentro del efecto visual y son modificados a través de la sección de detalles.

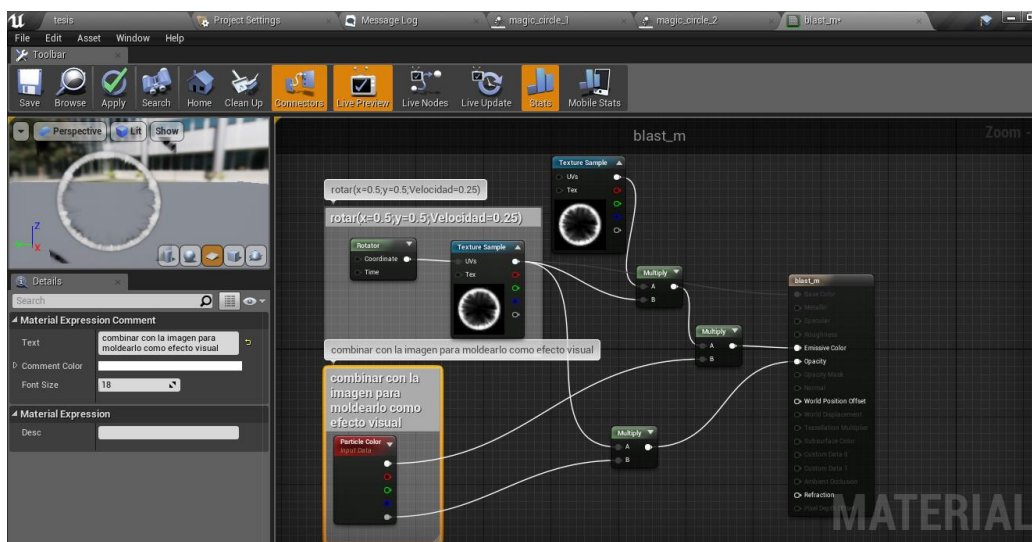
A continuación se mostrará el desarrollo de un efecto visual, paso a paso, en el motor Unreal Engine 4.

Se importan dos texturas dentro del motor en el explorador del proyecto.

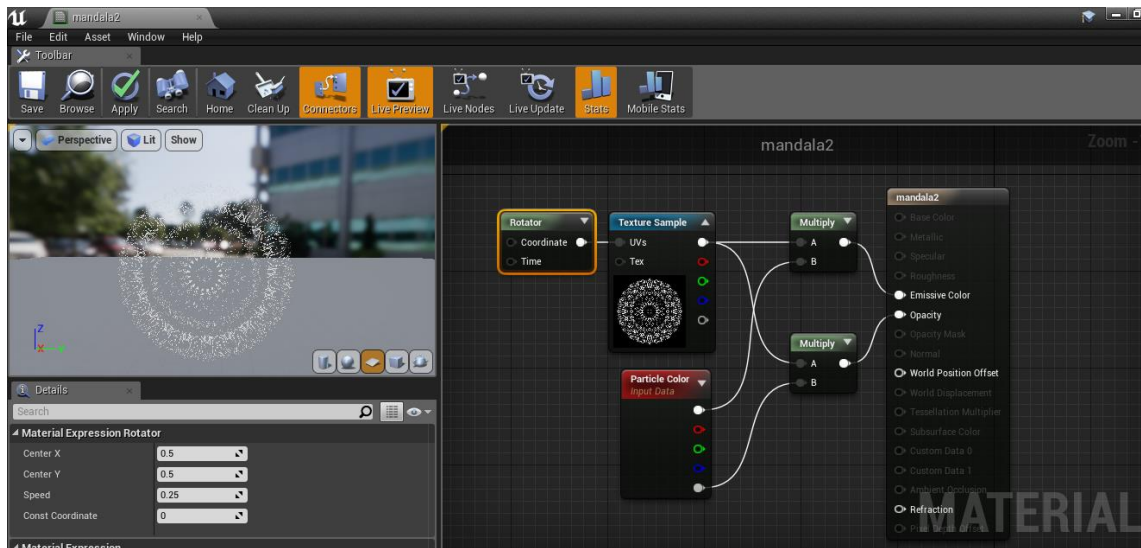


(Villasmil, 2018)

Luego se crean los materiales usando ambas texturas y se configuran como se muestran en las siguientes imágenes.



(Villasmil, 2018)

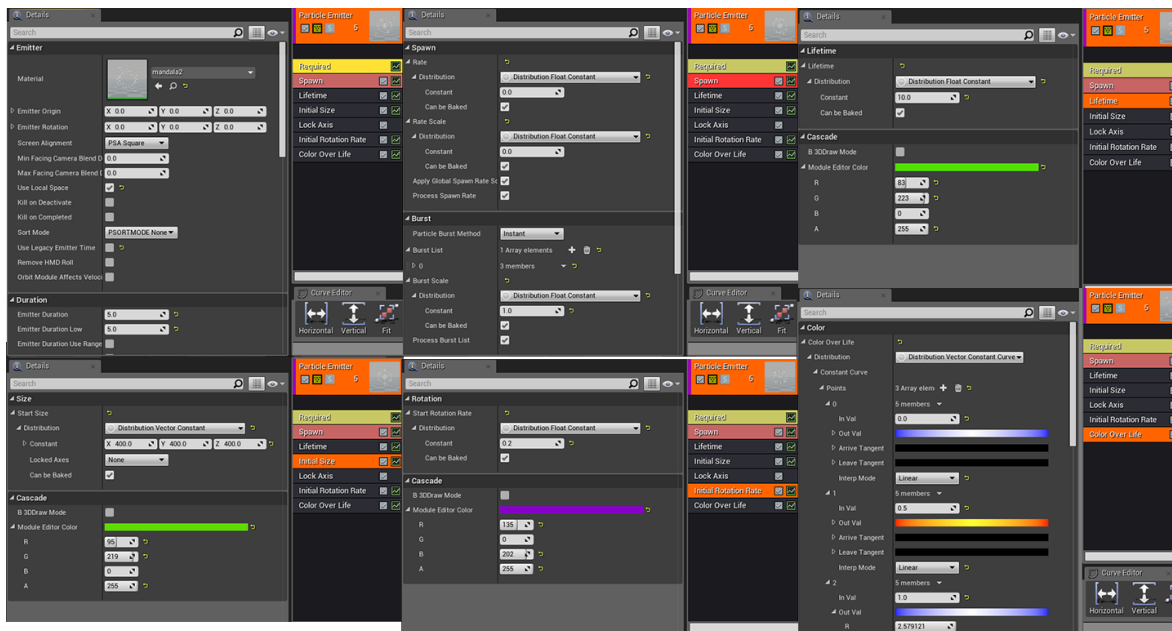


(Villasmil, 2018)

Se hace *click* derecho en un espacio vacío en el explorador de proyectos y se selecciona la opción *Particle System*; luego se le asigna el nombre y se hace doble *click* para abrir la interfaz del editor de efectos visuales mencionado anteriormente; existe de manera predeterminada un módulo de emisor que genera un máximo de 20 imágenes con un material de una X con círculo en el centro con movimiento ascendente, rotación de velocidad baja moderada con duración de 1 segundo por cada imagen, pero con ciclo de repetición indefinido (Valor 0) haciendo que el emisor no sea eliminado al completar su tiempo de duración, sino que repite al terminar y repite el patrón programado desde el principio.

Con el emisor existente en este caso será editado a conveniencia para el efecto visual, se hace *click* en el módulo *Required* y se le asigna el material “Mandala_M”, también más abajo en el panel de detalles en la sección de *Duration* se le asigna el valor 5.0, en *Emitter Loops* se asigna el valor 0, se modifica el *Spawn* con el valor 0 y su *Lifetime* en 10.0, luego se agrega el módulo *Initial Size* se asignan en las coordenadas (X, Y, Z) los valores (400, 400, 400), se agrega el módulo *Initial*

Rotation Rate y se le asigna el valor 0.2, luego el módulo *Lock Axis*, para ser fijado en el eje Z, luego se agrega el módulo *Color Over Life* y es colocado en los campos (R (Rojo),G (Verde),B (Azul)) los Valores (R (3.0),G (3.0),B (23)) cuando el valor del *Lifetime* es igual a 0.0 y a 1.0 mientras que cuando ese valor sea 0.5 los valores (RGB) asignados son (23.0, 2.0, 0.2), gracias a esta configuración el color del emisor cambiará de azul a naranja y de vuelta a azul en lo que transcurre 1 segundo.



(Villasmil, 2018)

En cuanto al segundo emisor se coloca en el módulo *Required* el material “Blast_M”; también, más abajo en el panel de detalles en la sección de *Duration* se le asigna el valor 1.0, en *Emitter Loops* se asigna el valor 0, se modifica el *Spawn* con el valor 2.0 y su *Lifetime* en 2.0, en el módulo *Initial Size* se asignan en las coordenadas (X, Y, Z) los valores (600, 600, 600), luego el módulo *Lock Axis*, para ser fijado en el eje Z, luego se agrega el módulo *Color Over Life* y es colocado en los campos (R(Rojo),G(Verde),B(Azul)) los Valores (R(3.0),G(3.0),B(23)) cuando el valor del *Lifetime* es igual a 0.0 y a 1.0 mientras que cuando ese valor sea 0.5 los

valores (RGB) asignados son (23.0,2.0,0.2), después se agrega el módulo *Size By Life* cuya función se asemeja al de *Color Over Life* en la ejecución basado en el tiempo, pero afectando el tamaño del emisor, en este caso, al inicio cuando pasen 0.0 segundos la imagen estará en su tamaño máximo (Valores $(X,Y,Z)=(1.0,1.0,1.0)$) y al pasar 1.0 segundos, su tamaño será del origen (Valores $(X,Y,Z)=(0,0,0)$), esta configuración causa una onda que se contrae hasta el centro y desaparece.

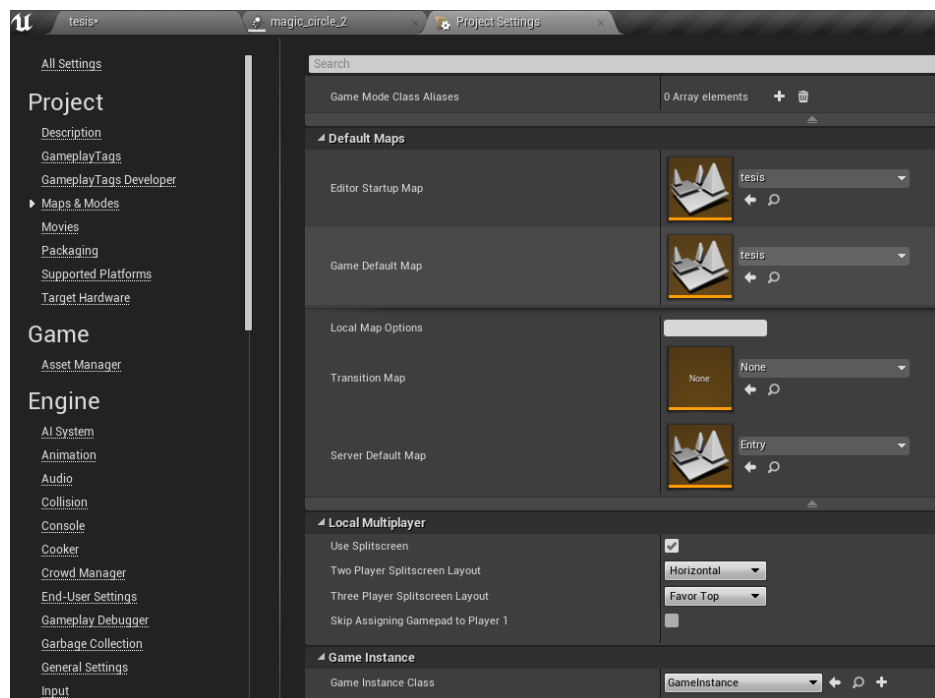


(Villasmil, 2018)

CONFIGURACIÓN Y CREACIÓN DEL PROYECTO

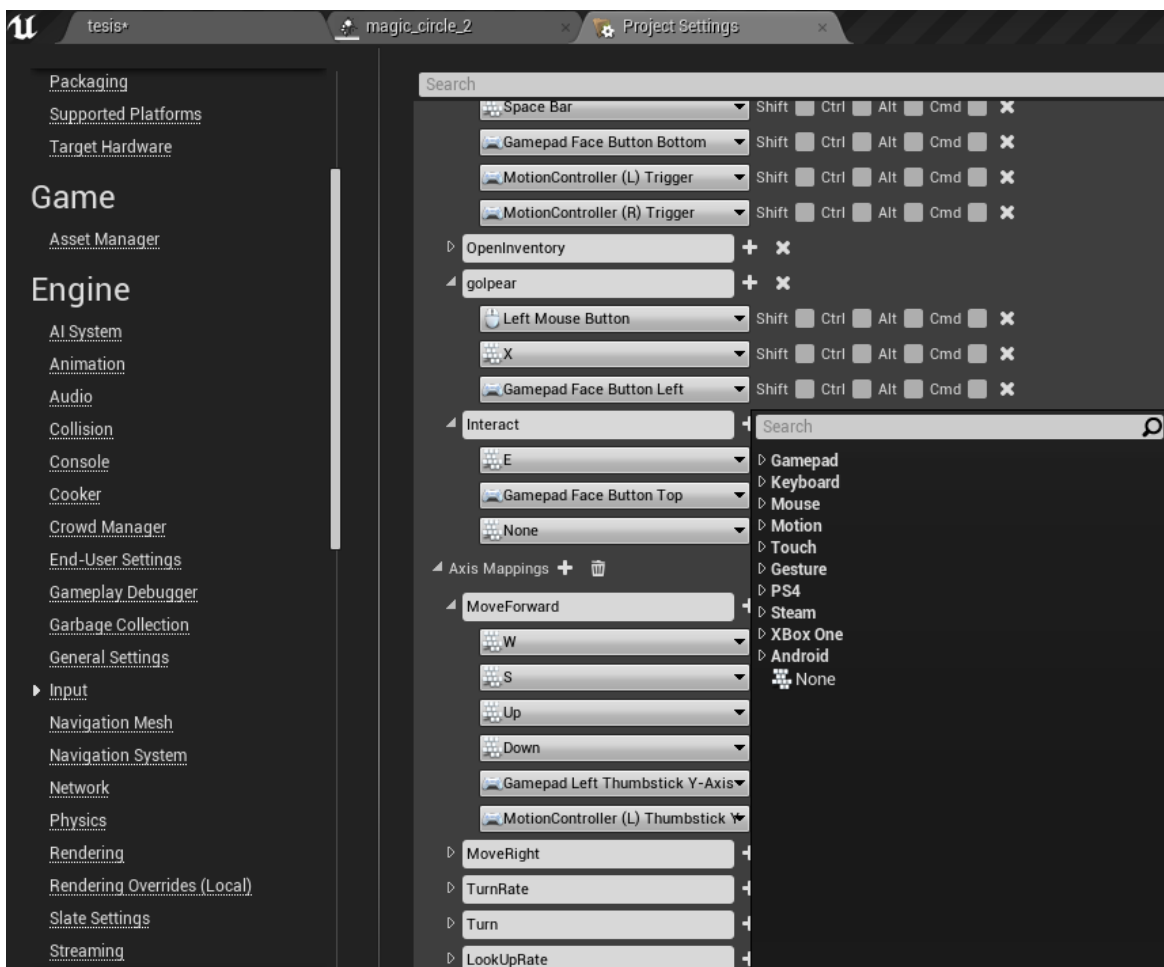
Una vez completada la creación y edición del proyecto se debe empaquetar dicho proyecto para que el código y el contenido del proyecto se encuentren actualizados y en el formato correspondiente para ser ejecutado en la plataforma deseada.

En la interfaz de Unreal Engine 4 se hace *click* en la pestaña de *Edit* y entre las opciones se hace *click* en *Project Settings*, este abrirá una ventana que despliega todas las opciones para configurar el proyecto a nivel funcional de controles, niveles, las librerías de programación, etc. En este caso se hará énfasis en dos partes que deben resaltar , los *Map and Modes* que se usan para la selección del nivel predeterminado con base en múltiples niveles creados dentro del proyecto, solo se debe hacer *click* en las barras de las categorías y escoger uno de los niveles del proyecto listado,



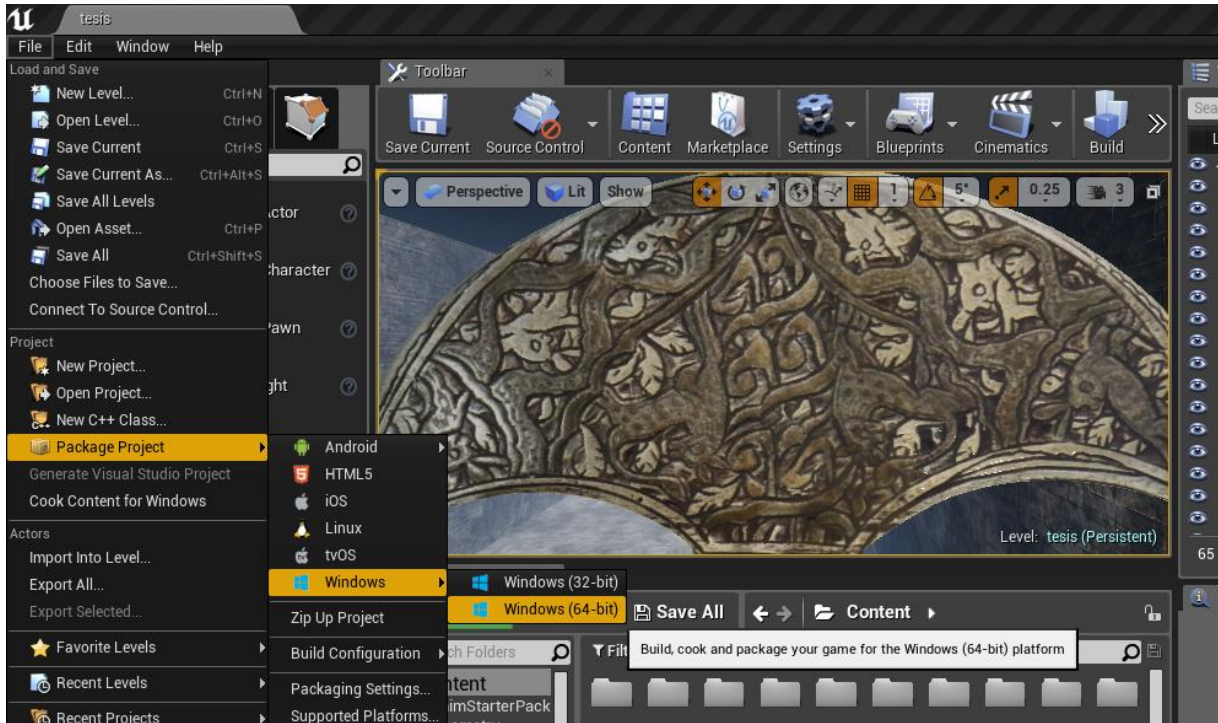
(Villasmil, 2018)

Después está *Input* el cual es usado para crear funciones, las cuales contendrán las diferentes entradas de valores como botones de teclado, mandos de consolas, gestos de movimiento. En este caso es posible crear funciones que contengan una o más entradas de valores, para ser usadas una vez sea empaquetado para múltiples plataformas o en el caso de la computadora, cambiar fácilmente del teclado a un mando de Microsoft una vez sea conectado por USB o conexión inalámbrica. Para crearlos y anexar los valores de entrada solo se debe hacer *click* en el botón con el símbolo “+” y dependiendo de la ubicación de jerarquía, se crea la función o los valores.



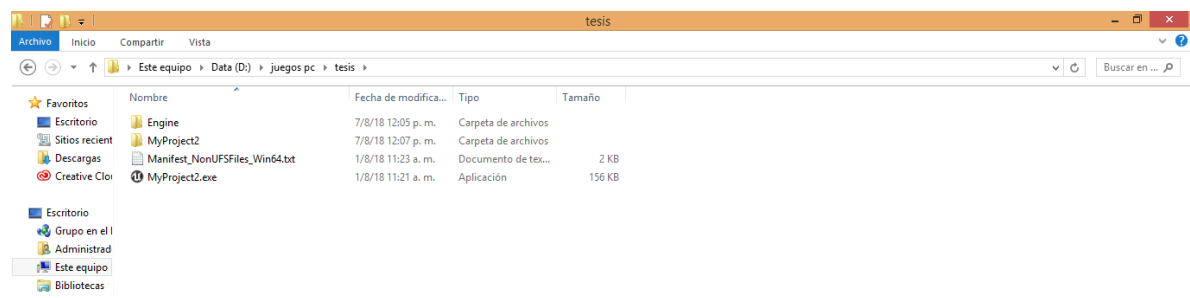
(Villasmil, 2018)

Una vez se completan las configuraciones, en la interfaz principal mientras se visualiza el nivel, se hace *click* en *File* y en la opción de *Package Project* se selecciona la plataforma deseada en la cual construir el proyecto.



(Villasmil, 2018)

Ya, a partir de este punto, el programa automáticamente procesará los elementos dentro del proyecto y generará en el transcurso del procesamiento el ejecutable en la ubicación del proyecto desarrollado; también se puede cambiar a otra ubicación en la computadora a través de una ventana de diálogo que usa el explorador de archivos. A partir del icono con el nombre del proyecto comienza a correr el ejecutable para la plataforma seleccionada; en este caso Windows.



(Villasmil, 2018)

RECOMENDACIONES

Se pueden realizar pruebas en dispositivos Windows que usen un mando de Microsoft (Xbox 360 o Xbox One). Por posibles inconvenientes con el precio, una alternativa mucho más económica y práctica es la adquisición de un mando genérico USB cualquiera y el programa “x360ce” el cual permite leer las entradas de dicho mando y las convierte en entradas de control de “Xbox 360” logrando la calidad de experiencia sin ningún inconveniente. Se debe mover el archivo dentro de la carpeta donde está el ejecutable del proyecto; dependiendo de la versión, la carpeta puede llamarse “Win64” o “Win32”.

Se recomienda estar al tanto de las actualizaciones del programa por las creaciones y ediciones de las funciones o configuraciones del mismo programa, que pudieran agilizar el desarrollo de proyectos o nuevas formas de desarrollo.

Al momento de empaquetar el proyecto a una plataforma específica, tener previo a este paso todas las librerías de desarrollador necesarias junto, con la documentación de las configuraciones para su correcto funcionamiento; también corregir cualquier error de contenido o programación que pueda interrumpir la generación del ejecutable.

Existe un límite de doscientos cincuenta y seis (256) caracteres para los nombres y ubicaciones de archivos al momento de procesar y generar el ejecutable; se recomienda usar nombres cortos para el contenido identificado, también posicionar la carpeta del proyecto en la ubicación básica del disco duro (Disco Local C(:)) para evitar un nombre que supere el límite de caracteres por la adición de las carpetas y subcarpetas que la conforman)

CONCLUSIONES

Lo que antes era un proyecto personal de estudiantes del MIT, ha llegado a ser hasta la fecha una industria con alcance internacional, con amplias posibilidades en varios ámbitos, no solo en el entretenimiento, también en la educación, simuladores y más.

Con la llegada de los motores se ha agilizado su desarrollo, también se amplió la gama de desarrolladores que con el paso del tiempo han evolucionado hasta los vigentes actualmente; con el cambio de Unreal Engine 4, que ofrece todas las funciones gratuitamente para el desarrollo personal o educativo y en el caso de crear por motivos comerciales solo toma el cinco por ciento de las ganancias cuatrimestrales, logrando un alto uso del programa e inclusive la aparición de nuevos desarrolladores y estudios, creando nuevo contenido interactivo que abarca no solo el entretenimiento, sino también los simuladores, animaciones, etc.

Las posibilidades de uso están limitadas solo por la imaginación de los desarrolladores y su conocimiento del programa; con el desarrollo del manual sobre este programa a disposición de los estudiantes de la Universidad Latina de Panamá se espera impulsar el desarrollo de diversos proyectos que puedan tener un impacto positivo y contribuir a la sociedad dentro y fuera de la universidad. Inclusive lograr el desarrollo de jóvenes con las herramientas necesarias para contribuir y hasta trabajar en una industria que compite con la televisión, el cine y hasta la música en todo el mundo.

BIBLIOGRAFÍA

- Bolívar, U. S. (12 de Diciembre de 2017). *Grupo de Desarrollo de Experiencias Lúdicas (DELU)*. Obtenido de Grupo de Desarrollo de Experiencias Lúdicas (DELU): <https://grupodelu.files.wordpress.com/2015/10/glosario-de-terminos-de-videojuegos.pdf>
- Canales, J. M. (17 de Diciembre de 2017). Desarrollo de un Videojuego en Unreal Engine 4. Alicante, Alicante, España. Obtenido de https://rua.ua.es/dspace/bitstream/10045/49409/1/Desarrollo_de_un_videojuego_con_Unreal_Engine_4_EGEA_CANALES_JOSE_MARIA.pdf
- Festini Wendorff, M. A., & Torres Ferreyros, C. M. (1 de Octubre de 2017). *Desarrollo de un videojuego con Unreal Engine*. Obtenido de REPOSITORIO ACADÉMICO UPC: https://repositorioacademico.upc.edu.pe/bitstream/handle/10757/622446/Festini_ma.pdf?sequence=5
- Games, E. (11 de Julio de 2014). *Level Editor*. Obtenido de <https://docs.unrealengine.com/en-us/Engine/UI/LevelEditor>
- López, A. G. (17 de Diciembre de 2017). Desarrollo de un videojuego Flame Knights chronicles. Jaén, Jaén, España.
- Playground, G. L. (07 de Febrero de 2018). *Gamer's Little Playground*. Obtenido de Gamer's Little Playground: https://www.youtube.com/watch?v=4KKRVvS_dUA&t=188s
- researchgate.net*. (08 de Febrero de 2008). Obtenido de Ejemplo de nodos y aristas en una red de co-citación y similitud

Sweeney, T. (2 de Marzo de 2015). Declaración Tim Sweeney sobre cambio de políticas de Unreal Engine 4.

Value Coders. (21 de Octubre de 2011). Obtenido de Comparación Unreal Engine vs Unity: <https://www.valuecoders.com/blog/technology-and-apps/unreal-engine-vs-unity-3d-games-development/>

Villasmil, R. (01 de 05 de 2018). Manual de Unreal Engine 4. Panamá, Panamá, Panamá.